

Perudo Webgame - Alexander Royston

May 2025

Author: Alexander Royston
Candidate number: 9330
Centre name: Magdalen College School Oxford
Centre number: 62323
Qualification code: 7517D
Date: May 2025

1 Analysis

1.1 Background Information

I have decided to produce a web-based implementation of a popular dice board game, Perudo. There are few existing digital Perudo programs and those that do exist have an insufficient player count to function, lacking bots to compensate for this issue. Furthermore, these implementations lack rating systems and tournaments to facilitate competitive Perudo gameplay. To gain insight into this problem, I interviewed Mr Konnar Lambert-Smith, an avid Perudo player and maths teacher, who often plays Perudo with friends and colleagues. Countless students and teachers alike who enjoy playing board games, such as Perudo, would benefit enormously from a convenient and portable Perudo game, encompassing all of the previously mentioned features. This project would be particularly beneficial to players struggling to find others to play with or yearning to play competitively, in tournament formats.

1.2 Key-User Interview

What methods do you currently use to play Perudo?

I currently, almost exclusively, use table-top with Perudo dice, although the recently introduced web-based Perudo was quite an interesting concept; I played it with some of the teachers here.

Are there any problems with the current way of playing Perudo?

I love the game; I don't think there are any structural problems. I suppose a benefit of a web-based Perudo game is that it is quite difficult to cheat off other people. With having your dice and looking at the cup, it's very easy to accidentally peak or intentionally peak at one person's dice and get a feeling of it. So it would mainly be the possibility of cheating.

What core functions would a Perudo program need to include?

Starting from the main menu, I think a rules menu would be nice. A randomiser for the dice. The ability to play with any number of people. A nice graphical interface to show your five dice on the table, as opposed to numbers on a screen. The ability for the game to take away dice. The ability to say two numbers including the number of the dice and face value. The ability to raise numbers according to the rules. The ability to call out liars. A method of removing a person who has been eliminated from the game, allowing them to spectate.

How would you like matchmaking to be implemented?

Typically it's just four or five people who will come together to play the game, which is easy to do if you are all in the room together. If not, and you are alone, it might be plausible to incorporate some kind of online matchmaking. That would benefit people without others to play with, but otherwise, the initial method seems perfectly fine. It would be nice to have a kind of waiting room where you can start a game and, as you wait for players, have a chat room involved where you can talk.

Would you prefer the system to be a website, desktop app or mobile app?

I think there is an untapped market for mobile phones. Playing on a desktop works if you are at home or at work, but if you have got a firmly rooted place to play on desktop, you are likely to have physical dice and cups anyway. However, if you wanted to play it on the go, you would not be able to play it on the train, so having it on your mobile phone would be really good. All options would be good, but I would definitely prefer the mobile implementation.

What type of players would benefit most from this system?

Anybody who wants to practice and play on the go.

How would you like for the program to be laid out and structured?

I would like to be able to simply enter any username, select an avatar and join a game, similar to a game like Scribble.io.

Are there any additional features you would like to see implemented?

In standard perudo, you are given a strict number of dice, with which you can play. I think it would be quite cool to be able to choose the number of dice, which is limited by the number of dice you have on hand when playing physically.

1.3 Key User Interview Analysis

Throughout the interview, Mr Lambert-Smith identified cheating and portability as some of the major issues with the current physical system, which could both be solved by either a mobile or web game Perudo implementation. Of these, he expressed a preference for a mobile phone implementation, however, he said a web game would also be suitable, especially given several practical issues with making a mobile implementation.

In terms of core functionality, Mr Lambert-Smith stressed all of the standard functions of any working Perudo game, along with the ability for eliminated players to spectate the game and the presence of a rules menu. He also mentioned an additional feature, allowing users to play with any number of dice, which would usually be restricted by the physical availability of dice in a set. Upon entering the game, Mr Lambert-Smith said he would like to simply choose a username and avatar before joining a game/waiting room. I understand the convenience and ease this might provide, however, this would limit the ability of players to view past games or store game history, which could be a valuable feature to certain players, especially intermediate or competitive players.

1.4 End-user Questionnaire

I reached out to fellow students who also enjoy Perudo or similar board games, to gain some more insight into their experience of the game and any ideas they might have regarding the project.

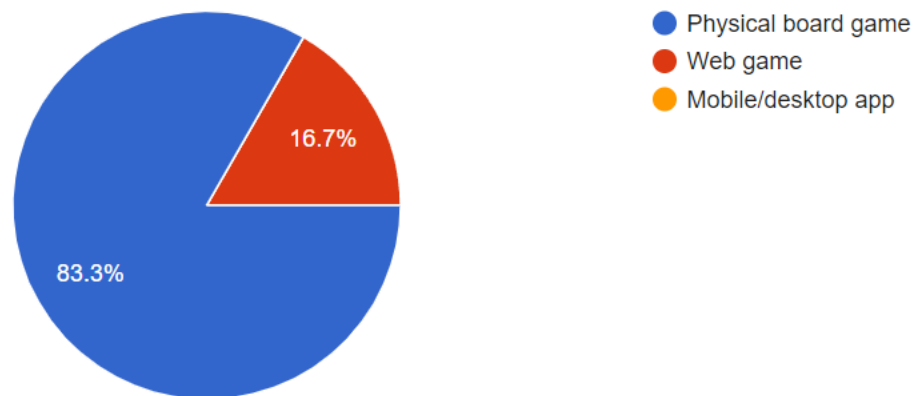


Figure 1: How do you most often play Perudo or similar board games?

My first question, seen in Figure 1 above, examined the various methods by which users most often play Perudo. 80% of the respondents said they most often played Perudo physically, suggesting current digital implementations are either unpopular or inaccessible.

On a scale from 1 to 10, how beneficial would a digital Perudo implementation be to you?

6 responses

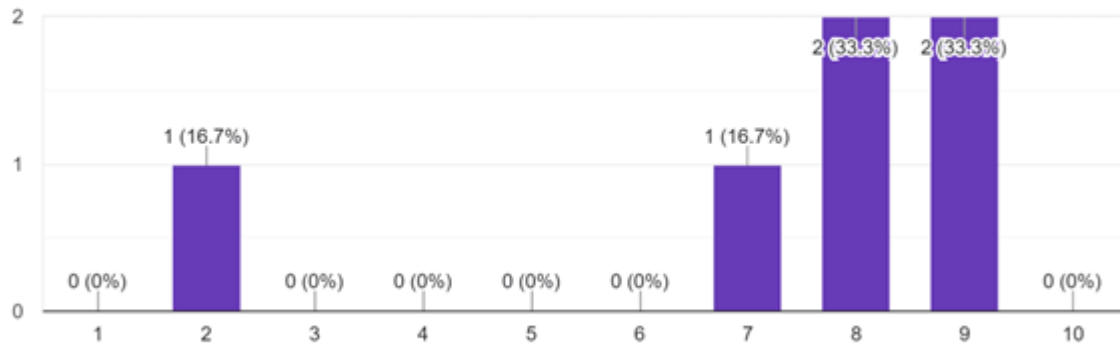


Figure 2:

How easy do you think it is to find other people to play Perudo with?

6 responses

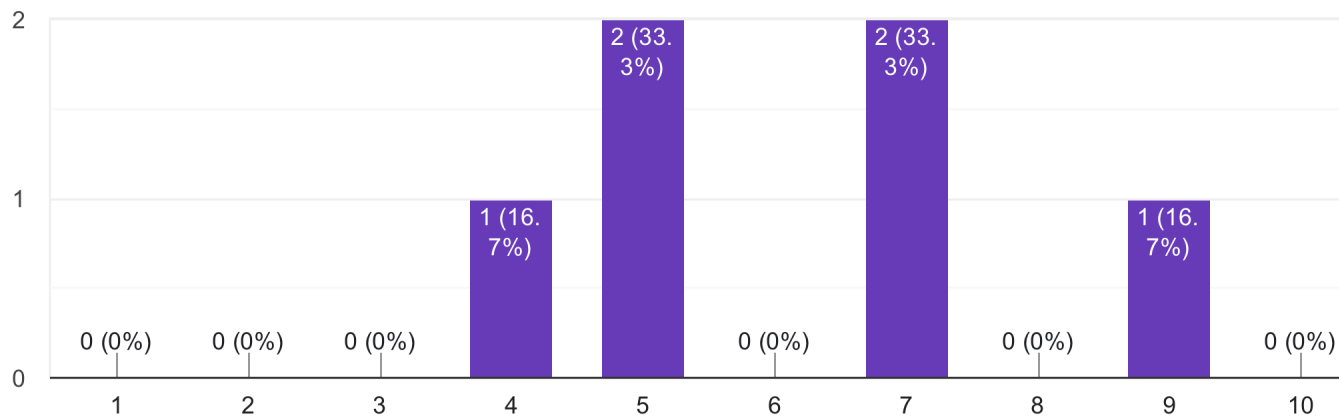


Figure 3:

This suggestion is further supported by the answers to my second question (figure 2). Here the average value of the benefit of a digital implementation was 7.2, providing more evidence for the additional convenience and benefit of a Perudo program. Upon being later asked whether or not they struggled to find people to play with (figure 3), the average response was 6.2, indicating this might be a significant issue a program might be able to reduce.

Users	Feedback
User 1	Ability to play the game Multiplayer capabilities, both pass and play and potentially networked on multiple systems Ability to create accounts and save scores Ability to run tournaments Potentially, the ability to make custom rules or change game parameters
User 2	Face cam, rating system
User 3	Ability to play against the computer to hone my abilities. Also, a wizard/new player guide would be helpful.
User 4	Multiplayer capabilities, chat interactions, rating system/leaderboard
User 5	Rolling dice, placing the bids, doubting the bids and taking away the dice
User 6	Play Perudo Support for multiple players Play against AI

Figure 4: What core features would you expect out of a Perudo Program?

When I asked my end-users about any core features they would like to see included, several responses mentioned the inclusion of accounts and saved scores/rating systems, urging me to incorporate an account database system, to store data and logins for different users. Furthermore, several users mentioned both multiplayer and bot functionality, stressing the importance of these two features.

On a scale from 1 to 10, how valuable would a bot be to your Perudo experience?

6 responses

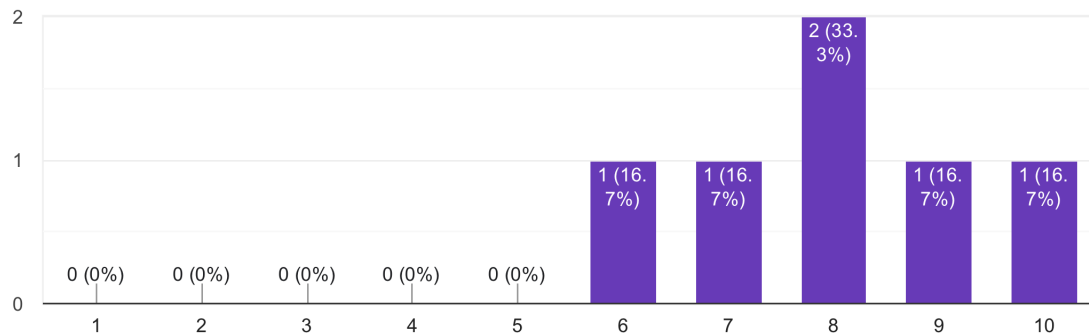


Figure 5:

Users	Feedback
User 1	Difficulty/skill differences Evaluation of moves
User 2	Difficulties, play styles
User 3	Different difficulty level: Beginner, Intermediate, Master, enabling me to level up as a player. Being able to name the bot player would be a nice touch.
User 4	Difficulty customisation
User 5	Have a bot that plays purely from statistics and have one which takes more risks

Figure 6: If a bot were to be implemented, what characteristics or customization options would you expect it to have?

Both figure 5 and 6, seem to further reinforce the potential benefit of a Perudo bot, along with customizable difficulty and play style options. A move evaluator, similar to that of several chess websites, was also an interesting suggestion, which could be particularly helpful to new players, as they attempt to improve their Perudo skills, through some sort of practice mode.

How would you prefer to access the program?

6 responses

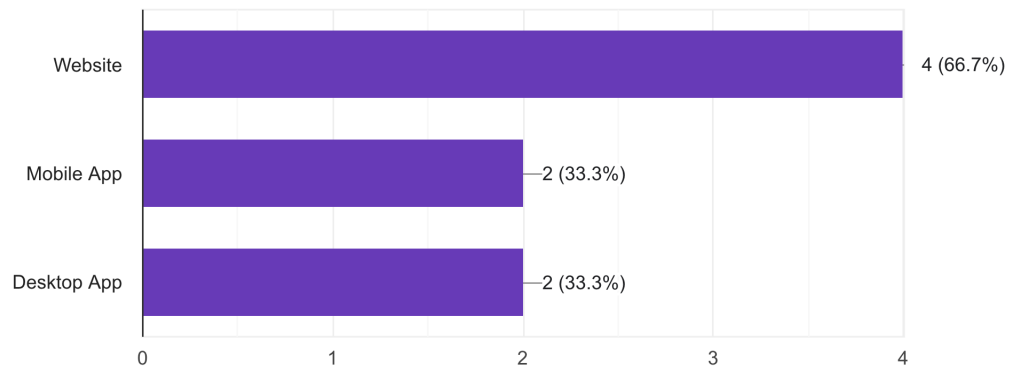


Figure 7:

In order to determine what platform would be most useful for a Perudo program, I asked my end-users how they would prefer to access the program (figure 7). Overall, a web game, mobile app and desktop app all seemed pretty evenly preferred, however, the website implementation seemed to be slightly more popular. ‘

1.5 Overview of the problem

Current systems for playing Perudo include many issues highlighted by my research, by both key users and end-users, including the lack of portability, bots to play against and rating/competitive systems. Furthermore, several people raised the expensive membership requirements, along with difficulty finding other players online, as major issues with existing Perudo programs. These issues, however, could be solved by a Perudo program, with bots, some sort of competitive features, such as a leaderboard and user accounts. This project aims to achieve all of these outlined features through a Perudo web game, allowing for portable and convenient gameplay for a range of potential demographics, including competitive players looking for a challenge, or casual users looking to play with friends on the go.

1.6 Modelling the problem

1.6.1 Input Process Storage Output

IPSO	Program Section	Details
Input	Player Actions	Place_bid Call_liar
	Game Information	GameMode Number of Players Number of Bots Bot Difficulty
	User Information	Username Password
Processing	Game	Roll Dice Check Round Over Check Round Winner Update Player Statistics Evaluate Liar Call Check Game Over Take Away Dice Update Bids
	Registration	Password Hashing Login Authentication Email Verification
Storage	Player Data	Username Password Games Wins
	Game State	Game settings Game status/information Game progress
Output	Display	Game GUI Settings GUI
	Leaderboard	Personal Player Statistics Global Player Statistics
	View Profile	Email Username Password
	Game Data	Bid Option List Player Dice Opponents' No. Dice Round Loser Game Winner

Figure 8:

This IPSO (Input Process Storage Output) diagram demonstrates a general overview of the flow of information and operations within a potential Perudo system.

1.6.2 User ERD

For a proposed solution, a database will be needed to store data about users, matches and types of Perudo game.

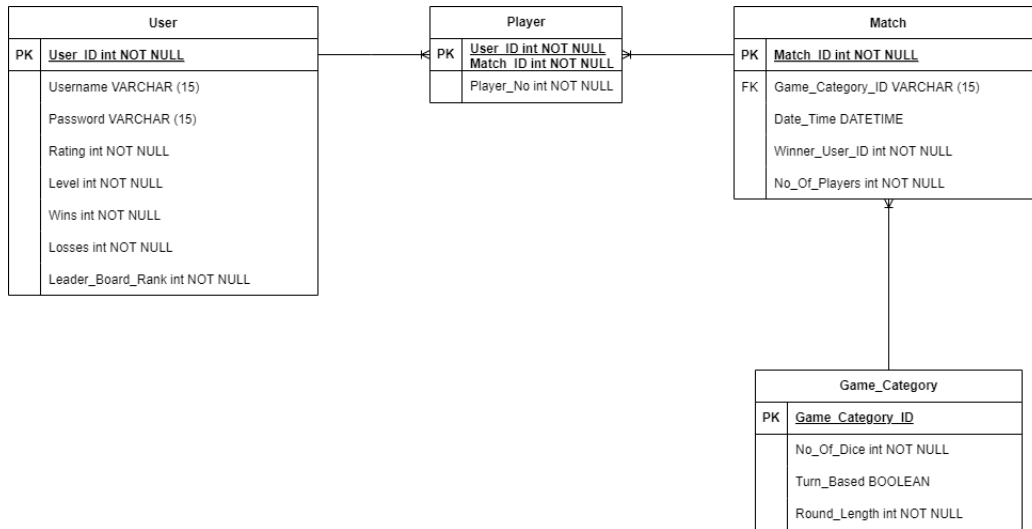


Figure 9:

As seen in the Entity relationship in figure 12, the system will require a User, Player, Match and Game Category class. The User class will store profile data such as their username and password, rating and level, and game statistics, using the field 'User_ID' as a primary key. The player class is a linking table between the User and Match tables, with User ID and Match ID as a composite primary key. The match class stores information about an individual game and, therefore, has a one-to-many relationship with players, as there are multiple players in a single game. Finally, the Game Category table stores the settings and details of different types of games. One match has one game category, whereas one game category accounts for many matches and, therefore, this table has a one-to-many relationship with the Match table.

1.6.3 Perudo game flow chart

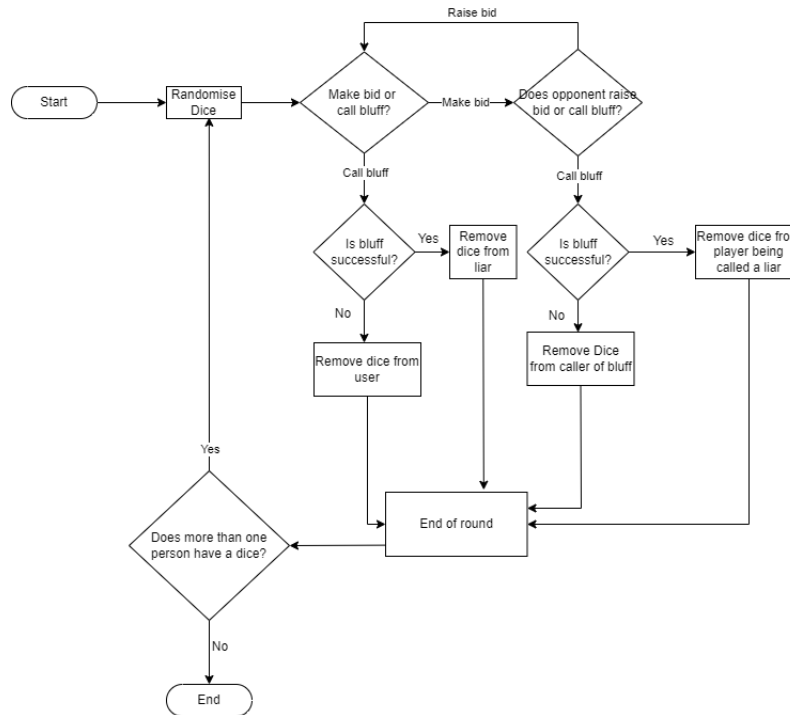


Figure 10:

To demonstrate the structure and game logic of a Perudo match, I made a flow chart (as seen in figure 13) to outline the core functionality required to play the general game. To make this work, a list of players must be stored, along with previous bids, to determine who to take a dice from in case of a successful or failed bluff.

1.7 Analysis of Similar Systems

1.7.1 System 1

The first existing system I would like to analyse is arguably the most popular system, the physical Perudo board game. Over 80% of my end-users said they most often played Perudo physically, with my key user citing its physical convenience and structural functionality as a major reason for its popularity. There are many aspects of this physical system that I would like to implement in my Perudo program, including the foundational functionality of the game, as well as the visually appealing and convenient nature of its gameplay. However, I aim to include several features not achievable by a physical board game, which I deem to be transformative to user gameplay, convenience and overall enjoyment.



Figure 11:

The first positive aspect of the Perudo board game that I aim to replicate is its visually appealing appearance. Instead of simply presenting the game as a series of numbers and words on a screen, I would like to maintain some semblance of the physical game's appearance, through colourful animations of the cups and dice. Furthermore, I also aim to incorporate pass-and-play functionality, allowing multiple users who are together physically to play, using the same system. This is a major advantage of the physical game, which I think can only be supplemented this way. The only issue with this is the prevention of cheating, however, this could be resolved by a screen hiding function within the game, allowing users to conceal their dice. The final feature of the physical game that I aim to maintain and enhance in my implementation, is the rules guide. This feature is particularly important for new and beginner players, so I aim to provide a tutorial video and rules guide within my program to help this group of players, potentially even including a video tutorial to better help players trying to learn.

Despite, these positive aspects of the physical game, there are also many shortcomings and flaws with the physical implementation of this game that I plan on addressing through a variety of features. The first, and arguably most significant issue with this system, is the complete reliance on players in near proximity to function. If an individual only wants to play Perudo with family around the house or at a friend's house, this is not all that much of an issue. However, if one is looking to play with other people in separate locations or if they do not have anyone to play with at all, this becomes a critical issue, preventing them from playing Perudo entirely. I aim to resolve this potentially game-ruining flaw, by introducing bots to play against. According to my end-user questionnaire, this issue is particularly important, with many end-users finding it difficult to find others to play with, as well as seeing the benefit of a bot to play against.

Another major aspect of user experience that the physical game lacks is progression and competitive gameplay. Although players could theoretically devise their own tournaments and even rating systems, this requires a lot of effort and time, two things that I doubt the average player is willing to invest in. However, through a digital implementation such as mine, I aim to implement a ranking system or tournament system, allowing for convenient and effortless competitive gameplay to counteract this issue.

1.7.2 System 2

I came across a web game used to play Perudo, on the website BoardGameArena, which had many features which I enjoyed and would like to implement in my own system. However, there were several flaws and issues that immensely hindered my Perudo experience, which I believe I could improve significantly within my own program.

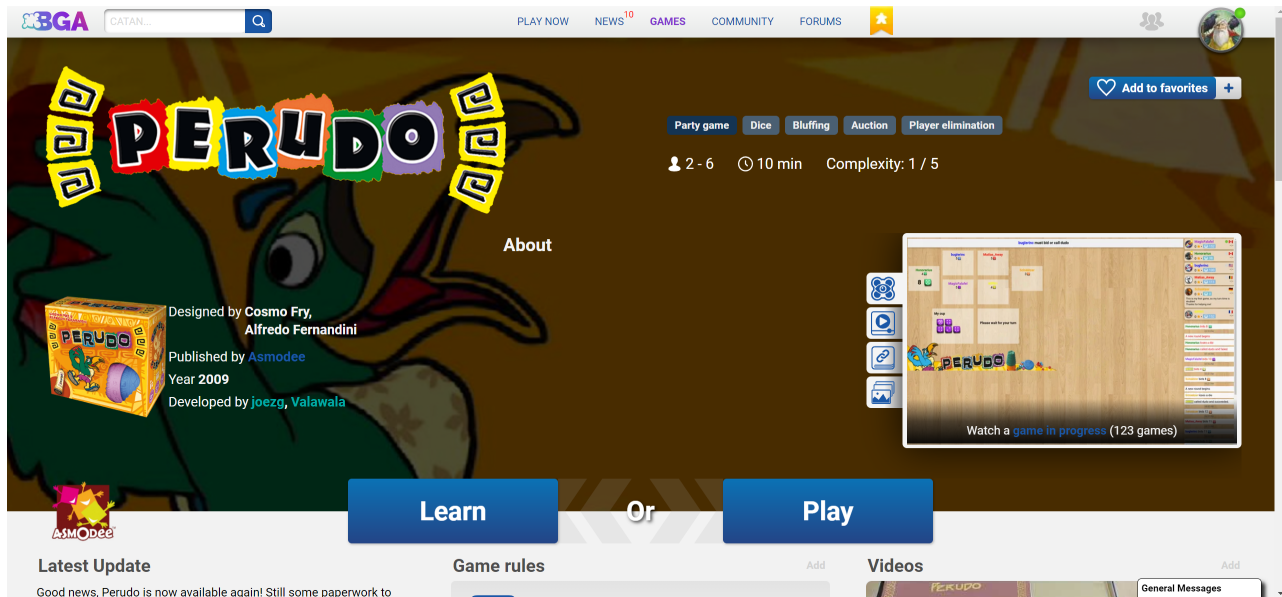


Figure 12:

Upon entering the web game, a user has the choice of either reading through the game rules or watching a tutorial by clicking 'Learn' or heading straight into a game through the 'Play' button. I think the inclusion of rule learning resources is a vital part of a beginner-friendly Perudo program, so I am eager to implement this into my program.

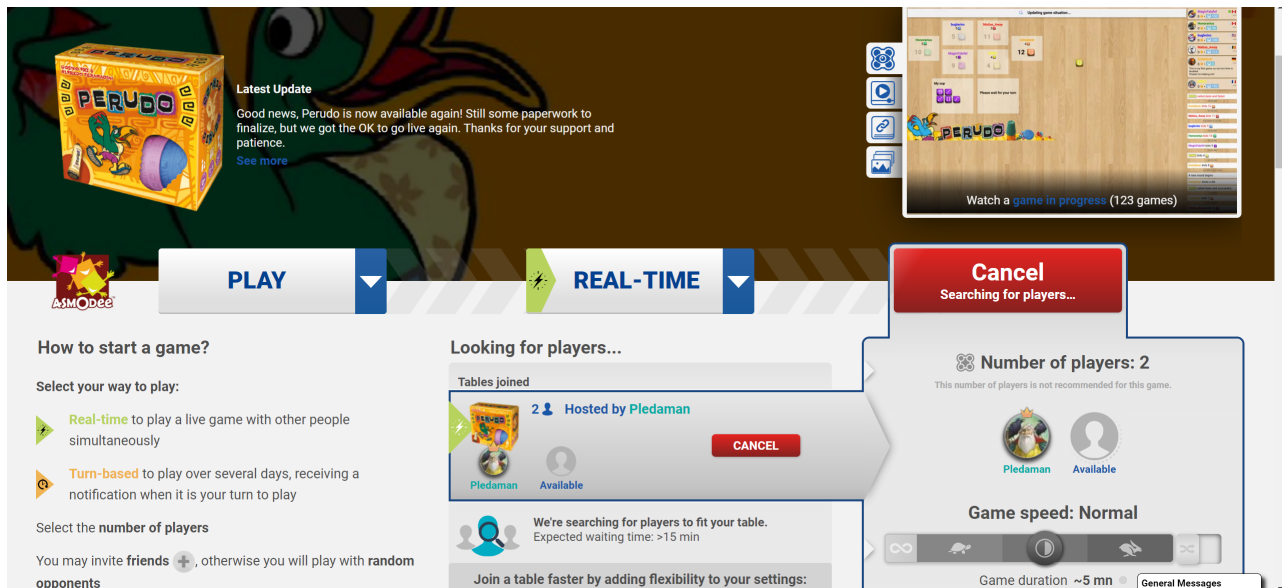
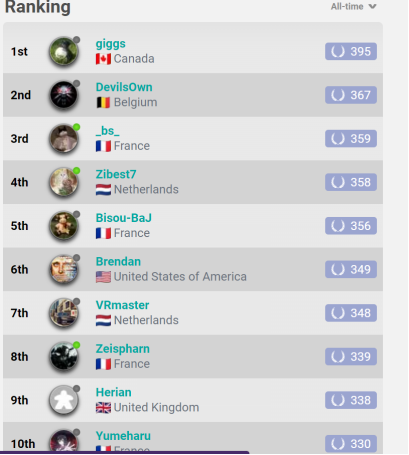


Figure 13:

However, upon starting a game, I was completely unable to find other players to play with (as seen in figure 9), despite retrying this at various times over multiple days of the week. Despite this undermining flaw of the game, there are no bots to play against, making this game, for anyone alone, very difficult to enjoy. I believe the use of bots to compensate for this issue to be an essential feature of a Perudo program, so I aim to include customizable bots in my implementation.



Ranking		All-time ▾
1st	 gigge Canada	395
2nd	 DevilsOwn Belgium	367
3rd	 _bs_ France	359
4th	 Zibest7 Netherlands	358
5th	 Bisou-BaJ France	356
6th	 Brendan United States of America	349
7th	 VRmaster Netherlands	348
8th	 Zeispharn France	339
9th	 Herian United Kingdom	338
10th	 Yumeharu France	330

Figure 14:

Despite this issue with finding other players, the system seemed to have an elo and ranking system, allowing for players to compete against each other and fight for a place at the top of the table (as seen in figure 10). Furthermore, this involved saving results and elo lost/gained of individual users, allowing for progression within the game. From speaking to my key-user and end users, this aspect of the game appears to be very important, convincing me to employ some of these features. However, this system did lack the ability to run tournaments, which, in my opinion, could facilitate Perudo competition even more.

1.8 Objectives

1. User Management		
Number	Objective	Core or additional feature?
1.1	User given the option of logging in or registering.	Core
1.2	Allow the user to enter their username and password, if they choose to log in.	Core
1.3	If the user fails to enter the correct details, display an error message, asking them to try again.	Core
1.4	Allowing for user to enter and verify email, before choosing username and password, if they chose to register.	Core
1.5	Require email authentication when registering, by sending the user an email to authenticate their email.	Additional
1.6	Use a simple hashing algorithm to hash passwords for secure storage.	Additional
2. Main Menu		-
Number	Objective	-
2.1	User given the option of 'view profile', 'game rules and tutorial', 'view leaderboard', 'pass-and-play' and 'bot game'.	Core
2.2	If the user selects 'view profile', display their username, statistics and other information.	Core
2.3	If the user selects 'game rules and tutorial', display rules and allow them to watch a tutorial video.	Additional
2.4	If the user selects 'view leaderboard', display the leaderboard with their name and stats	Additional
2.5	The view profile page should provide the option of changing the user's username or password.	Additional
3. New Game		-
Number	Objective	-
3.1	Give User choice of playing 'pass and play' or 'against bots'.	Core
3.2	Present user with the choice of game settings.	Core
3.3	After user has chosen settings, open a match instance with the selected settings.	Core
3.4	Allow the user to customise their own in-game name and other player's names.	Additional
4. Play Game		-
Number	Objective	-
4.1	Randomise dice and display them to the User.	Core
4.2	Allow user to make a bid or call the previous player's bluff.	Core
4.3	Only display bids which follow the rules of the game.	Core
4.4	If the User calls another player's bluff successfully, takeaway that player's dice. Otherwise, take away one of the User's dice.	Core
4.5	Eliminate players if they run out of dice.	Core
4.6	End game if all but one of the players are eliminated, displaying the winner of the game.	Core
4.7	Once a game has ended, the user's games played and wins should be updated appropriately	Core
4.8	Display a nice graphical interface, showing dice on a table.	Additional
5. Bot Players		-
Number	Objective	-
5.1	Allow the user to choose the number of bots and initial dice.	Core
5.2	Ensure the bid system works just as in pass-and-play, only allowing bids which follow the Perudo rules.	Core
5.3	Ensure the bluff system works properly, with dice being removed from the correct player/bot, players/bots being eliminated if out of dice and game ending successfully if only one player/bot has dice.	Core
5.4	Allow the user to select a bot difficulty.	Additional
6. Game Settings		-
Number	Objective	-
6.1	Enable User to choose number of dice to be used.	Core
6.2	Enable User to choose maximum number of players.	Core
6.3	Enable User to choose the number of dice lost if a round is lost.	Additional

My objectives for User Management were chosen based on several pre-existing solutions and end-user feedback, both of which reinforced the benefit of establishing user accounts, to store profile data, statistics and rating. Several of my end-users stressed the importance of rating systems, as well as customizable user profiles within my end-user questionnaire, convincing me to include these kinds of requirements within my user management objectives. Furthermore, I do think properly hashing user information, such as passwords is an important feature to implement to maintain account security.

The objectives for this project's main menu were largely informed by both my end-user questionnaire and key-user interview. For example, I chose to include a tutorial option in my objectives since a 'rules menu' (mentioned in the key user interview) and 'new player guide' (mentioned in the end-user questionnaire) were mentioned as important features of a Perudo program. Furthermore, this would enable beginners to easily and conveniently learn how to play the game, whilst not requiring enormous computational depth to provide. I also chose to include a player leaderboard, having seen the several requests for a competitive rating system and leaderboard as additional features in my end-user questionnaire.

Many of the objectives under the 'Play Game' subheading form the structural foundation for the Perudo game, and were, thus, essential to include in any Perudo program. Furthermore, according to my key-user interview, a nice graphical interface would be much preferred over simple numbers on a screen and this objective would be both achievable and significantly enhance the user's gameplay.

When I explored pre-existing solutions, one of the major shortcomings of all of these systems was the difficulty with which I could find someone to play against. This major issue can be partially solved by the inclusion of a bot player, which could enable anyone, despite being alone, to play Perudo and enjoy my program. That is why I thought it essential to include the 'bot player' objective subheading, along with customizable difficulty settings to further enhance a user's experience, as an additional objective. If a bot were either unbeatable or ridiculously easy to beat, it would not be long before playing against said bot would feel pointless and monotonous. However, including difficulty levels drastically reduces, if not eliminates this issue, persuading me to include it in my list of objectives. Despite this, I don't believe this feature is crucial to the bot game, as well as the bot's strategy is well tuned and competitive to play against.

Another immensely valuable feature of a Perudo program, highlighted by my key-user interview, was the customizability of settings and game rules. More specifically, Mr Lambert-Smith explicitly mentioned choosing the number of dice as an important feature within a Perudo program, which I decided to include in my list of game-setting objectives. I decided to also include several other features such as a randomised play order and a customizable number of dice lost, to increase the range of possibilities for custom games, allowing for a more varied and exciting user experience. Although I believe these to be fun and interesting features to include, they are by no means essential, simply providing a few extra tools to play around with.

2 Documented Design

2.1 Wire Frames

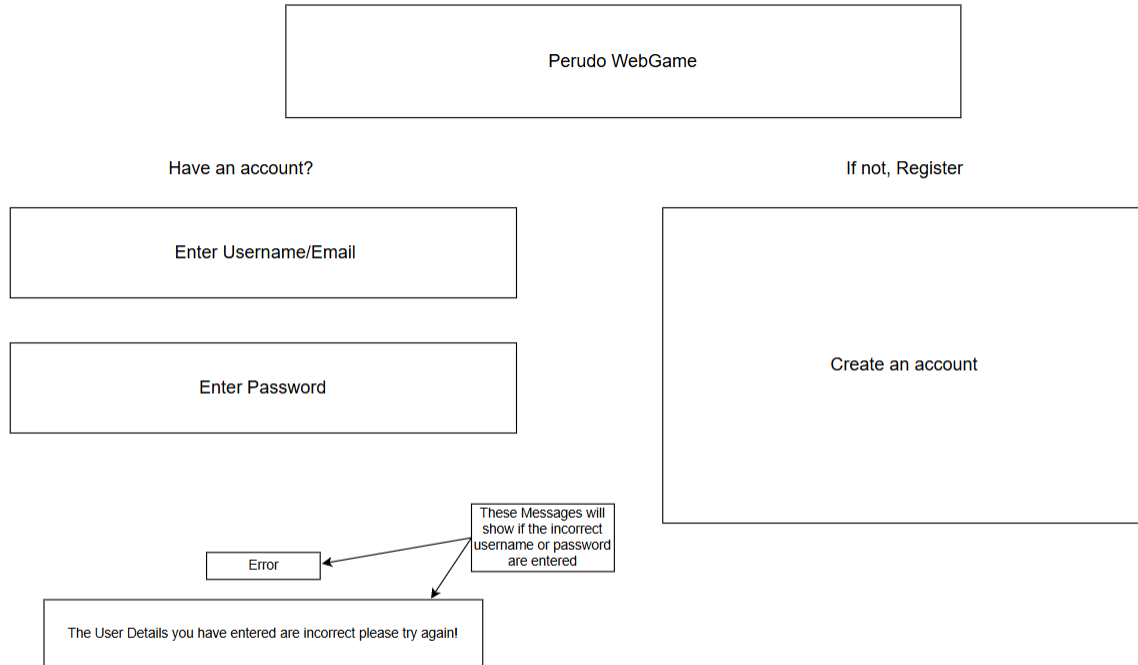


Figure 15: Introduction page

Upon entering the game, the user will be asked to create an account or enter their username and password to open their profile and begin playing. If they happen to enter the incorrect username or password, an error message will appear, as seen, asking them to try again.

	Cancel	Register
Email		
Username		
Password		
Confirm Password		

Figure 16: Registration Page

If the User does not already have an account, clicking 'Register' will open the above page, allowing them to enter their email, username and password, before confirming their password to create an account. The user will only be able to register fully if the email they have used is different from any previous email, in which case they will be redirected to the previous page to get them to enter their login details.

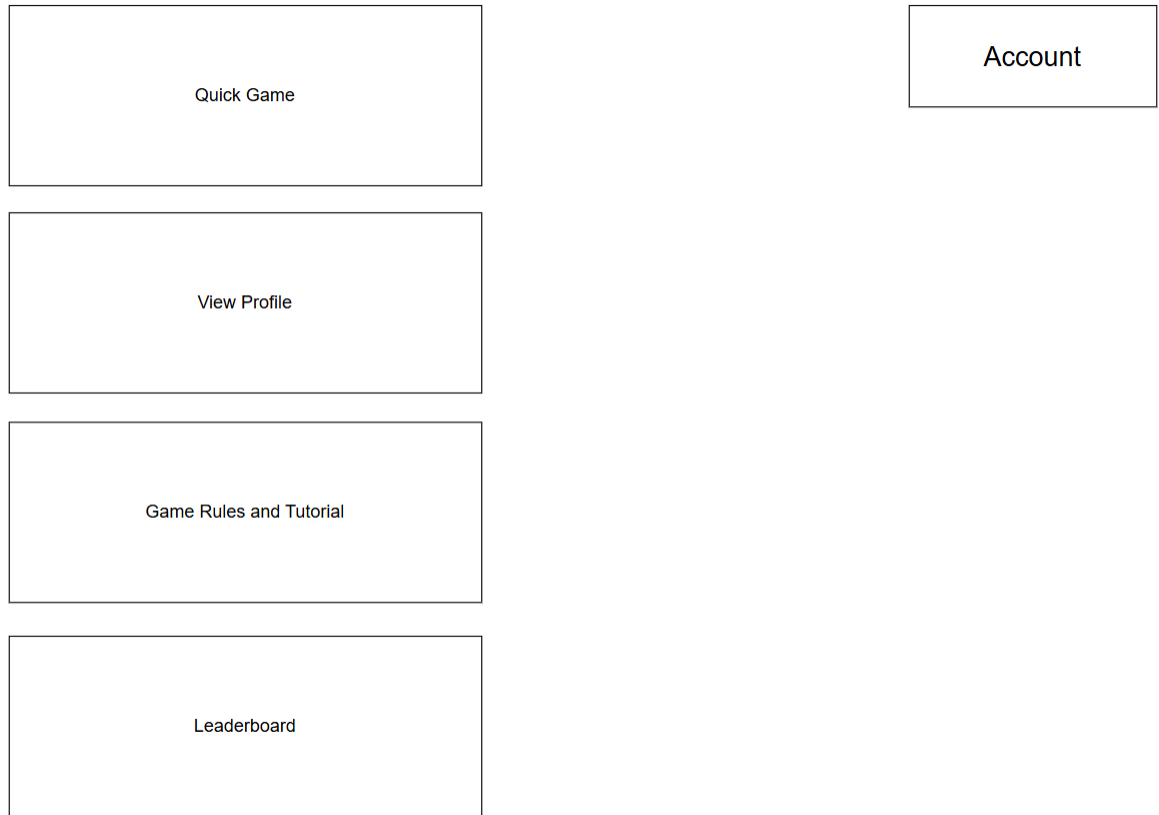


Figure 17: Main Menu Page

Having logged into their account, a user will be taken to the Main Menu, at which point they can either play a game or view their profile, the game rules and the global leaderboard.

Profile

Username: *****

Email: *****

Password: *****

To change the password enter and confirm your new password

Enter new password

Confirm new password

Change Password

Figure 18: Profile Info Page

If the user chooses to view their profile, they will be shown their information, and given the option to change their password, by entering and confirming their new password.

Game Rules and Tutorial

How to play:

Step-by-step rules

Figure 19: Rules/Tutorial Page

Instead, if they decide to select the Game Rules option from the main menu, they will be met with a list of rules, with a link to a video tutorial at the end.

Leaderboard

Username	Wins	Games	Win %
ExampleUserName1	563	1000	56.3%
ExampleUserName2	241	2000	12.1%

Sort By:

Wins☒

Games☐

Win %☐

Figure 20: LeaderBoard Page

By choosing to view the leaderboard from the main menu, the user will be able to view their individual game statistics, alongside others, including their number of wins, games and win percentage. The user will also be able to sort these players based on a chosen statistic.

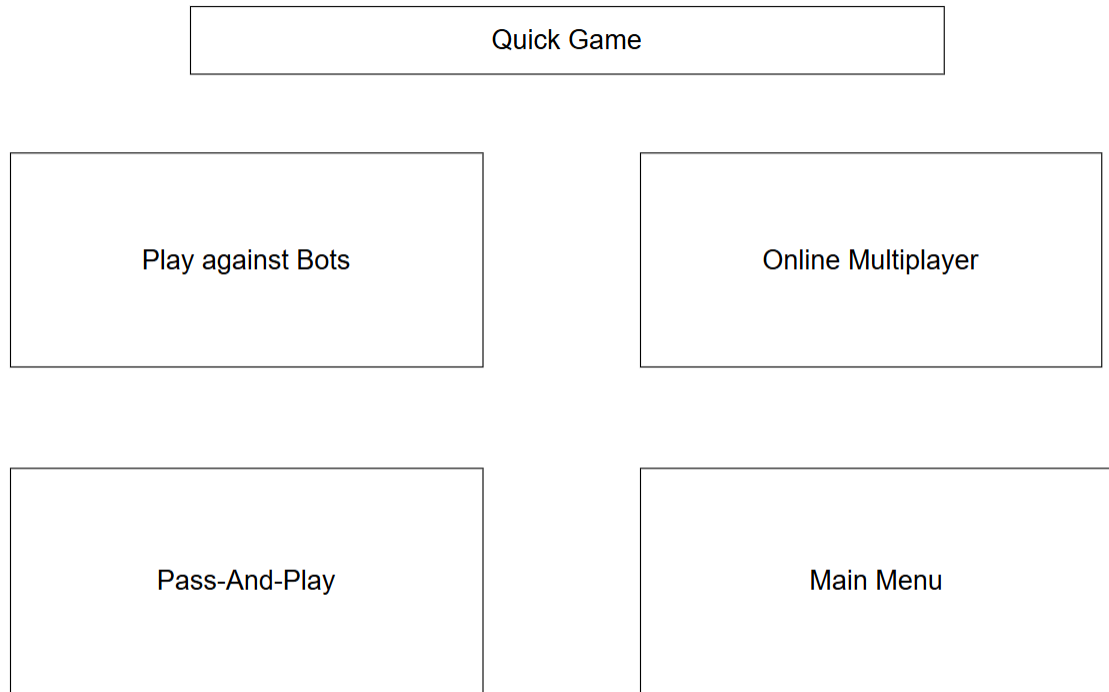


Figure 21: Game Options Page

To begin playing Perudo, players will be given the option of playing against bots, Pass-And-Play and on-line multiplayer, or returning to the main menu.

Pass and Play

Please Select Number of Initial Dice: ▼

Please Select Number of Players: ▼

Start Game

Figure 22: Pass and Play Settings Page

The Pass-and-Play settings page will allow users the choice of the number of starting dice, as well as the number of other players, before enabling a user to enter the game.

Bot Game

Please Select Number of Initial Dice: ▼

Please Select Number of Bots: ▼

Start Game

Figure 23: Bot Game Settings Page

Similarly, the Bot Game Settings will provide the choice of the number of initial dice and bots to play against.

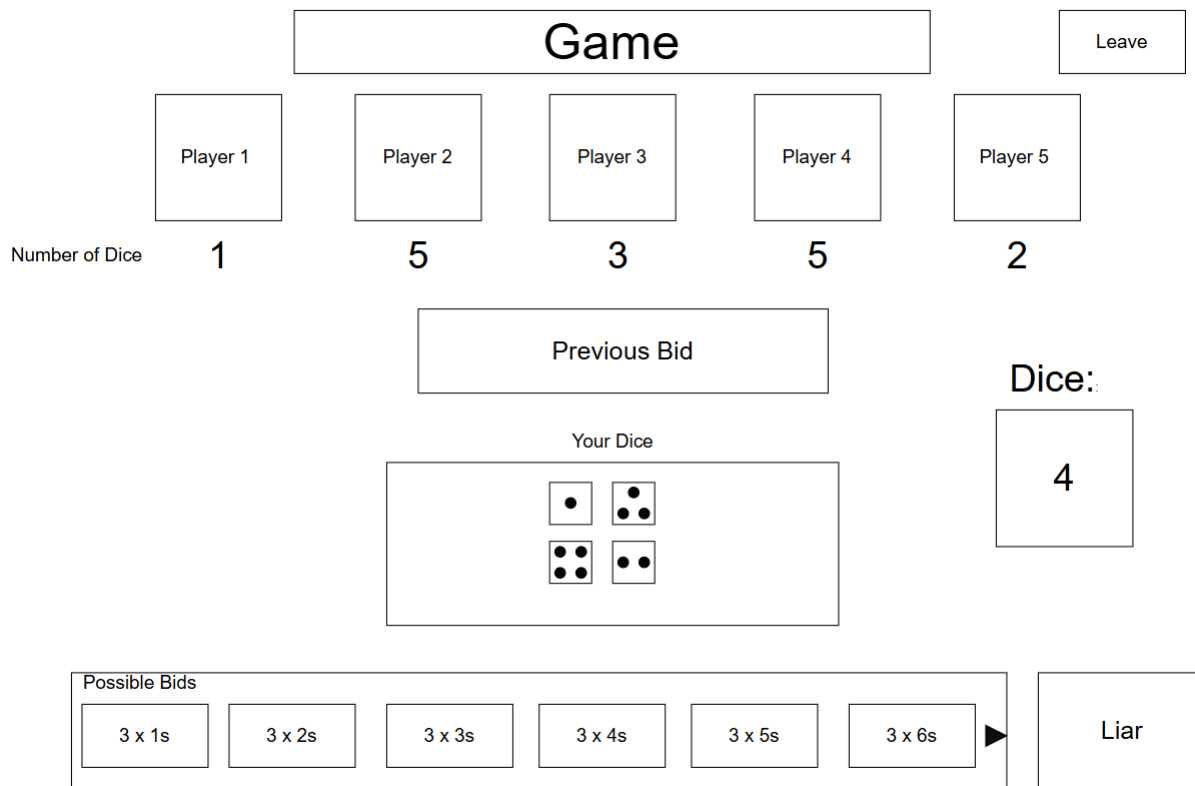


Figure 24: In-Game View

Once in the game, the user will be given a list of bids and a randomised set of dice to begin playing. They will also be able to see other players' or bots' names and number of dice, as well as their bids in the section labelled 'previous bid'.

2.2 Program Design

2.2.1 Top-Down Diagram

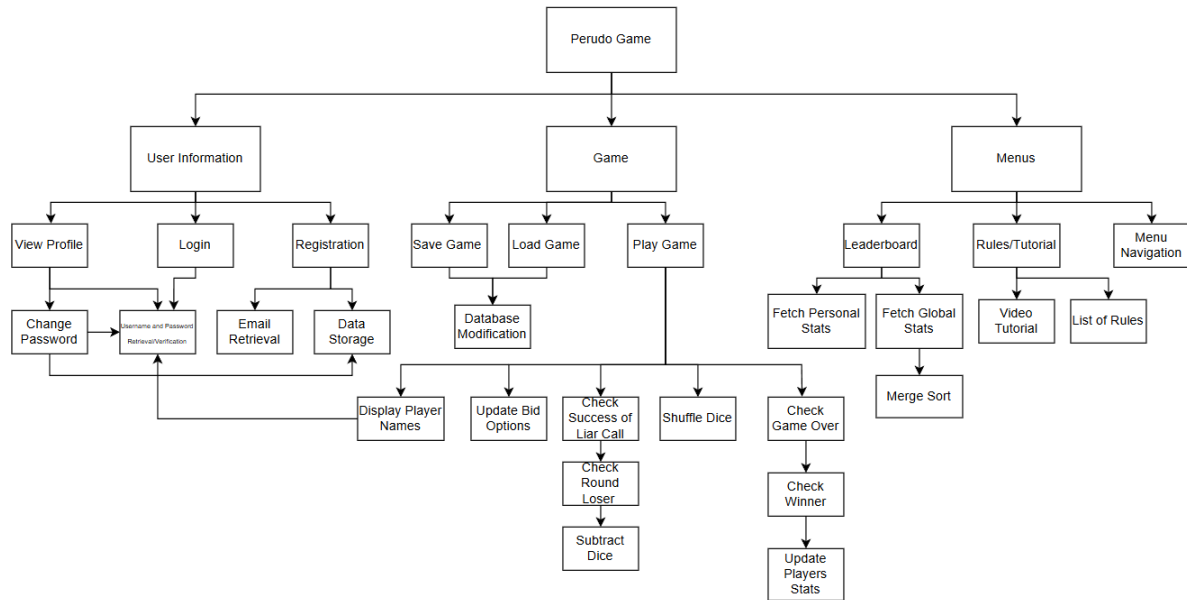


Figure 25: Top-down Diagram

This top-down diagram displays the various decomposed sections of this project.

2.2.2 Core Algorithms

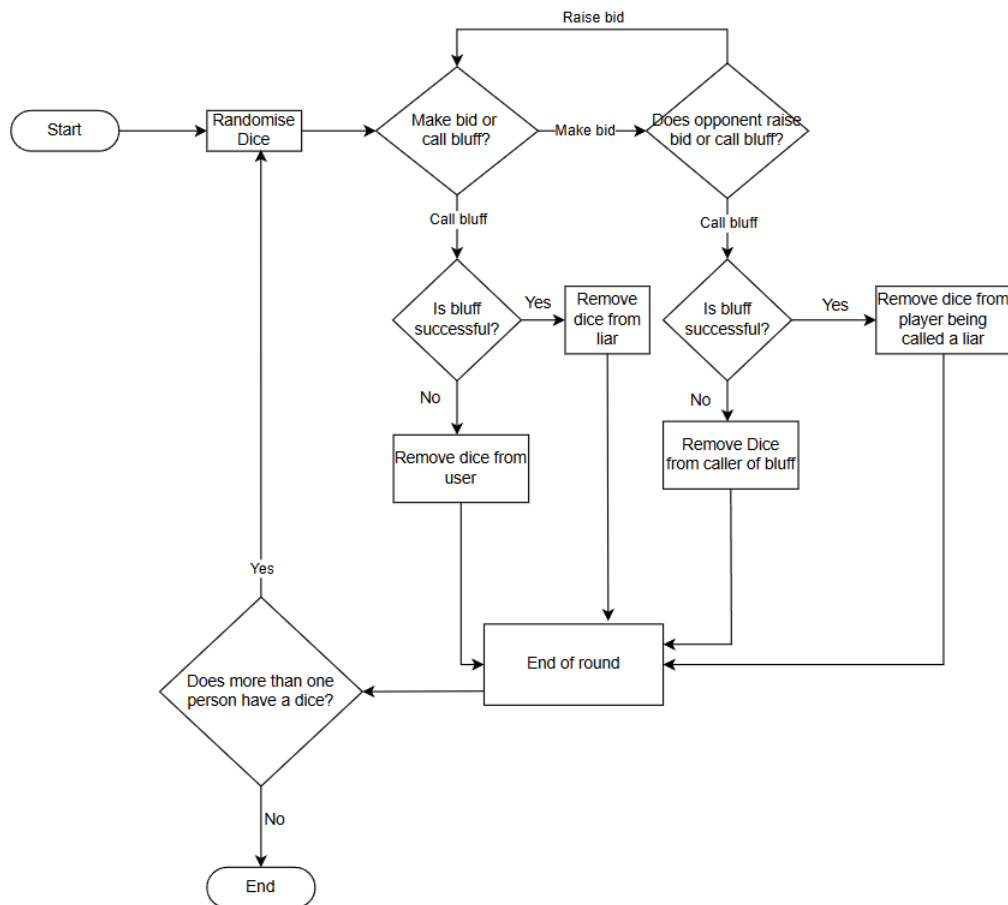


Figure 26: Game Flow Diagram

This flow chart represents a flow diagram of decisions and processes within a basic Perudo game against 1 player, which can be conveniently applied to a game of many other players.

```
#SUBROUTINE TO MERGE TWO SORTED LISTS
SUBROUTINE merge(array1, array2)
  # CREATING EMPTY LIST TO ADD SORTED ELEMENTS TO FROM EACH SORTED LIST
  merged_list ← []
  array1_index ← 0
  array2_index ← 0
  #MERGING EACH SORTED LIST
  WHILE array1_index < LEN(array1) and array2_index < LEN(array2)
    IF array1[array1_index] <= array2[array2_index] THEN
      APPEND(merged_list, array1[array1_index])
    ELSE
      APPEND(merged_list, array2[array2_index])
    ENDIF
  ENDWHILE
  #ACCOUNTING FOR LEFTOVER ELEMENTS IN EITHER LIST, ONCE ALL ELEMENTS OF THE OTHER LIST HAVE BEEN APPENDED
  #FIRST CHECKING IF LEFTOVER ELEMENTS IN FIRST LIST
  WHILE array1_index < LEN(array1)
    APPEND(merged_list, array1[array1_index])
  ENDWHILE
  #NEXT CHECKING IF LEFTOVER ELEMENTS IN SECOND LIST
  WHILE array2_index < LEN(array2)
    APPEND(merged_list, array2[array2_index])
  ENDWHILE
  RETURN merged_list
ENDSUBROUTINE

SUBROUTINE mergeSort(array)
  #CHECKING TO SEE IF 1 OR 0 ELEMENTS IN LIST. IF SO, THE LIST IS ALREADY SORTED AND CAN SIMPLY BE RETURNED
  IF LEN(array) <= 1 THEN
    RETURN array
  ENDIF
  #FINDING THE MIDPOINT USING INTEGER DIVISION AND THEN SPLITTING THE LIST INTO TWO SORTED HALVES
  midpoint ← LEN(array) DIV 2
  array1 ← mergeSort(array[midpoint:])
  array2 ← mergeSort(array[:midpoint])
  #MERGING THE TWO SORTED LISTS INTO THE FINAL SORTED LIST TO BE RETURNED
  RETURN merge(array1, array2)
ENDSUBROUTINE
```

Figure 27: Merge Sort Algorithm

The above merge sort pseudocode will be used to sort the players in the leaderboard based on the selected statistic using the data from the leaderboard database.

```
#SUBROUTINE TO HASH PASSWORDS
SUBROUTINE hashing_algorithm(password)
  #INITIALIZING THE HASHING ALGORITHM
  hash_value ← 0

  # ITERATE THROUGH EACH CHARACTER IN THE PASSWORD
  FOR i ← 0 TO LENGTH(password) - 1
    char ← password[i]

    #GET THE UNICODE VALUE OF THE CHARACTER
    code ← ASC(char)

    #PERFORM A LEFT SHIFT BASED ON THE CHARACTER POSITION
    shifted_code ← code * (2 ^ (i MOD 5))

    #ADD THE SHIFTED CODE MULTIPLIED BY i + 1 TO FURTHER INCREASE VARIATION BASED ON POSITION
    hash_value ← hash_value + (shifted_code * (i + 1))
  NEXT i
ENDFOR

#LIMIT THE SIZE OF THE VALUE, WITHOUT RESTRICTING IT TOO MUCH
hash_value ← hash_value MOD 100000

RETURN STRING(hash_value)
ENDSUBROUTINE
```

Figure 28: Hashing Algorithm

This is that will be used to hash passwords when storing passwords in the backend database, during registration program.

```
#SUBROUTINE TO DETERMINE IF BLUFFS ARE SUCCESSFUL
SUBROUTINE bluff_successful(game_info)
  #INITIALIZING AN EMPTY ARRAY OF ALL DICE
  total_dice ← []

  #ITERATING THROUGH EACH PLAYER AND APPENDING THEIR DICE TO THE LIST
  FOR EVERY Player IN players

    dice_list ← []

    #ITERATING THROUGH THEIR DICE AND APPENDING THEM TO THE DICE LIST
    FOR EVERY Dice IN Player_dice
      APPEND(dice_list, Dice)
    ENDFOR

    #NOW ADD THIS INDIVIDUAL PLAYER'S DICE TO THE TOTAL LIST
    APPEND(total_dice, dice_list)
  ENDFOR

  #RETRIEVING THE BID BEING INVESTIGATED FROM THE GAME INFO
  bid ← (last made bid from game_info)

  bluff_success = True

  #IF THE NUMBER OF THE BID VALUE ACTULLY IN THE GAME IS GREATER OR EQUAL TO THE BID QUANTITY
  THE BID IS VALID, SO BLUFF CALL IS NOT SUCCESSFFUL
  IF total_dice.count( bid['value'] ) >= bid['quantity'] THEN

    bluff_success ← False
  ELSE
    CONTINUE

  #RETURN BOOLEAN VALUE INDICATING SUCCESS OF THE BLUFF CALL
  RETURN bluff_success
ENDSUBROUTINE
```

Figure 29: bluff_successful pseudocode algorithm

This Pseudocode algorithm defines a function used to determine whether or not a bluff call is successful, based on a previous bid. This function should be called during the game logic, when assessing the result of a player/bot's decision to call another player's bluff.

2.2.3 Data Flow Diagram

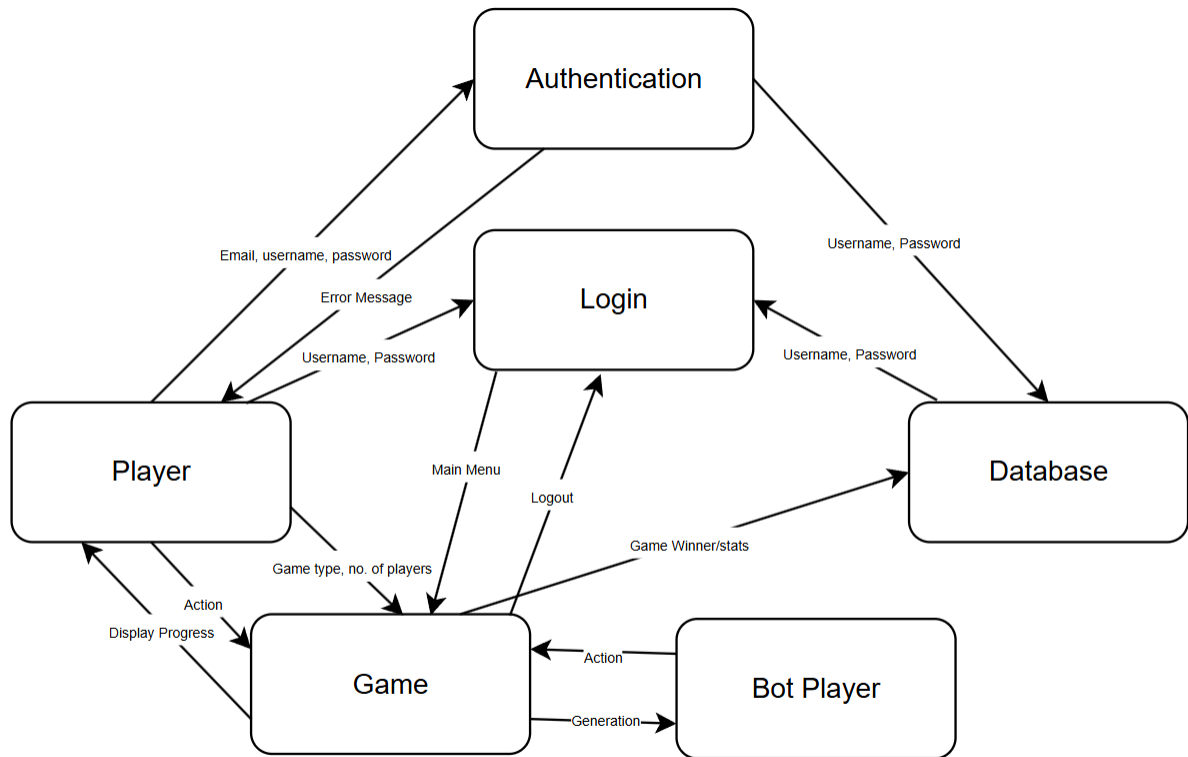


Figure 30: Data Flow Diagram

This level 1 DFD demonstrates the 6 major entities of my perudo game and their interactions and their relationships.

2.2.4 Input Process Output Storage Diagram

IPSO	Program Section	Details
Input	Player Data	Username Password Email Wins
	Game Information	GameMode Number of Players Number of Bots Time Limit Bot Difficulty
Processing	Game	CheckRoundOver CheckRoundLoser BluffCallSuccess SubtractDice ShuffleDice CheckGameOver CheckWinner UpdateBidOptions
	Leaderboard	SortUsersByKey MergeSort Win % calculation
	Miscellaneous	HashingAlgorithm
Storage	UserInformation	Username Password Games Wins Win %
	Load/Save Bot Game	Game settings Game status/information
Output	Display	Game GUI Settings GUI
	Leaderboard	Personal Player Statistics Global Player Statistics
	View Profile	Email Username Password
	Game Data	Player Dice Number of Dice List of Bid Options Previous Bids Other Players

Figure 31: IPSO diagram

This is the rough Input, Processing and Output (IPSO) diagram for my system, modelling the various systems that will be required in this implementation.

2.3 Back-end Database for User Management

User
PK User_id ; INT
Username; VARCHAR
Password; VARCHAR
Email; VARCHAR
Player_stats; TEXT

Figure 32: Initial Database Table

I decided to use a basic flat-file database for my game's user management, as seen in figure 28, consisting of a single table, named user, storing all the relevant user information. However, this initial design was not in first normal form (1NF), since the Player_stats field was non-atomic, containing multiple pieces of information, including number of games played and wins.

User	
PK	User_id ; INT
	Username; VARCHAR
	Password; VARCHAR
	Email; VARCHAR
	Games_played; INT
	Wins; INT

Figure 33: 1NF Database Design

Therefore, I broke down the player_stats field into number of games_played and wins, making every field atomic, satisfying the first normal form. Furthermore, since the primary key is simple, and all non-key attributes rely entirely on the primary key, there are no partial dependencies, making this structure satisfy Second Normal Form. In addition to this, this structure also satisfies Third Normal Form, because there are also no non-key dependencies.

```
SELECT Username, Wins, Games_played
FROM user;
```

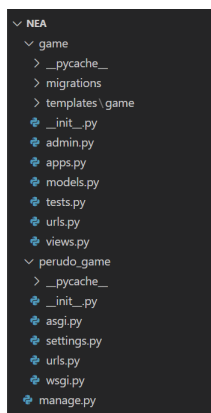
Figure 34: SQL Leaderboard Query

Figure 30 shows a simple SQL query that could be used to retrieve data for the leaderboard display, before calculating the win percentage and ordering the players using my merge sort algorithm.

3 Program Implementation

3.1 File Structure and Django

To implement this web-game I used the python Django extension, along with my own database, created using MySQL. When creating a new django project and app a skeleton structure is automatically generated, consisting of the following file structure:



The game folder refers to the django app, whereas the perudo_game refers to the more general app folder.

It is worth nothing, that there are several features of the django framework that I never used throughout this project, including the django testing, models and admin file features. However, many of these files still had to exist for the functioning of the program, so I had to leave these in the file structure.

3.2 The Project Folder

Within the project file ('perudo_game'), there are several files involved with the global settings, configurations and URLs of the web application.

Of these project files, the settings.py file is responsible for many of the settings of the django application, and is automatically generated with default configurations for a variety of features of the project, including the mySQL database connection.

```
1 """
2 Django settings for perudo_game project.
3
4 Generated by 'django-admin startproject' using Django 5.1.5.
5
6 For more information on this file, see
7 https://docs.djangoproject.com/en/5.1/topics/settings/
8
9 For the full list of settings and their values, see
10 https://docs.djangoproject.com/en/5.1/ref/settings/
11 """
12
13 from pathlib import Path
14
15 # Build paths inside the project like this: BASE_DIR / 'subdir'.
16 BASE_DIR = Path(__file__).resolve().parent.parent
17
18
19 # Quick-start development settings - unsuitable for production
20 # See https://docs.djangoproject.com/en/5.1/howto/deployment/checklist/
21
22 # SECURITY WARNING: keep the secret key used in production secret!
23 SECRET_KEY = 'django-insecure-pm%nv&v49#qx@gkdvpp*94gmt6r2!apa7nkt=-gi!khd!g9qqg'
24
25 # SECURITY WARNING: don't run with debug turned on in production!
26 DEBUG = True
27
28 ALLOWED_HOSTS = []
29
30
31 #Outlines the default django applications used in project
32
33 INSTALLED_APPS = [
34     'django.contrib.admin',
35     'django.contrib.auth',
36     'django.contrib.contenttypes',
37     'django.contrib.sessions',
38     'django.contrib.messages',
39     'django.contrib.staticfiles',
40     'game',
41 ]
42
43 MIDDLEWARE = [
44     'django.middleware.security.SecurityMiddleware',
45     'django.contrib.sessions.middleware.SessionMiddleware',
46     'django.middleware.common.CommonMiddleware',
47     'django.middleware.csrf.CsrfViewMiddleware',
48     'django.contrib.auth.middleware.AuthenticationMiddleware',
49     'django.contrib.messages.middleware.MessageMiddleware',
50     'django.middleware.clickjacking.XFrameOptionsMiddleware',
```

```
51 ]
52
53 #Directs the url configuration to the urls.py file
54 ROOT_URLCONF = 'perudo_game.urls'
55
56 TEMPLATES = [
57     {
58         'BACKEND': 'django.template.backends.django.DjangoTemplates',
59         'DIRS': [],
60         'APP_DIRS': True,
61         'OPTIONS': {
62             'context_processors': [
63                 'django.template.context_processors.debug',
64                 'django.template.context_processors.request',
65                 'django.contrib.auth.context_processors.auth',
66                 'django.contrib.messages.context_processors.messages',
67             ],
68         },
69     ],
70 ]
71
72 #Defines the correct Web Server Gateway Interface service is used
73 WSGI_APPLICATION = 'perudo_game.wsgi.application'
74
75
76 #Connects the database to the local mySQL ser
77
78 DATABASES = {
79     'default': {
80         'ENGINE': 'django.db.backends.mysql',
81         'NAME': 'NEA_db',
82         'USER': 'root',
83         'PASSWORD': 'NEApassword2025',
84         'HOST': 'localhost',
85         'PORT': '3306', # Default mySQL port
86     }
87 }
88
89
90 #Auto-generated default password validation settings (not used)
91
92 AUTH_PASSWORD_VALIDATORS = [
93     {
94         'NAME': 'django.contrib.auth.password_validation.
95         UserAttributeSimilarityValidator',
96     },
97     {
98         'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
99     },
100    {
101        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
102    },
103    {
104        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator'
105    },
106 ]
107
```

```
108 # Internationalization
109 # https://docs.djangoproject.com/en/5.1/topics/i18n/
110
111 LANGUAGE_CODE = 'en-us'
112
113 TIME_ZONE = 'UTC'
114
115 USE_I18N = True
116
117 USE_TZ = True
118
119
120 # Static files (CSS, JavaScript, Images)
121 # https://docs.djangoproject.com/en/5.1/howto/static-files/
122
123 STATIC_URL = 'static/'
```

Listing 1: Settings.py file

The urls.py file is the central URL dispatcher for the django project, referencing the app's urls.py file, to properly coordinate the projects URLs.

```
1 """
2 URL configuration for perudo_game project.
3
4 The 'urlpatterns' list routes URLs to views. For more information please see:
5     https://docs.djangoproject.com/en/5.1/topics/http/urls/
6 Examples:
7 Function views
8     1. Add an import:  from my_app import views
9     2. Add a URL to urlpatterns:  path('', views.home, name='home')
10 Class-based views
11     1. Add an import:  from other_app.views import Home
12     2. Add a URL to urlpatterns:  path('', Home.as_view(), name='home')
13 Including another URLconf
14     1. Import the include() function: from django.urls import include, path
15     2. Add a URL to urlpatterns:  path('blog/', include('blog.urls'))
16 """
17 from django.contrib import admin
18 from django.urls import path, include
19
20 urlpatterns = [
21     path('admin/', admin.site.urls),
22     path('', include('game.urls')),
23 ]
```

Listing 2: urls(project).py file

Django also generates asgi and wsgi files to handle web requests:

```
1 """
2 WSGI config for perudo_game project.
3
4 It exposes the WSGI callable as a module-level variable named ``application``.
5
6 For more information on this file, see
7 https://docs.djangoproject.com/en/5.1/howto/deployment/wsgi/
8 """
9
10 import os
```

```
11
12 from django.core.wsgi import get_wsgi_application
13
14 os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'perudo_game.settings')
15
16 application = get_wsgi_application()
```

Listing 3: wsgi.py file

```
1 """
2 ASGI config for perudo_game project.
3
4 It exposes the ASGI callable as a module-level variable named ``application``.
5
6 For more information on this file, see
7 https://docs.djangoproject.com/en/5.1/howto/deployment/asgi/
8 """
9
10 import os
11
12 from django.core.asgi import get_asgi_application
13
14 os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'perudo_game.settings')
15
16 application = get_asgi_application()
```

Listing 4: asgi.py file

The `__init__.py` file is an empty file used to tell python to treat the directory as a package, facilitating imports and being crucial to the functioning of any django project.

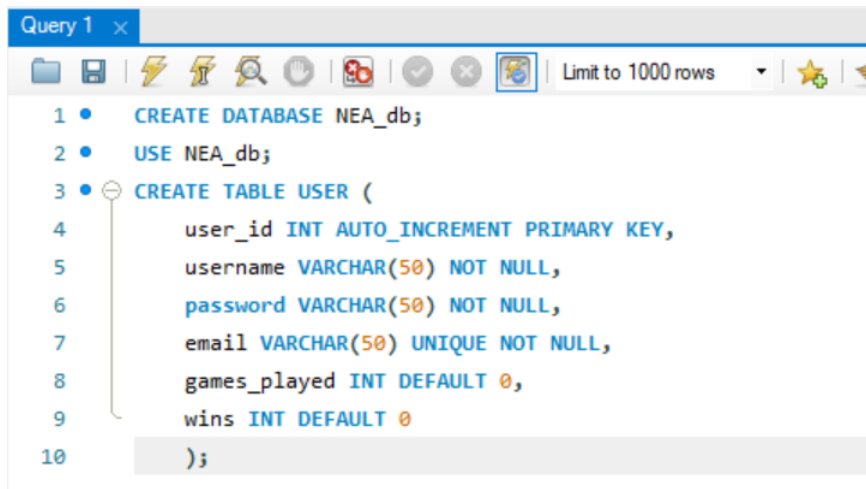
There is also a file called `manage.py` outside of the app or project folder, which is responsible for running administrative commands and running the server. It is also responsible for directing the django framework's settings to the project's settings.py file.

```
1 #!/usr/bin/env python
2 """Django's command-line utility for administrative tasks."""
3 import os
4 import sys
5
6
7 def main():
8     """Run administrative tasks."""
9     os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'perudo_game.settings')
10     try:
11         from django.core.management import execute_from_command_line
12     except ImportError as exc:
13         raise ImportError(
14             "Couldn't import Django. Are you sure it's installed and "
15             "available on your PYTHONPATH environment variable? Did you "
16             "forget to activate a virtual environment?"
17         ) from exc
18     execute_from_command_line(sys.argv)
19
20
21 if __name__ == '__main__':
22     main()
```

Listing 5: manage.py file

3.3 Backend database

To store user information for this project, I created my own mySQL database, connecting, establishing a database connection to the views.py program, allowing me to use parameterized SQL statements embedded in my view.py file for various functions. I created the database, as seen in my documented design, using the following Data Definition Language statements.



3.4 The App Folder and Actual Program.

Within the app folder, several files are automatically generated as part of the django framework, including another empty `__init__.py` file. It also generates a `models.py` file, used for creating django models, however I did not use this throughout this project. Here is the default generated file:

```
1 from django.db import models
2 class ExampleModel(models.Model):
3     name = models.CharField(max_length = 100)
```

Listing 6: models.py file

Another generated file which I never used in this project, is the `tests.py` file. Considering I would be testing this program thoroughly myself anyway, I did not see any need to use this django feature.

```
1 from django.test import TestCase
2 # Create your tests here.
```

Listing 7: tests.py file

Like in the project folder, there is also a `urls.py` file, which routes all the URLs used by the app itself, connecting each view to its corresponding HTML template page, as shown below.

```
1 from django.urls import path
2 from . import views
3
4 urlpatterns = [
5     path('', views.home, name='home'), # Route for the homepage
6     path('main-menu/', views.main_menu, name='main_menu'),
7     path('logout/', views.user_logout, name='logout'),
8     path('register/', views.register, name='register'),
9     path('pass-and-play/', views.pass_and_play, name='pass_and_play'),
10    path('game-over/', views.game_over, name='game_over'),
11    path('setup-pass-and-play/', views.setup_pass_and_play, name='
    setup_pass_and_play'),
12    path('game-rules/', views.game_rules, name='game_rules'),
```

```

13 path('view_profile/', views.view_profile, name='view_profile'),
14 path('setup_bot_game/', views.setup_bot_game, name='setup_bot_game'),
15 path('bot_game/', views.bot_game, name='bot_game'),
16 path('view_leaderboard/', views.view_leaderboard, name = 'view_leaderboard'),
17 ]

```

Listing 8: urls(app).py file

In addition to these, an admin.py file was also generated, being responsible for registering models with the Django admin interface. However, since I did not use any of this functionality, I did not touch this file.

```

1 from django.contrib import admin
2
3 # Register your models here.

```

Listing 9: admin.py file

Furthermore, an apps.py file was also generated, simply configuring the application and establishing the python path to the app. I left this file with its default contents.

```

1 from django.apps import AppConfig
2
3
4 class GameConfig(AppConfig):
5     default_auto_field = 'django.db.models.BigAutoField'
6     name = 'game'

```

Listing 10: apps.py file

Finally, the views.py file is where my actual program resides, with various views and functions being stored here. I created the following views for the purpose of this program.

View	Purpose
Home	This view is responsible for controlling the login process and redirecting the user to the registration page, if they so choose. It allows the user to enter their username and password, before verifying these details and redirecting them to the main menu if correct.
Register	The register view is used when the user chooses to register from the home page, and manages the user's account creation. The user enters their email, username, password and confirmed password, before creating an account, if these entries are validated. They should then be redirected back to the home page to log in.
Main Menu	Once the user has logged in, they should be taken to this page, where the various buttons take redirect the user to parts of the program. There is relatively little actual processing in this view, aside from retrieving the user's username to display at the top corner of the screen, to confirm they are logged into the correct account.
View Profile	This view manages the view profile page, displaying the user's username and password, as well as managing a form for the user to change their password if they so choose. This view interacts heavily with the backend database to not only update passwords, but to also validate entered details.
View Leaderboard	This view is responsible for correctly displaying a leaderboard of all users' stats, allowing these to be sorted by the user's choice of statistic.

Game Rules and Tutorial	This view simply renders in the pre-written game instructions and embedded tutorial video.
Setup pass and play	This view is responsible for storing the user's choice of game settings, including the number of initial dice and players, before passing this to the actual pass-and-play gamemode.
Setup bot game	This view is responsible for storing the user's choice of game settings, including the number of initial dice and bots, before passing this to the actual bot game gamemode.
Pass-and-Play	This view holds the game logic for the actual pass-and-play gamemode, providing all the relevant information and game data for the interface/html page to use, as well as processing any actions made by the user. Within this view various functions such as the bluff_successful function.
Bot Game	This view is responsible for coordinating the 'bot game' gameplay, providing all the relevant information and game data to the interface, as well as processing any actions taken by the user. This view also uses several functions, including a separate 'Bot_turn' function to manage each bot's decision making.
Game Over	The Game-Over view is primarily responsible for displaying the final result of the game, as well as incrementing the wins of the user by 1, if they have won the game.

There are also many separate functions used by these various views as shown in the table below.

Functions	Purpose	Views/functions used in
user_logout	This function is called whenever the user decides to logout from the main menu, being responsible for flushing any session data, as well as redirecting the user to the home page.	Main menu
bluff_successful	This function determines the result of a bluff call made by a bot or player.	pass_and_play, bot_game, bot_turn
sort_users_by_key	This function takes a list of users, a list of a certain stat from the users and a key to sort by, before returning a sorted user list, for use in the leaderboard view. It relies on the mergeSort and merge functions to do this.	view_leaderboard
merge	This function is called by the mergeSort function which uses recursion to properly sort the list of users in the leaderboard.	mergeSort, sort_users_by_key
mergeSort	This function calls the merge function and is responsible for properly sorting lists of stats, to aid the sort_users_by_key function.	sort_users_by_key
Bot_turn	This is the function responsible for controlling the bot's decision making, as well as how the program progresses following the bot turn. It takes in the current game data and returns a bot action message to be displayed, as well as flag determining whether or not the round has ended	Bot_game
hashing_algorithm	This function contains a relatively simple but effective hashing algorithm responsible for hashing passwords to be registered in the database securely.	Register, view_profile, home

Here is my views.py with detailed comments to describe the purpose and implementation of each function or view.

```

1 #importing the random module to randomise dice
2 import random

```

```
3
4 #importing module required to connect the mySQL backend database to my game
5 import mysql.connector
6
7 #Standard pre-generated Django imports
8 from django.shortcuts import render, redirect
9 from django.contrib.auth.models import User
10 from django.contrib import messages
11
12 #importing hte regular expression module to help wiht later data validation
13 import re
14
15 #sets up the backend database connection
16 def get_db_connection():
17     return mysql.connector.connect(
18         host = "localhost",
19         user = "root",
20         password = "NEApassword2025",
21         database = "NEA_db",
22     )
23
24 #Creating a hashing algorithm to store hashed passwords in the database
25 def hashing_algorithm(password):
26
27     #Initialize the hash value
28     hash_value = 0
29
30     #Looping through each character
31     for i, char in enumerate(password):
32
33         #Finding the unicode representation of the character
34         code = ord(char)
35
36         #To introduce more variation based on character position we now Left shift
37         #by i % 5
38         shifted_code = code << (i % 5)
39
40         #Now I will sum up these shifted values, further multiplied by their
41         #position (+1 so no multiplication by 0)
42         hash_value += shifted_code * (i + 1)
43
44         #Using modular arithmetic to prevent the value being too high, whilst still
45         #giving it enough possible values to function properly
46         hash_value = hash_value % 100000
47     return str(hash_value)
48
49 #This is the view corresponding to the homepage, where the user can login or
50 #choose to register
51 def home(request):
52
53     #selection code runs when the user submits the form
54     if request.method == 'POST':
55
56         # Retrieve the username and password from the form entered by the user
57         username = request.POST.get('username')
58         password = request.POST.get('password')
59
60         # Connects to the database to retrieve the user's details using an SQL
61         # query, before storing these details in the variable 'user'.
```

```
57     conn = get_db_connection()
58     cursor = conn.cursor(dictionary = True)
59     cursor.execute("SELECT * FROM User WHERE username = %s", (username,))
60     user = cursor.fetchone()
61     cursor.close()
62     conn.close()
63
64     #If there is an account in the database associated with the entered
username and the account's password matches the one entered, log the user into
the session.
65     if user and hashing_algorithm(password) == user['password']:
66
67         #The user's id and username are stored in the django session, which
can later be retrieved.
68         request.session['user_id'] = user['user_id']
69         request.session['username'] = user['username']
70         request.session['unhashed_password'] = password
71
72         #Once the user's session has been accounted for they are redirected to
the main menu page.
73         return redirect('main_menu')
74     else:
75
76         #If there is no account associated with the username or the password
is incorrect an error is printed.
77         messages.error(request, "Invalid username or password.")
78
79         #They are then redirected to the same page, preventing them from
continuing to the main menu.
80         return redirect('home')
81     return render(request, 'game/home.html')
82
83 #This view handles the main menu screen.
84 def main_menu(request):
85
86     #Any game data from previous pages must be deleted, once the user has gone
back to this menu, as they have left the game and this might interfere with
future games.
87     if 'game_data' in request.session:
88         del(request.session['game_data'])
89
90     #Yet again retrieves any user data, to be used by the page (when displaying
the username in the corner of the screen)
91     conn = get_db_connection()
92     cursor = conn.cursor(dictionary=True)
93     cursor.execute("SELECT username FROM User WHERE user_id = %s", (request.
session['user_id'],))
94     user_data = cursor.fetchone()
95     cursor.close()
96     conn.close()
97
98     #The corresponding html file is then rendered
99     return render(request, 'game/main_menu.html', {'user': user_data})
100
101 #This function is called when the presses 'log out', sending them back to the home
screen
102 def user_logout(request):
103
104     #Clear the session data to log out the user
```

```
105     request.session.flush()
106     #Sends the user back to the home page to login or register again.
107     return redirect('home')
108
109 #Once the user decides to register, this view will manage the registration process
110 def register(request):
111
112
113     if request.method == 'POST':
114
115         #Retrieves the form data
116         username = request.POST.get('username')
117         password = request.POST.get('password')
118         email = request.POST.get('email')
119         confirm_password = request.POST.get('confirm_password')
120
121         #Makes sure the password matches the confirmed password, redirecting the
122         user back if it does not and displaying an error
123         if password != confirm_password:
124             messages.error(request, 'Passwords do not match!')
125             return redirect('register')
126
127         #Defining an appropriate regular expression for the entered email, and
128         making sure the email fits this regex, using the re module
129         email_regex = '^.+@.+$'
130         valid_email = re.search(email_regex, email)
131
132         #If the email does not fit this expression, it is redirected to the
133         register page once more, with the forms havin been reset
134         if not valid_email:
135             messages.error(request, "Email must consist of an '@' between two
136             strings of characters.")
137             return redirect('register')
138
139         #Connecting the database again to insert new account details
140         conn = get_db_connection()
141         cursor = conn.cursor()
142
143         #Makes sure there is user input
144         if not username or not password:
145             messages.error(request, 'Username and password are required.')
```

```
146         else:
147
148             #Retrieves a pre-existing user from the database with either this
149             username or email, if it exists
150             cursor.execute("SELECT * FROM User WHERE username = %s OR email = %s",
151             (username, email))
152             existing_user = cursor.fetchone()
153
154             #If there is a pre-existing user in the database, an error is
155             displayed and the user must try again
156             if existing_user:
157                 messages.error(request, "Username or email already exists.")
158                 cursor.close()
159                 conn.close()
160             else:
```

```
156         #Inserts the entered details as a new record into the database
157         #First make sure to hash the password before storing it
158         hashed_password = hashing_algorithm(password)
159         cursor.execute("INSERT INTO User (username, email, password)
VALUES (%s, %s, %s)", (username, email, hashed_password))
160         conn.commit()
161         cursor.close()
162         conn.close()
163
164         messages.success(request, "Account created successfully!")
165
166         return redirect('home')
167
168
169     return render(request, 'game/register.html')
170
171 #This is the view responsible for setting up the pass-and-play game
172 def setup_pass_and_play(request):
173
174     if request.method == 'POST':
175
176         #Retrieves the number of players and initial dice from the form
177         num_players = int(request.POST.get('num_players'))
178         initial_dice = int(request.POST.get('initial_dice'))
179
180         #Stores game configuration in the session
181         request.session['game_settings'] = {
182             'num_players': num_players,
183             'initial_dice': initial_dice,
184         }
185
186         #Now the user's stats are updated, incremented their games played by 1
187         conn = get_db_connection()
188         cursor = conn.cursor()
189         cursor.execute("UPDATE User SET games_played = games_played + 1 WHERE
user_id = %s", ( request.session['user_id'], ))
190         conn.commit()
191         cursor.close()
192         conn.close()
193
194         #The user is then sent to the pass and play page
195         return redirect('pass_and_play')
196
197     return render(request, 'game/setup_pass_and_play.html')
198
199
200 def pass_and_play(request):
201
202     #Now the game settings are retrieved from what was stored by the setup page
203     game_settings = request.session.get('game_settings', None)
204
205     #Initialize game data if not already in the session
206     if 'game_data' not in request.session:
207         num_players = game_settings['num_players']
208         initial_dice = game_settings['initial_dice']
209
210     #The default game data is now set for the game, as well as initializing
all the players in the game, with their initial dice numbers and dice
211     request.session['game_data'] = {
```

```
212         'players': [
213             {'name': f'Player {i+1}', 'dice': initial_dice, 'dice_values': [],
214             'rolled': False}
215             for i in range(num_players)
216         ],
217         'current_player': 0,
218         'round_active': True,
219         'current_bid': None,
220         'bluff_result': None,
221         'round_ended': False,
222     }
223
224     #Now all the values of the session['game_data'] dictionary are retrieved to be
225     #used in the game
226     game_data = request.session['game_data']
227     players = game_data['players']
228     current_player_index = game_data['current_player']
229     current_player_name = players[current_player_index]['name']
230     current_bid = game_data['current_bid']
231
232     if request.method == 'POST':
233
234         #What ever the user has decided to do is now retrieved and stored
235         action = request.POST.get('action')
236
237         #If the round has ended the user should not be able to do anything but end
238         #the round
239         if game_data['round_ended'] and action != 'end_round':
240             request.session['error_message'] = "You must press 'End Round' before
241             continuing!"
242             return redirect('pass_and_play')
243
244         #Rolling the player's dice
245         if action == 'roll_dice':
246
247             #Only roll if the current player has not rolled yet
248             if not players[current_player_index]['rolled']:
249
250                 #Randomises the player's current dice values
251                 dice_values = []
252                 for i in range(players[current_player_index]['dice']):
253                     dice_values.append(random.randint(1, 6))
254
255                 players[current_player_index]['dice_values'] = dice_values
256
257                 #Now we must mark the player as having rolled so they do not roll
258                 #again
259                 players[current_player_index]['rolled'] = True
260                 request.session['game_data'] = game_data
261
262                 #Prevent the user from rolling again, providing an appropriate error
263                 #message
264                 else:
265                     request.session['error_message'] = f"{current_player_name} has
266                     already rolled their dice this round!"
267                     return redirect('pass_and_play')
268
269                 #Making a bid
270                 elif action == 'make_bid':
```

```
264
265     #Retrieving the user's bid entries
266     bid_count = int(request.POST.get('bid_count'))
267     bid_value = int(request.POST.get('bid_value'))
268
269     #Validating the bid
270     if current_bid is not None:
271
272         #Making sure the bid is raised
273         if bid_count < current_bid['count'] or (bid_count == current_bid['
274 count'] and bid_value <= current_bid['value']):
275             request.session['error_message'] = "Your bid must be higher
276 than the current bid!"
277             return redirect('pass_and_play')
278         #Updating the current bid
279         game_data['current_bid'] = {'count': bid_count, 'value': bid_value}
280
281         #The current player should now be incremented by 1 or loop back to the
282         first player using modular arithmetic.
283         game_data['current_player'] = (current_player_index + 1) % len(players
284 )
285
286         request.session['game_data'] = game_data
287         return redirect('pass_and_play')
288
289     #Calling a bluff
290     elif action == 'call_bluff':
291
292         #Making sure there is a bid to call a bluff fon
293         if not current_bid:
294             request.session['error_message'] = "No bid to call a bluff on!"
295             return redirect('pass_and_play')
296
297         #Counting up the number of dice of the bid's value, by going through
298         each player's dice and seeing if they have dice of that value
299         total_count = 0
300         for player in players:
301             for value in player['dice_values']:
302                 if value == current_bid['value']:
303                     total_count += 1
304
305         #Determining whether or not the current bid is valid, to see if the
306         bluff call is successful
307         if total_count >= current_bid['count']:
308
309             #Removing one of the current player's dice since they called a
310             bluff unsuccessfully
311             players[current_player_index]['dice'] -= 1
312             game_data['bluff_result'] = f"Bluff failed! {current_player_name}
313 loses a die."
314
315         else:
316             #Removing one of the previous player's dice since the bluff call
317             was successful and their bid was incorrect
318             previous_player_index = (current_player_index - 1) % len(players)
319             players[previous_player_index]['dice'] -= 1
320             game_data['bluff_result'] = f"Bluff successful! {players[
321 previous_player_index]['name']} loses a die."
```

```
313         #Makes sure the round is ended
314         game_data['round_ended'] = True
315         request.session['game_data'] = game_data
316         return redirect('pass_and_play')
317
318     #Ending the round
319     elif action == 'end_round':
320
321         #Goes through each player and eliminates them if they have no dice
322         game_data['players'] = []
323         for player in players:
324             if player['dice'] > 0:
325                 game_data['players'].append(player)
326
327         #Now we have to check if the game is over, by seeing if there is more
than 1 player left after an elimination (or lack thereof)
328         if len(game_data['players']) == 1:
329             winner = game_data['players'][0]['name']
330             request.session['game_winner'] = winner
331             return redirect('game_over')
332
333         #The game states have to be reset before starting the next round
334         game_data['current_bid'] = None
335         game_data['round_ended'] = False
336         game_data['bluff_result'] = None
337
338         #The current player now increments by 1 or loops back to the beginning
339         game_data['current_player'] = (current_player_index + 1) % len(
game_data['players'])
340
341         #We must now set rolled to false for all the players, so they can roll
again next round and also reset their dice values
342         for player in game_data['players']:
343             player['rolled'] = False
344             player['dice_values'] = []
345
346         request.session['game_data'] = game_data
347         return redirect('pass_and_play')
348
349     #Now, treating the session error messages as a stack, we can set the error
message to be shown as the last error message displayed,
350     #as a stack has a last in first out structure.
351     error_message = request.session.pop('error_message', None)
352
353     #Retrieving the bluff result we previously determined when the player made a
bluff call
354     bluff_result = game_data.get('bluff_result', None)
355
356     #We now pass all of the relevant data to the html template to correctly
display the gameplay.
357     return render(request, 'game/pass_and_play.html', {
358         'game_data': game_data,
359         'current_player_name': current_player_name,
360         'error_message': error_message,
361         'bluff_result': bluff_result,
362         'current_player_dice': players[current_player_index]['dice_values'],
363         'rolled': players[current_player_index]['rolled'],
364         'round_ended': game_data['round_ended'],
365     })
```

```
366
367 #This is the view responsible for setting up the bot game (fairly similar to setup
    pass and play view)
368 def setup_bot_game(request):
369     if request.method == 'POST':
370         #Retrieving the dice and number of bot options from the form
371         initial_dice = int(request.POST.get('initial_dice'))
372         num_bots = int(request.POST.get('num_bots'))
373
374         #Stores game configuration in the session
375         request.session['game_settings'] = {
376             'num_bots': num_bots,
377             'initial_dice': initial_dice,
378         }
379
380         #We must now update the user's games played, using the following SQL query
381         .
382         conn = get_db_connection()
383         cursor = conn.cursor()
384         cursor.execute("UPDATE User SET games_played = games_played + 1 WHERE
user_id = %s", ( request.session['user_id'], ))
385         conn.commit()
386         cursor.close()
387         conn.close()
388         return redirect('bot_game')
389         return render(request, 'game/setup_bot_game.html')
390
391 def bot_game(request):
392
393     #Retrieve the game settings from the setup page
394     game_settings = request.session.get('game_settings')
395
396     #Now to initialize the game data if game has not already started
397     if 'game_data' not in request.session:
398         num_bots = game_settings['num_bots']
399         initial_dice = game_settings['initial_dice']
400
401         #We now set the default game data, like in the pass and play page, but
this time we initialize both player 1 and Bots 1, 2 e.t.c
402         request.session['game_data'] = {
403             'players': [{ 'name': 'Player 1', 'dice': initial_dice } +
404                         [ { 'name': f'Bot {i + 1}', 'dice': initial_dice } for i in
range(num_bots) ],
405             'current_player': 0,
406             'current_bid': None,
407             'rolled': False,
408             'game_over': False,
409         }
410
411         game_data = request.session['game_data']
412
413         players = game_data['players']
414
415         current_player = game_data.get('current_player', 0)
416         #If the current player index has reached or exceeded the number of players (
likely due to eliminations), reset it back to the first player to avoid
indexing errors and continue the round properly.
417         if current_player >= len(players):
```

```
418     game_data['current_player'] = 0
419     current_player = 0
420
421     current_bid = game_data.get('current_bid')
422
423     #Useful Debugging statements
424     print(f"Bot-game: debug statements")
425     print(f"Current Player: {players[current_player]['name']}")
426     print(f"Current Bid: {game_data.get('current_bid')}")
427     print(f"Rolled: {players[current_player].get('rolled', False)}")
428     print(f"Game Over: {game_data.get('game_over', False)}")
429
430     #Counting the number of remaining 'alive' players with dice, storing each as
431     #winner, in case they are the only one, to later find the winner
432     valid_players = 0
433     winner = {}
434     for player in players:
435         if player['dice'] > 0:
436             valid_players += 1
437             winner = player
438
439     #If there is only one player left with a die, end the game, and store the
440     #winner for the game over page
441     if valid_players <= 1:
442         game_data['game_over'] = True
443         request.session['game_winner'] = winner['name']
444         request.session['game_data'] = game_data
445         return redirect('game_over')
446
447     #If the round has not started, roll all the bots' dice, allowing the player to
448     #roll their own.
449     if not game_data.get('round_started', False):
450         for player in players:
451             if 'Bot' in players[current_player]['name']:
452                 rolled_dice = []
453                 for i in range(player["dice"]):
454                     rolled_dice.append(random.randint(1, 6))
455                 player['rolled_dice'] = rolled_dice
456                 player['rolled'] = True
457             game_data['round_started'] = True
458
459     #More debug statements, printing players' dice
460     print(f"Players/bots' dice:")
461     for player in players:
462         print(f"{player['name']}: {player.get('rolled_dice', 'Not Rolled')}")
463
464     #Dealing with player's actions
465     if request.method == 'POST':
466         action = request.POST.get('action')
467         #Player action debug statement
468         print(f"Player action: {action}")
469
470         #Rolling the player's dice
471         if action == 'roll_dice':
472             if players[current_player].get("rolled", False):
473                 request.session['error_message'] = f"{players[current_player]['name']} has already rolled their dice this round!"
474             else:
```

```
473
474         #Randomising each of the player's dice, before marking them as
having rolled their dice, to prevent re-rolls
475         rolled_dice = []
476         for i in range(players[current_player]["dice"]):
477             rolled_dice.append(random.randint(1, 6))
478
479         players[current_player]["rolled_dice"] = rolled_dice
480         players[current_player]["rolled"] = True
481         print(f"Player {players[current_player]['name']} rolled {players[
current_player]['rolled_dice']}")
482
483         request.session["game_data"] = game_data
484         return redirect("bot_game")
485
486     #Making a bid
487     elif action == 'make_bid':
488
489         #Collecting the given bid count and value
490         bid_count = int(request.POST.get('bid_count', 0))
491         bid_value = int(request.POST.get('bid_value', 0))
492
493         #Debugging player's bid
494         print(f"Player's bid")
495         print(f"Bid Count: {bid_count}, Bid Value: {bid_value}")
496
497         #Fetching the previous bid (or current bid, as far as the game data
knows)
498         current_bid = game_data.get('current_bid', None)
499
500         #Checking to see if there was a previous bid, in which case the bid
will have to raised higher than the previous
501         if current_bid:
502
503             #Making sure the entered bid count and value is valid.
504             if bid_count <= 0 or bid_value <1 or bid_value > 6:
505                 request.session['error_message'] = 'Invalid bid! Make sure to
enter a valid number and dice face.'
506             else:
507                 previous_quantity = current_bid['quantity']
508                 previous_face = current_bid['face']
509
510                 #Determining whether or not the bid has been raised
511                 if bid_count > previous_quantity or (bid_count ==
previous_quantity and bid_value > previous_face):
512                     game_data['current_bid'] = {'quantity': bid_count, 'face':
bid_value}
513
514                 #Incrementing the current player or looping back to the
first player
515                 game_data['current_player'] = (current_player + 1) % len(
players)
516                 request.session['game_data'] = game_data
517                 return redirect('bot_game')
518             else:
519
520                 #Alert the user that the bid must be raised
521                 request.session['error_message'] = "Invalid bid! You must
bid higher than the previous bid."
```

```

522         else:
523
524             #If the bid value and count are valid, set the new bid as the
525             current bid and move on to the next player
526             if bid_count > 0 and 1 <= bid_value <= 6:
527                 game_data['current_bid'] = {'quantity': bid_count, 'face':
bid_value}
528                 game_data['current_player'] = (current_player + 1) % len(
players)
529                 request.session['game_data'] = game_data
530                 return redirect('bot_game')
531
532             #Otherwise return an error message and force the user to re-enter
a bid
533             else:
534                 request.session['error_message'] = "Invalid bid! Make sure to
enter a valid number and dice face."
535
536                 request.session['game_data'] = game_data
537                 return redirect('bot_game')
538
539             #Calling a Bluff
540             elif action == 'call_bluff':
541                 if current_bid:
542                     #Establishing who the previous player was who made the current bid
543                     previous_player = (current_player - 1) % len(players)
544                     previous_name = players[previous_player]['name']
545
546                     #Uses the bluff_successful function to determine if the bluff call
works
547                     if bluff_successful(game_data):
548
549                         #If it works, remove one dice from the previous player
550                         players[previous_player]['dice'] -= 1
551
552                         #*
553
554                         game_data['bot_action_message'] = f"{players[current_player]['
name']}] called bluff successfully! {previous_name} loses a die."
555
556
557                     else:
558
559                         #If the bluff call is unsuccessful, the current player should
lose a dice
560                         players[current_player]['dice'] -= 1
561
562                         game_data['bot_action_message'] = f"{players[current_player]['
name']}] called bluff incorrectly and loses a die."
563
564                         #Ensure the round is marked as having ended
565                         game_data['round_ended'] = True
566                         request.session['game_data'] = game_data
567                     else:
568                         request.session['error_message'] = "Invalid Bluff Call! There is
no bid to call bluff on."
569
570                 return redirect('bot_game')

```

```
571     #Ending a round
572     elif action == 'end_round':
573
574         #Goes through each player and eliminates them if they have no dice
575         game_data['players'] = []
576         for player in players:
577             if player['dice'] > 0:
578                 game_data['players'].append(player)
579
580         real_player_num = 0
581         for player in players:
582             if 'Player' in player['name']:
583                 real_player_num += 1
584
585
586
587         #Checking to see if either the game is over because only one real
588         player has dice, or if no real players are left
589         if len(game_data['players']) == 1 or real_player_num == 0:
590             winner = game_data['players'][0]['name']
591
592             request.session['game_winner'] = winner
593             return redirect('game_over')
594
595         #Resetting the game data for the next round
596         game_data['current_bid'] = None
597         game_data['round_ended'] = False
598
599         game_data['current_player'] = (current_player + 1) % len(game_data['
600         players'])
601
602         #Making sure all player's dice are reset for next round
603         for player in game_data['players']:
604             player['rolled'] = False
605             player['rolled_dice'] = []
606
607         request.session['game_data'] = game_data
608         return redirect('bot_game')
609
610         #Now to handle Bot Actions, if the current player is a bot, and the round has
611         not ended (in which case the player needs to press end_round)
612         if 'Bot' in players[current_player]['name'] and not game_data.get('round_ended
613         ', False):
614
615             #If round has already ended, stop any more bot turns
616             if game_data.get('round_ended', False):
617                 return redirect('bot_game')
618
619             #If the bot has not rolled their dice, we now do this, before setting
620             rolled to true
621             if not players[current_player].get('rolled', False):
622                 rolled_dice = []
623                 for i in range(players[current_player]["dice"]):
624                     rolled_dice.append(random.randint(1, 6))
625                 players[current_player]['rolled_dice'] = rolled_dice
626                 players[current_player]['rolled'] = True
627
628             #We pass the current game_data over to the bot_turn function, which will
```

```
return an action message and determine whether or not the round has ended. The
bot turn function will also make the necessary changes to game data
625     action_message, round_ended = bot_turn(game_data)
626
627     #More debug statements
628     print(action_message)
629
630     game_data['round_ended'] = round_ended
631     game_data['bot_action_message'] = action_message
632
633     #Now we wave the game data in the session
634     request.session['game_data'] = game_data
635
636     return redirect('bot_game')
637
638     #Retrieve the error message stack from the session and pop the most recent
message off (previous messages are irrelevant)
639     error_message = request.session.pop('error_message', None)
640
641     #Now we pass all of the following data, for the html to manage and use to
correctly display the game
642     return render(request, 'game/bot_game.html', {
643         'game_data': game_data,
644         'current_bid': game_data.get('current_bid', {}),
645         'players': players,
646         'current_player_name': players[current_player]['name'],
647         'current_player_dice': players[current_player].get('rolled_dice', []),
648         'rolled': players[current_player].get('rolled', False),
649         'round_ended': game_data.get('round_ended', False),
650         'bot_action_message': game_data.get('bot_action_message', ''),
651         'error_message' : error_message
652     })
653
654 #Handles the bot's turn and decision making
655 def bot_turn(game_data):
656
657     #Retrieving all the needed game data
658     players = game_data['players']
659     current_player = game_data['current_player']
660     bot = players[current_player]
661     current_bid = game_data.get('current_bid', None)
662
663     #If there is no current bid we must make a bid
664     if not current_bid:
665
666         #To allow the bot to use some basic strategy, we calculate the most common
value in its dice and its quantity, so we can make a good bid
667         dice_values = {}
668         for die in bot['rolled_dice']:
669             dice_values[die] = dice_values.get(die, 0) + 1
670
671         #Use the max function to calculate the most common value
672         most_common_value = max(dice_values, key=dice_values.get)
673         quantity = dice_values[most_common_value]
674
675         #Now we can slightly inflate the quantity to make the bot more
unpredictable
676         quantity = max(1, quantity)
677
```

```
678     #Now we must increment the current player and set the current bid to be
displayed
679     game_data['current_bid'] = {'quantity': quantity, 'face':
most_common_value}
680     game_data['current_player'] = (current_player + 1) % len(players)
681
682     #The functions returns this bot action message to be displayed
683     return f"{bot['name']} started the bidding with {quantity} x {
most_common_value}.", False
684
685     #Now if there is already a bid, we randomly decide to either raise the bid or
call a bluff, with an 80% chance we bid and 20% chance of bluffing, to make the
game more fun/realistic
686     decision = random.choices(['bid', 'bluff'], weights=[0.8, 0.2])[0]
687
688     #If the bot decides to raise the bid, we now calculate the new bid
689     if decision == 'bid':
690
691         #Keeping track of the current bid and face, so ours is higher
692         prev_quantity = current_bid['quantity']
693         prev_face = current_bid['face']
694
695         #Again, use the find the most common value
696         dice_counts = {}
697         for die in bot['rolled_dice']:
698             dice_counts[die] = dice_counts.get(die, 0) + 1
699         most_common_value = max(dice_counts, key=dice_counts.get)
700
701         #If the previous value is less than the most common value in the bot's
dice, we simply raise this bit with the same count to the common value.
702         if prev_face < most_common_value:
703             new_quantity = prev_quantity
704             new_face = most_common_value
705
706         #Otherwise, we increase the count by 1 and set the value to the most
common value to maintain the bot's chance of winning
707         else:
708             new_quantity = prev_quantity + 1
709             new_face = most_common_value
710
711         #Now we set the current bid and increment the player
712         game_data['current_bid'] = {'quantity': new_quantity, 'face': new_face}
713         game_data['current_player'] = (current_player + 1) % len(players)
714
715         #Return the corresponding bot action message
716         return f"{bot['name']} raised the bid to {new_quantity} x {new_face}.",
False
717
718     #Handling the bluff logic
719     elif decision == 'bluff':
720
721         #Determining the player who called the bid
722         previous_index = (current_player - 1) % len(players)
723         previous_player = players[previous_index]
724
725         #Use the bluff_successful function to see if the bluff call works or fails
726         if bluff_successful(game_data):
727
728             #If the bluff call is successful we must remove a die from the
```

```
previous player and return the appropriate action message
729     players[previous_index]['dice'] -= 1
730     action_message = f"{bot['name']} called bluff successfully! {
previous_player['name']} loses a die."
731     else:
732
733         #Otherwise, we remove one of the current player's dice and display a
different action message
734         bot['dice'] -= 1
735         action_message = f"{bot['name']} called bluff incorrectly and loses a
die."
736
737     #Now we must remove any eliminated players from the game, by checking if
they have dice
738     game_data['players'] = []
739     for player in players:
740         if player['dice'] > 0:
741             game_data['players'].append(player)
742
743     #Checking if there is only one player remaining and game is over
744     if len(game_data['players']) <= 1:
745         game_data['game_over'] = True
746         winner = game_data['players'][0]['name']
747         game_data['game_winner'] = winner
748         return f"Game Over! {winner} wins.", True
749
750     #Reset the current bid and dice status of each player since round has
ended
751     game_data['current_bid'] = None
752     for player in game_data['players']:
753         player['rolled'] = False
754         player['rolled_dice'] = []
755
756     #Now we return the action message and that the round has ended
757     return action_message, True
758
759
760
761 #Defining the function used to determine if a bluff call is successful or not
762 def bluff_successful(game_data):
763
764     #Appending every dice value from all players to total dice
765     total_dice = []
766     for player in game_data['players']:
767         for value in player.get('rolled_dice', []):
768             total_dice.append(value)
769
770     #Now we compare the actual number of the bid value in the game, to the
quantity of the bid, returning a Boolean value
771     bid = game_data['current_bid']
772     return total_dice.count(bid['face']) < bid['quantity']
773
774
775
776
777 #View for the game rules page. No variable information is required so this view
can remain relatively empty
778 def game_rules(request):
779     return render(request, 'game/game_rules.html')
```

```
780
781 #View for the view profile page
782 def view_profile(request):
783
784     #Now we retrieve the user's details to display and also to use throughout the
change password process
785     conn = get_db_connection()
786     cursor = conn.cursor(dictionary=True)
787     cursor.execute("SELECT username, password, email, games_played, wins FROM User
WHERE user_id = %s", (request.session['user_id'],))
788     user_data = cursor.fetchone()
789     cursor.close()
790     conn.close()
791
792
793
794     if request.method == 'POST':
795
796         #Collecting the information entered by the user
797         old_password = request.POST.get('old_password')
798         new_password = request.POST.get('new_password')
799         confirm_password = request.POST.get('confirm_password')
800
801         #Checking to see if the user's old password entry is correct
802         if hashing_algorithm(old_password) != user_data['password']:
803             messages.error(request, "Old password is incorrect.")
804
805         #Ensuring the confirmed password matches the new password
806         elif new_password != confirm_password:
807             messages.error(request, "New passwords do not match.")
808         else:
809
810             #If the user has entered valid information, we now update the backend
database with the new password
811             conn = get_db_connection()
812             cursor = conn.cursor(dictionary = True)
813             new_hashed_password = hashing_algorithm(new_password)
814             cursor.execute("UPDATE User SET password = %s WHERE user_id = %s", (
new_hashed_password, request.session['user_id']))
815             conn.commit()
816             cursor.close()
817             conn.close()
818
819             #Making sure to also update the session
820             request.session['password'] = hashing_algorithm(new_password)
821             request.session['unhashed_password'] = new_password
822             messages.success(request, "Password changed succesfully!")
823             return redirect('view_profile')
824
825
826     #Here we make sure to give the html access to the necessary information for
the view profile page
827     return render(request, 'game/view_profile.html', {'user': user_data, '
unhashed_password': request.session['unhashed_password']})
828
829 #View for the leaderboard page
830 def view_leaderboard(request):
831
832     #retrieve all the required information from the database about each user
```

```
833 conn = get_db_connection()
834 cursor = conn.cursor(dictionary=True)
835 cursor.execute("SELECT username, games_played, wins FROM User")
836 users = cursor.fetchall()
837 cursor.close()
838 conn.close()
839
840 #Debug Statement
841 print(users)
842
843 #Now we can create an empty array of dictionaries to add user information to
844 user_list = []
845 for user in users:
846     username, games_played, wins = user['username'], user['games_played'],
user['wins']
847
848     #Avoiding any division by 0 errors if games played = 0
849     if games_played > 0:
850         win_pct = (wins/games_played)*100
851     else:
852         win_pct = 0
853
854     user_list.append({"username": username, "games_played": games_played, "
wins": wins, "win_pct": win_pct})
855
856 #Initializing the empty lists of each statistic, which can later be sorted
857 wins_list = []
858 win_pct_list = []
859 games_played_list = []
860
861 #Now we append each game statistic from each user to the corresponding
initially empty array, which we will later use to sort the information by these
lists.
862 for user in user_list:
863     games_played_list.append(user['games_played'])
864     wins_list.append(user['wins'])
865     win_pct_list.append(user['win_pct'])
866
867 #Retrieve the user's sorting preference from the webpage check boxes,
defaulting to sorting by wins
868 sort_by = request.GET.get('sort', 'wins')
869
870
871
872 #For each sorting option we now use the sort_users_by_key function, which
returns a correctly sorted list of users
873 if sort_by == "wins":
874     sorted_user_list = sort_users_by_key(user_list, wins_list, 'wins')
875
876 #We now simply repeat the process for the other sorting options
877 elif sort_by == "games":
878     sorted_user_list = sort_users_by_key(user_list, games_played_list, '
games_played')
879
880 elif sort_by == "win_pct":
881     sorted_user_list = sort_users_by_key(user_list, win_pct_list, 'win_pct')
882
883 return render(request, 'game/view_leaderboard.html', {'users':
sorted_user_list, 'sort_by': sort_by})
```

```
884
885 #Function used to return a sorted list of users depending on a key, to be used in
    the leaderboard view
886 def sort_users_by_key(user_list, stats_list, key):
887     #Initialize an empty list to store the correctly sorted user list
888     sorted_user_list = []
889
890     #Now we use the mergeSort algorithm to sort the list of stats
891     sorted_stats = mergeSort(stats_list)
892
893     #Creating a copy of the user_list so i can modify it and remove already sorted
    users later on
894     remaining_users = user_list.copy()
895
896     #For each value in the sorted stat list, we find the first corresponding user
    who has that same statistics, appending it to the sorted list.
897     for stat in sorted_stats:
898         for user in remaining_users:
899             if user[key] == stat:
900                 next_record = user
901                 break
902             sorted_user_list.append(next_record)
903
904     #Now we must make sure to remove this user from the list, so if two
    players share the same stat the first player is not repeated instead of the
    next one
905     remaining_users.remove(next_record)
906     return sorted_user_list
907
908
909 #Setting up the merging algorithms
910 def merge(array1, array2):
911
912     #Making sure null values do not crash the algorithms
913     if array1 is None:
914         array1 = []
915     if array2 is None:
916         array2 = []
917
918     #Creating an empty list to add sorted elements to
919     merged_list = []
920     array1_index = 0
921     array2_index = 0
922
923     #Merging each sorted list
924     while array1_index < len(array1) and array2_index < len(array2):
925
926         #Comparing the current array1 value with the array 2 value and appending
    it to the merged list if it is greater
927         if array1[array1_index] <= array2[array2_index]:
928             merged_list.append(array1[array1_index])
929
930         #Now we make sure to increment the array1_index to move onto the next
    term
931         array1_index += 1
932
933         #If the current array1 value is greater than the array2 value, we simply
    do the opposite
934         else:
```

```
935         merged_list.append(array2[array2_index])
936         array2_index +=1
937
938     #If there are any leftover elements in the first array we simply append the
939     #rest to the merged list
940     while array1_index < len(array1):
941         merged_list.append(array1[array1_index])
942         array1_index +=1
943
944     #Now we do the same for array2
945     while array2_index < len(array2):
946         merged_list.append(array2[array2_index])
947         array2_index +=1
948
949     return merged_list
950
951 def mergeSort(array):
952     if array is None:
953         return []
954
955     #If the array is 1 long, then it is already sorted so simply return it
956     if len(array) <=1:
957         return array
958
959     #Next break the array down into two separate halves, storing the index of the
960     #midpoint
961     midpoint = len(array)//2
962
963     #We use recursion to continue breaking this down until each list has 1 term
964     array1 = mergeSort(array[midpoint:])
965     array2 = mergeSort(array[:midpoint])
966
967     #Now we merge the final two sorted halves into the final sorted list to be
968     #returned
969     return merge(array1, array2)
970
971 #View for the Game-over page
972 def game_over(request):
973
974     #Retrieve the winner of the game from the session
975     winner = request.session.get('game_winner')
976
977     #Only update user's wins if the winner is player 1, as they have one the game
978     if winner == 'Player 1':
979
980         #Use the following SQL query to update the User table, incrementing wins
981         conn = get_db_connection()
982         cursor = conn.cursor()
983         cursor.execute("UPDATE User SET wins = wins + 1 WHERE user_id = %s", (
984             request.session['user_id'], ))
985         conn.commit()
986         cursor.close()
987         conn.close()
988
989     #We now have to clear the game session for any future games
990     request.session.pop('game_winner', None)
991     request.session.pop('game_data', None)
```

```
990  
991     return render(request, 'game/game_over.html', {'winner': winner})
```

Listing 11: Views.py file

3.5 The HTML Templates

The templates folder holds many HTML files corresponding to the views in the views.py file, which are responsible for the way in which information processed by the views is displayed to the user, as well as the general interface and visual appeal of the website.

```
1  <!DOCTYPE html>  
2  <html>  
3  <head>  
4      <title>Perudo WebGame</title>  
5      <!-- Here are the css style tags for this page-->  
6      <style>  
7          body {  
8              font-family: Arial, sans-serif;  
9              background-color: #f4f4f9;  
10             margin: 0;  
11             padding: 0;  
12             display: flex;  
13             justify-content: center;  
14             align-items: center;  
15             height: 100vh;  
16         }  
17  
18         .container {  
19             width: 80%;  
20             max-width: 1200px;  
21             display: grid;  
22             grid-template-columns: 1fr 1fr;  
23             gap: 20px;  
24             background-color: white;  
25             padding: 20px;  
26             border: 1px solid #ccc;  
27             border-radius: 8px;  
28             box-shadow: 0px 4px 8px rgba(0, 0, 0, 0.1);  
29         }  
30  
31         .title {  
32             grid-column: span 2;  
33             text-align: center;  
34             margin-bottom: 20px;  
35         }  
36  
37         .title h1 {  
38             margin: 0;  
39             font-size: 2rem;  
40         }  
41  
42         .login-section, .register-section {  
43             display: flex;  
44             flex-direction: column;  
45             justify-content: center;  
46             align-items: center;  
47         }  
48
```

```
49     .login-section h2, .register-section h2 {
50         margin-bottom: 20px;
51     }
52
53     .form-group {
54         margin-bottom: 15px;
55         width: 100%;
56         max-width: 300px;
57     }
58
59     .form-group label {
60         display: block;
61         margin-bottom: 5px;
62     }
63
64     .form-group input {
65         width: 100%;
66         padding: 10px;
67         font-size: 14px;
68         border: 1px solid #ccc;
69         border-radius: 4px;
70     }
71
72     .error-message {
73         margin-top: 20px;
74         color: red;
75         text-align: center;
76     }
77
78     .button-group {
79         display: flex;
80         justify-content: space-between;
81         width: 100%;
82         max-width: 300px;
83     }
84
85     .button-group button {
86         padding: 10px 20px;
87         font-size: 14px;
88         border: 1px solid #ccc;
89         border-radius: 4px;
90         cursor: pointer;
91         background-color: white;
92     }
93
94     .button-group button:hover {
95         background-color: #f4f4f4;
96     }
97 </style>
98 </head>
99 <body>
100     <div class="container">
101         <!-- Page Title -->
102         <div class="title">
103             <h1>Perudo WebGame</h1>
104         </div>
105
106         <!-- Section for user to enter details and login -->
107         <div class="login-section">
```

```

108     <h2>Have an account?</h2>
109     <form method="post">
110         <!-- Using django to prevent against csrf attacks-->
111         {% csrf_token %}
112         <!-- Display any error messages that might have occurred -->
113         {% if messages %}
114             <div class="error-message">
115                 {% for message in messages %}
116                     <p>{{ message }}</p>
117                 {% endfor %}
118             </div>
119         {% endif %}
120
121         <div class="form-group">
122             <label for="username">Username</label>
123             <input type="text" id="username" name="username" value="{{
request.POST.username }}" required>
124         </div>
125         <div class="form-group">
126             <label for="password">Password</label>
127             <input type="password" id="password" name="password" required>
128         </div>
129         <button type="submit">Login</button>
130     </form>
131 </div>
132
133 <!-- Register Section -->
134 <div class="register-section">
135     <h2>If not, Register</h2>
136     <!-- If the register button is pressed, this redirects the user to the
registration page using the 'register' url -->
137     <form action="{% url 'register' %}" method="get">
138         <button type="submit">Create an account</button>
139     </form>
140 </div>
141 </div>
142 </body>
143 </html>

```

Listing 12: home.html file

```

1 <!DOCTYPE html>
2 <html>
3 <head>
4     <title>Register</title>
5     <!-- Setting out the css style tags for this page-->
6     <style>
7         body {
8             font-family: Arial, sans-serif;
9             background-color: #f4f4f9;
10            margin: 0;
11            padding: 0;
12            display: flex;
13            justify-content: center;
14            align-items: center;
15            height: 100vh;
16        }
17
18        .container {

```

```
19         width: 500px;
20         padding: 20px;
21         background-color: white;
22         border: 1px solid #ccc;
23         border-radius: 8px;
24         box-shadow: 0px 4px 8px rgba(0, 0, 0, 0.1);
25     }
26
27     .header {
28         display: flex;
29         justify-content: space-between;
30         margin-bottom: 20px;
31     }
32
33
34
35
36     form {
37         display: flex;
38         flex-direction: column;
39         gap: 15px;
40     }
41
42     label {
43         font-weight: bold;
44     }
45
46     input {
47         width: 95%;
48         padding: 10px;
49         font-size: 14px;
50         border: 1px solid #ccc;
51         border-radius: 4px;
52     }
53
54 </style>
55 </head>
56 <body>
57     <div class="container">
58         <div class="header">
59             <!-- Setting up the cancel button -->
60             <form action="{% url 'home' %}" method="get" style="margin: 0;">
61                 <button type="submit" class="cancel">Cancel</button>
62             </form>
63
64             <!-- Setting the register button to submit the form -->
65             <button type="submit" form="registerForm" class="submit">Register</
button>
66         </div>
67         <!-- Display any error messages if there are any-->
68         {% if messages %}
69             <div class="error-message">
70                 {% for message in messages %}
71                     <p>{{ message }}</p>
72                 {% endfor %}
73             </div>
74         {% endif %}
75         <!-- Setting up the form for the user to enter their registration details
-->
```

```
76     <form id="registerForm" method="post">
77         <!-- Using django to prevent against csrf attacks-->
78         {% csrf_token %}
79         <label for="email">Email</label>
80         <input type="text" id="email" name="email" required>
81
82         <label for="username">Username</label>
83         <input type="text" id="username" name="username" required>
84
85         <label for="password">Password</label>
86         <input type="password" id="password" name="password" required>
87
88         <label for="confirm_password">Confirm Password</label>
89         <input type="password" id="confirm_password" name="confirm_password"
required>
90     </form>
91 </div>
92 </body>
```

Listing 13: register.html file

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4     <!-- setting up css style tags for the page-->
5     <style>
6         body {
7             font-family: Arial, sans-serif;
8             background-color: #f4f4f9;
9             margin: 0;
10            padding: 0;
11            display: flex;
12            flex-direction: column;
13            align-items: center;
14            justify-content: center;
15            height: 100vh;
16        }
17
18        .container {
19            width: 80%;
20            max-width: 1000px;
21            text-align: center;
22            position: relative;
23        }
24
25        h1 {
26            font-size: 2.5rem;
27            margin-bottom: 20px;
28            color: #333;
29        }
30
31        .username-box {
32            position: absolute;
33            top: 20px;
34            right: 20px;
35            border: 1px solid #ccc;
36            padding: 10px 20px;
37            border-radius: 8px;
38            background-color: white;
```

```
39     font-size: 1.2rem;
40   }
41
42   .button-container {
43     display: flex;
44     flex-direction: column;
45     gap: 20px;
46     margin-top: 40px;
47   }
48
49   .button-container a {
50     display: inline-block;
51     padding: 15px 30px;
52     font-size: 1.5rem;
53     color: black;
54     text-decoration: none;
55     border: 1px solid #ccc;
56     border-radius: 8px;
57     background-color: white;
58   }
59
60   </style>
61 </head>
62 <body>
63   <div class="container">
64     <!-- Display the username of the currently logged in user-->
65     <div class="username-box">
66       {{ user.username }}
67     </div>
68
69     <!-- Page title -->
70     <h1>Main Menu</h1>
71
72     <!-- Providing all the buttons using urls to all the different pages-->
73     <div class="button-container">
74       <a href="{% url 'setup_pass_and_play' %}">Pass and Play</a>
75       <a href="{% url 'setup_bot_game' %}">Play with Bots</a>
76       <a href="{% url 'game_rules' %}">Game Rules and Tutorial</a>
77       <a href="{% url 'view_leaderboard' %}">Leaderboard</a>
78       <a href="{% url 'view_profile' %}">View Profile</a>
79       <a href="{% url 'home' %}">Log Out</a>
80
81     </div>
82   </div>
83 </body>
84 </html>
```

Listing 14: main_menu.html file

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>View Profile</title>
5   <!-- Setting up the css style tags for the page-->
6   <style>
7     body {
8       font-family: Arial, sans-serif;
9       background-color: #f4f4f9;
10      text-align: center;
```

```
11         padding: 50px;
12     }
13     .container {
14         max-width: 500px;
15         margin: auto;
16         background: white;
17         padding: 20px;
18         border: 1px solid #ccc;
19         border-radius: 8px;
20         box-shadow: 0px 4px 8px rgba(0, 0, 0, 0.1);
21     }
22     h1 {
23         font-size: 2rem;
24         margin-bottom: 20px;
25     }
26     .profile-info {
27         text-align: left;
28         margin-bottom: 20px;
29     }
30     .profile-info p {
31         margin: 5px 0;
32     }
33     form {
34         text-align: left;
35     }
36     label {
37         display: block;
38         margin-bottom: 5px;
39         font-weight: bold;
40     }
41     input {
42         width: 100%;
43         padding: 10px;
44         margin-bottom: 15px;
45         border: 1px solid #ccc;
46         border-radius: 4px;
47     }
48
49     </style>
50 </head>
51 <body>
52     <div class="container">
53         <h1>Your Profile</h1>
54
55         <!-- Displaying the user's username and unhashed password -->
56         <div class="profile-info">
57             <p><strong>Username:</strong> {{ user.username }}</p>
58             <p><strong>Password</strong> {{ unhashed_password }}</p>
59         </div>
60
61         <!-- Displaying any error messages if there are any-->
62         {% if messages %}
63             <div class="Message">
64                 {% for message in messages %}
65                     <p>{{ message }}</p>
66                 {% endfor %}
67             </div>
68         {% endif %}
69
```

```
70     <!-- Setting up the form for the user to enter their password details to
change password -->
71     <form method="post">
72         {% csrf_token %}
73         <label for="old_password">Old Password</label>
74         <input type="password" id="old_password" name="old_password" required>
75
76         <label for="new_password">New Password</label>
77         <input type="password" id="new_password" name="new_password" required>
78
79         <label for="confirm_password">Confirm New Password</label>
80         <input type="password" id="confirm_password" name="confirm_password"
required>
81
82         <button type="submit">Change Password</button>
83     </form>
84
85     <!-- Setting up back to main menu button, using the main_menu url -->
86     <a href="{% url 'main_menu' %}">Back to Main Menu</a>
87 </div>
88 </body>
89 </html>
```

Listing 15: view_profile.html file

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4     <!-- Setting up the css style tags for the page-->
5     <style>
6         body {
7             font-family: Arial, sans-serif;
8             margin: 20px;
9             text-align: center;
10        }
11        .main-container {
12            display: flex;
13            flex-direction: column;
14            align-items: center;
15        }
16        .leaderboard-table {
17            border-collapse: collapse;
18            width: 60%;
19            min-width: 500px;
20        }
21        th, td {
22            border: 2px solid black;
23            padding: 10px;
24            text-align: center;
25        }
26        th {
27            background-color: #f4f4f4;
28            font-weight: bold;
29        }
30        .sort-options {
31            margin-top: 10px;
32        }
33        .checkbox-label {
34            display: flex;
```

```

35         align-items: center;
36         gap: 5px;
37     }
38 </style>
39 </head>
40 <body>
41     <div class="main-container">
42         <!-- Title -->
43         <h1>Leaderboard</h1>
44
45         <!-- Setting up the leaderboard table, with each row containing a user's
46         username, wins, games and win pct -->
47         <table class="leaderboard-table">
48             <tr>
49                 <th>Username</th>
50                 <th>Wins</th>
51                 <th>Games</th>
52                 <th>Win Percentage</th>
53             </tr>
54             <!-- Iterate through the sorted user list, displaying each user's
55             stats in the correct order -->
56             {% for user in users %}
57             <tr>
58                 <td>{{ user.username }}</td>
59                 <td>{{ user.wins }}</td>
60                 <td>{{ user.games_played }}</td>
61                 <td>{{ user.win_pct|floatformat:1 }}%</td>
62             </tr>
63             {% endfor %}
64         </table>
65
66         <!-- Giving the user to select how they would like to sort the leaderboard
67         , using checkboxes -->
68         <div class="sort-options">
69             <p><strong>Sort By:</strong></p>
70             <form method="get">
71                 <label class="checkbox-label">
72                     <input type="radio" name="sort" value="wins" onchange="this.
73                     form.submit()" {% if sort_by == 'wins' %}<checked{% endif %}>
74                     Wins
75                 </label>
76                 <label class="checkbox-label">
77                     <input type="radio" name="sort" value="games" onchange="this.
78                     form.submit()" {% if sort_by == 'games' %}<checked{% endif %}>
79                     Games
80                 </label>
81                 <label class="checkbox-label">
82                     <input type="radio" name="sort" value="win_pct" onchange="this
83                     .form.submit()" {% if sort_by == 'win_pct' %}<checked{% endif %}>
84                     Win %
85                 </label>
86             </form>
87         </div>
88
89         <!-- Allowing the user to use main_menu url to return to menu -->
90         <a href="{% url 'main_menu' %}">Back to Main Menu</a>
91     </body>

```

86 </html>

Listing 16: view_leaderboard.htmlfile

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Game Rules and Tutorial</title>
5   <!-- Setting up the css style tags-->
6   <style>
7     body {
8       font-family: Arial, sans-serif;
9       background-color: #f4f4f9;
10      padding: 50px;
11      line-height: 1.6;
12    }
13    .container {
14      max-width: 800px;
15      margin: auto;
16      background: white;
17      border: 1px solid #ccc;
18      border-radius: 8px;
19      padding: 20px;
20      box-shadow: 0px 4px 8px rgba(0, 0, 0, 0.1);
21    }
22    h1 {
23      font-size: 2rem;
24      text-align: center;
25      color: #333;
26      margin-bottom: 20px;
27    }
28    h2 {
29      font-size: 1.5rem;
30      color: #333;
31      margin-top: 20px;
32    }
33    p {
34      font-size: 1rem;
35      color: #666;
36      margin-bottom: 20px;
37    }
38  </style>
39 </head>
40 <body>
41   <div class="container">
42     <!-- Displaying the relevant advice and rules for the user to read-->
43     <h1>Game Rules and Tutorial</h1>
44
45     <h2>How to Play</h2>
46     <p>Every player starts with a pre-selected number of dice and players take
47       turns making bids over the number of dice with a certain value in the game.
48       Each turn the player can either raise the current bid (or make a bid if no bid
49       has been made) or call the previous player's bluff. If a bluff call is
50       successful, the player who made the bid loses a dice and a new round begins. If
51       it is unsuccessful and their bid turns out to be true, the current player who
52       called the bluff loses a dice, and the game continues.</p>
53
54     <h2>Winning the Game</h2>
```

```

50     <p>You win the game by outlasting your opponents and being the last player
      in the game with any dice left.</p>
51
52     <h2>Additional Tips</h2>
53     <p>When playing pass and play with friends, make sure to do your best to
      trick your opponents into losing their dice against you. Bluffing, double
      bluffing or even triple bluffing can be an extremely effective way of coercing
      your opponents into making poor decisions. Remember: play the man, not the game
      .</p>
54
55     <a href="{% url 'main_menu' %}">Back to Main Menu</a>
56
57     <h3> If you are still unsure of the rules of Liar's dice, watch this video
      by 'Gather Together Games' on Youtube.</h3>
58
59     <!-- Setting up an embedded youtube video for the user to watch-->
60     <iframe width="560" height="315" src="https://www.youtube.com/embed/
      fAbnMuiR734?si=uE4xQQmvmTNBgOlQ" title="YouTube video player" frameborder="0"
      allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope;
      picture-in-picture; web-share" referrerpolicy="strict-origin-when-cross-origin"
      allowfullscreen></iframe>
61
62 </div>
63
64
65 </body>
66 </html>

```

Listing 17: game_rules.html file

```

1 <!DOCTYPE html>
2 <html>
3 <head>
4     <title>Pass and Play Setup</title>
5     <!-- Here are the css style tags for this page-->
6     <style>
7         body {
8             font-family: Arial, sans-serif;
9             background-color: #f4f4f9;
10            margin: 0;
11            padding: 0;
12            display: flex;
13            justify-content: center;
14            align-items: center;
15            height: 100vh;
16        }
17
18        .container {
19            text-align: center;
20            background-color: white;
21            padding: 30px;
22            border: 1px solid #ccc;
23            border-radius: 8px;
24            box-shadow: 0px 4px 8px rgba(0, 0, 0, 0.1);
25            width: 400px;
26        }
27
28        h1 {
29            font-size: 2rem;

```

```
30     margin-bottom: 20px;
31 }
32
33 .form-group {
34     margin-bottom: 20px;
35     text-align: left;
36 }
37
38 .form-group label {
39     display: inline-block;
40     font-size: 1rem;
41     margin-bottom: 10px;
42 }
43
44 .form-group select {
45     width: 100%;
46     padding: 10px;
47     font-size: 1rem;
48     border: 1px solid #ccc;
49     border-radius: 4px;
50 }
51
52 button {
53     padding: 15px 30px;
54     font-size: 1rem;
55     border: none;
56     border-radius: 4px;
57     background-color: white;
58     border: 1px solid #ccc;
59     cursor: pointer;
60     margin-top: 20px;
61 }
62
63 button:hover {
64     background-color: #f4f4f4;
65 }
66
67 button:active {
68     transform: scale(0.98);
69 }
70 </style>
71 </head>
72 <body>
73     <div class="container">
74         <h1>Pass and Play</h1>
75         <form method="post">
76             <!-- Using django to prevent against csrf attacks-->
77             {% csrf_token %}
78             <!-- Setting up the form for initial dice choicse, only allowing valid
79             inputs-->
80             <div class="form-group">
81                 <label for="initial_dice">Please Select Number of Initial Dice:</
82                 label>
83                 <select id="initial_dice" name="initial_dice" required>
84                     <option value="1">1</option>
85                     <option value="2">2</option>
86                     <option value="3">3</option>
87                     <option value="4">4</option>
88                     <option value="5">5</option>
```

```

87         <option value="5">6</option>
88
89     </select>
90 </div>
91 <!-- Setting up the form for initial number of players, only allowing
a reasonable number of players, given the space on the screen-->
92 <div class="form-group">
93     <label for="num_players">Please Select Number of Players:</label>
94     <select id="num_players" name="num_players" required>
95         <option value="2">2</option>
96         <option value="3">3</option>
97         <option value="4">4</option>
98         <option value="5">5</option>
99         <option value="6">6</option>
100     </select>
101 </div>
102
103 <!-- Allowing user to start game -->
104 <button type="submit">Start Game</button>
105 </form>
106 </div>
107 </body>
108 </html>

```

Listing 18: setup_pass_and_play.html file

```

1 <!DOCTYPE html>
2 <html>
3 <head>
4     <title>Play with Bots Setup</title>
5     <!-- Here are the css style tags for this page-->
6     <style>
7         body {
8             font-family: Arial, sans-serif;
9             text-align: center;
10            background-color: #f4f4f9;
11            padding: 50px;
12        }
13
14        .container {
15            max-width: 400px;
16            margin: auto;
17            background: white;
18            padding: 20px;
19            border: 1px solid #ccc;
20            border-radius: 8px;
21            box-shadow: 0px 4px 8px rgba(0, 0, 0, 0.1);
22        }
23
24        h1 {
25            font-size: 2rem;
26            margin-bottom: 20px;
27        }
28
29        form {
30            text-align: left;
31        }
32
33        label {

```

```

34         font-weight: bold;
35     }
36
37     select {
38         width: 100%;
39         padding: 10px;
40         margin-bottom: 20px;
41         border: 1px solid #ccc;
42         border-radius: 4px;
43     }
44
45
46 </style>
47 </head>
48 <body>
49     <div class="container">
50         <h1>Play with Bots</h1>
51         <form method="post">
52             <!-- Using django to prevent against csrf attacks-->
53             {% csrf_token %}
54             <!-- Setting up the form for initial dice choicse, only allowing valid
inputs-->
55             <label for="initial_dice">Number of Initial Dice:</label>
56             <select id="initial_dice" name="initial_dice" required>
57                 <option value="1">1</option>
58                 <option value="2">2</option>
59                 <option value="3">3</option>
60                 <option value="4">4</option>
61                 <option value="5">5</option>
62             </select>
63
64             <!-- Setting up the form for number of bots, only allowing up to 6
players total, so the interface works properly-->
65             <label for="num_bots">Number of Bots:</label>
66             <select id="num_bots" name="num_bots" required>
67                 <option value="1">1</option>
68                 <option value="2">2</option>
69                 <option value="3">3</option>
70                 <option value="4">4</option>
71                 <option value="5">5</option>
72             </select>
73
74             <!-- Allowing user to start game -->
75             <button type="submit">Start Game</button>
76         </form>
77     </div>
78 </body>
79 </html>

```

Listing 19: setup_bot_game.html file

```

1 <!DOCTYPE html>
2 <html>
3 <head>
4     <title>Pass and Play</title>
5     <!-- Setting out the css style tags for this page -->
6     <style>
7         body {
8             font-family: Arial, sans-serif;

```

```
9      margin: 0;
10     padding: 0;
11     background-color: #f4f4f9;
12     display: flex;
13     flex-direction: column;
14     align-items: center;
15     justify-content: center;
16   }
17   .container {
18     max-width: 800px;
19     width: 90%;
20     background: white;
21     border: 1px solid #ccc;
22     border-radius: 8px;
23     padding: 20px;
24     box-shadow: 0px 4px 8px rgba(0, 0, 0, 0.1);
25     position: relative;
26   }
27   .player-bar {
28     display: flex;
29     justify-content: space-around;
30     align-items: center;
31     padding: 10px;
32     color: black;
33     border-radius: 8px;
34     margin-bottom: 20px;
35   }
36   .player {
37     display: flex;
38     flex-direction: column;
39     align-items: center;
40     margin: 0 10px;
41   }
42   .player .name {
43     font-weight: bold;
44   }
45   .player .dice-count {
46     font-size: 14px;
47   }
48   .Leave_game {
49     text-align: center;
50   }
51   h1 {
52     text-align: center;
53     margin-bottom: 20px;
54   }
55   .current-info {
56     text-align: center;
57     margin-bottom: 20px;
58   }
59   .message {
60     color: blue;
61     text-align: center;
62     margin-bottom: 20px;
63   }
64   .error-message {
65     color: red;
66     text-align: center;
67     margin-bottom: 20px;
```

```

68     }
69     .dice-values span {
70         display: inline-block;
71         margin: 0 5px;
72         padding: 5px 10px;
73         background-color: #f4f4f4;
74         border: 1px solid #ccc;
75         border-radius: 4px;
76     }
77     .actions {
78         display: flex;
79         justify-content: space-around;
80         margin-bottom: 20px;
81     }
82     .actions button {
83         padding: 10px 20px;
84         border: none;
85         border-radius: 4px;
86         cursor: pointer;
87         font-size: 16px;
88     }
89
90 </style>
91 </head>
92 <body>
93     <div class="container">
94         <!-- Setting up a player bar to show players and their dice number at top
of screen -->
95         <div class="player-bar">
96             {% for player in game_data.players %}
97                 <div class="player">
98                     <div class="name">{{ player.name }}</div>
99                     <div class="dice-count">Dice: {{ player.dice }}</div>
100                 </div>
101             {% endfor %}
102         </div>
103
104
105
106
107         <h1>Pass and Play</h1>
108
109         <!-- Display who the current player is and their bid, if there is one -->
110         <div class="current-info">
111             <p><strong>Current Player:</strong> {{ current_player_name }}</p>
112             {% if game_data.current_bid %}
113                 <p><strong>Current Bid:</strong> {{ game_data.current_bid.count }}
of {{ game_data.current_bid.value }}</p>
114             {% else %}
115                 <p>No current bid.</p>
116             {% endif %}
117         </div>
118
119         <!-- Displaying the result of a bluff call -->
120         {% if bluff_result %}
121             <div class="message">{{ bluff_result }}</div>
122         {% endif %}
123
124         <!-- Making sure to display the most recent error messages if there is one

```

```

-->
125     {% if error_message %}
126         <div class="error-message">{{ error_message }}</div>
127     {% endif %}
128
129     <!-- Allow the user to leave the game, and go back to the main menu-->
130
131     <div class='Leave_game'>
132         <a href="{% url 'main_menu' %}">Back to Main Menu</a>
133     </div>
134
135     <!-- Display the current player's dice, if they have rolled -->
136     <h2>Your Dice</h2>
137     <div class="dice-values">
138         {% if rolled %}
139             {% for value in current_player_dice %}
140                 <span>{{ value }}</span>
141             {% endfor %}
142
143         {% endif %}
144     </div>
145
146     <!-- Setting up the role dice and bluff call buttons -->
147     {% if not round_ended %}
148         <form method="POST" class="actions">
149             <!-- Using django to prevent against csrf attacks-->
150             {% csrf_token %}
151             <button type="submit" name="action" value="roll_dice">Roll Dice</
button>
152             <button type="submit" name="action" value="call_bluff">Call Bluff<
/button>
153         </form>
154
155         <!-- Setting up the form for user to enter new bid-->
156         <h2>Make a Bid</h2>
157         <form method="POST" class="make-bid">
158             <!-- Using django to prevent against csrf attacks-->
159             {% csrf_token %}
160             <label for="bid_count">Count:</label>
161             <input type="number" id="bid_count" name="bid_count" min="1"
required>
162             <label for="bid_value">Value:</label>
163             <input type="number" id="bid_value" name="bid_value" min="1" max="
6" required>
164             <button type="submit" name="action" value="make_bid">Submit Bid</
button>
165         </form>
166         {% endif %}
167
168         <!-- Only showing the end round button, if the round has ended -->
169         {% if round_ended %}
170             <form method="POST">
171                 {% csrf_token %}
172                 <button type="submit" name="action" value="end_round" class="end-
round">End Round</button>
173             </form>
174         {% endif %}
175
176     </div>

```

```
177 </body>
178 </html>
```

Listing 20: pass_and_play.html file

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Bot Game</title>
5   <!-- Setting up the css style tages for the page-->
6   <style>
7     body {
8       font-family: Arial, sans-serif;
9       margin: 0;
10      padding: 0;
11      background-color: #f4f4f9;
12      display: flex;
13      flex-direction: column;
14      align-items: center;
15      justify-content: center;
16    }
17    .container {
18      max-width: 800px;
19      width: 90%;
20      background: white;
21      border: 1px solid #ccc;
22      border-radius: 8px;
23      padding: 20px;
24      box-shadow: 0px 4px 8px rgba(0, 0, 0, 0.1);
25      position: relative;
26    }
27    .player-bar {
28      display: flex;
29      justify-content: space-around;
30      align-items: center;
31      padding: 10px;
32      color: black;
33      border-radius: 8px;
34      margin-bottom: 20px;
35    }
36    .player {
37      display: flex;
38      flex-direction: column;
39      align-items: center;
40      margin: 0 10px;
41    }
42    .player .name {
43      font-weight: bold;
44    }
45    .player .dice-count {
46      font-size: 14px;
47    }
48    .Leave_game {
49      text-align: center;
50      margin-top: 20px;
51    }
52    h1 {
53      text-align: center;
54      margin-bottom: 20px;
```

```

55     }
56     .current-info {
57         text-align: center;
58         margin-bottom: 20px;
59     }
60     .message {
61         color: blue;
62         text-align: center;
63         margin-bottom: 20px;
64     }
65     .error-message {
66         color: red;
67         text-align: center;
68         margin-bottom: 20px;
69     }
70     .dice-values span {
71         display: inline-block;
72         margin: 0 5px;
73         padding: 5px 10px;
74         background-color: #f4f4f4;
75         border: 1px solid #ccc;
76         border-radius: 4px;
77     }
78     .actions {
79         display: flex;
80         justify-content: space-around;
81         margin-bottom: 20px;
82     }
83     .actions button {
84         padding: 10px 20px;
85         border: none;
86         border-radius: 4px;
87         cursor: pointer;
88         font-size: 16px;
89     }
90 </style>
91 </head>
92 <body>
93     <div class="container">
94         <!-- Setting up the player bar at the top, to display each remaning player
95         and their dice left-->
96         <div class="player-bar">
97             {% for player in game_data.players %}
98                 <div class="player">
99                     <div class="name">{{ player.name }}</div>
100                     <div class="dice-count">Dice: {{ player.dice }}</div>
101                 </div>
102             {% endfor %}
103         </div>
104         <h1>Bot Game</h1>
105
106         <!-- Displays the current player and the current bid -->
107         <div class="current-info">
108             <p><strong>Current Player:</strong> {{ current_player_name }}</p>
109             {% if current_bid and current_bid.quantity and current_bid.face %}
110                 <p>Current Bid: {{ current_bid.quantity }} x {{ current_bid.face
111             }}</p>
112             {% else %}

```

```

112         <p>No current bid.</p>
113     {% endif %}
114 </div>
115
116 <!-- Displaying the bot's action message -->
117 {% if bot_action_message %}
118     <div class="message">{{ bot_action_message }}</div>
119 {% endif %}
120
121 <!-- Making sure to display any error messages if present -->
122 {% if error_message %}
123     <div class="error-message">{{ error_message }}</div>
124 {% endif %}
125
126 <!-- Including a leave game button to allow users to go back to main menu
-->
127 <div class='Leave_game'>
128     <a href="{% url 'main_menu' %}">Back to Main Menu</a>
129 </div>
130
131 <!-- Now to display the player's dice, if they have rolled -->
132 <h2>Your Dice</h2>
133 <div class="dice-values">
134     {% if rolled %}
135         {% for value in current_player_dice %}
136             <span>{{ value }}</span>
137         {% endfor %}
138     {% endif %}
139 </div>
140
141 <!-- Providing action buttons for the user -->
142 {% if not round_ended %}
143     {% if not is_bot_turn %}
144         <form method="POST" class="actions">
145             <!-- Using django to prevent against csrf attacks-->
146             {% csrf_token %}
147             <button type="submit" name="action" value="roll_dice">Roll
Dice</button>
148             <button type="submit" name="action" value="call_bluff">Call
Bluff</button>
149         </form>
150         <h2>Make a Bid</h2>
151         <form method="POST" class="make-bid">
152             <!-- Using django to prevent against csrf attacks-->
153             {% csrf_token %}
154
155             <!-- Allowing the user to submit a bid-->
156             <label for="bid_count">Count:</label>
157             <input type="number" id="bid_count" name="bid_count" required
>
158             <label for="bid_value">Value:</label>
159             <input type="number" id="bid_value" name="bid_value" required
>
160             <button type="submit" name="action" value="make_bid">Submit
Bid</button>
161         </form>
162     {% endif %}
163 {% endif %}
164

```

```

165     <!-- If the round has ended, display the round ended button -->
166     {% if round_ended %}
167         <form method="POST">
168             {% csrf_token %}
169             <button type="submit" name="action" value="end_round" class="end-
round">End Round</button>
170         </form>
171     {% endif %}
172 </div>
173 </body>
174 </html>

```

Listing 21: bot_game.html file

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <meta name="viewport" content="width=device-width, initial-scale=1.0">
6     <title>Game Over</title>
7 </head>
8 <body>
9     <h1>Game Over</h1>
10    <!-- Retrieving and displaying the current winner-->
11    <h2>Winner: {{ winner }}</h2>
12    <a href="{% url 'main_menu' %}">Back to Menu</a>
13 </body>
14 </html>

```

Listing 22: game_over.html file

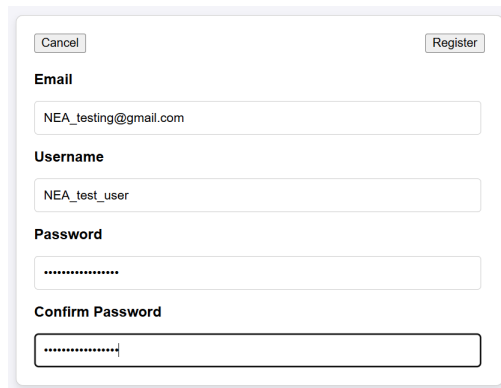
4 Programme Testing

I decided to use both black-box testing for each of my objectives, followed by end-user testing, in the form of a recording of three end-users using the program to complete several tasks and having them complete a brief questionnaire giving feedback for various objectives.

4.1 Black-Box testing

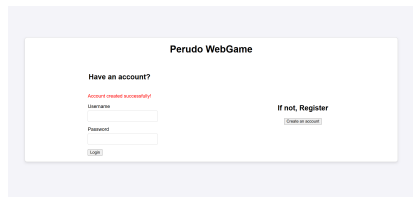
Test Number	1
Objectives Used	1.4, 1.6
Test Purpose	Testing the registration process, allowing the user to enter their email, chosen username, and password, creating their user account, as well as testing the password hashing system.
Expected Result	The user should be taken to the registration page upon pressing 'register' and offered to enter their email, username and password before creating their account or displaying any error messages if the data entered is invalid. This account should then be inserted into a database containing their details.

Actual Result	The program allowed a user (NEA_test_user) to register with their email (Nea_testing@gmail.com) and password (NEA_test_password), creating a new record in the database. Furthermore, when the user entered an invalid email address (InvalidEmail) without a string of characters followed by an '@' followed by another string of characters, the user was made to reenter their email. The user was also forced to confirm their password, invalidating their registration if the two passwords did not match.
----------------------	---

**Test
Evi-
dence**

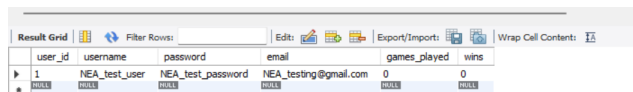
A registration form titled 'Perudo WebGame' with a 'Cancel' button at the top left and a 'Register' button at the top right. The form contains four input fields: 'Email' with the value 'NEA_testing@gmail.com', 'Username' with the value 'NEA_test_user', 'Password' with masked characters '*****', and 'Confirm Password' with masked characters '*****'.

The form correctly allows the user to enter each field of the form, before selecting register to attempt to register with these details.



A login page titled 'Perudo WebGame'. It has a heading 'Have an account?' and a sub-heading 'Account created successfully!'. Below this are two input fields: 'Username' and 'Password'. To the right of these fields is a button labeled 'If not, Register'. At the bottom left is a 'Login' button.

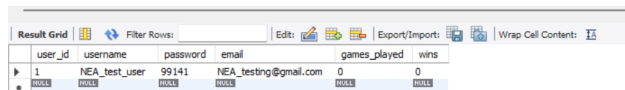
Upon entering a valid username, password, email and confirming their password, they are redirected to the login page, with a message saying their account has been created successfully.



A screenshot of a database record viewer showing a single record with the following data:

user_id	username	password	email	games_played	wins
1	NEA_test_user	NEA_test_password	NEA_testing@gmail.com	0	0

Upon retrieving the records of the database, we can see the entered details have been safely stored in the database correctly. For the purpose of clarity, I did not use the hashing algorithm to store the password for the purpose of this test, to verify the plain_test password was correctly stored. Repeating the test, but using the hashing algorithm this time, yielded the following result:

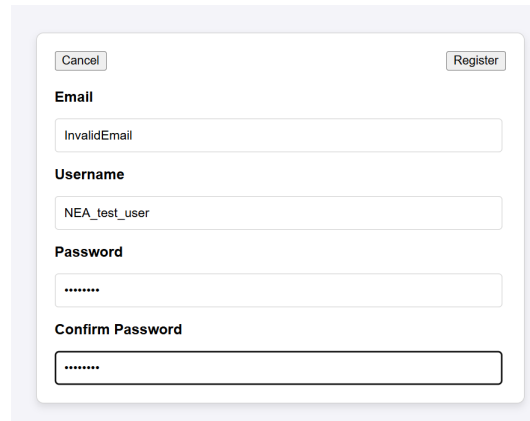


A screenshot of a database record viewer showing a single record with the following data:

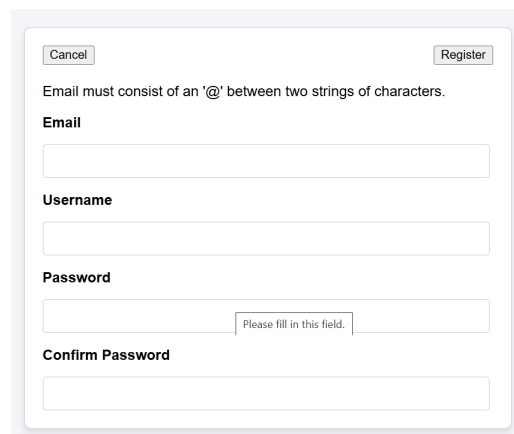
user_id	username	password	email	games_played	wins
1	NEA_test_user	99141	NEA_testing@gmail.com	0	0

**Test
Evi-
dence*****Testing data validation***

Upon entering an invalid email in the registration page as shown, which does not follow the predefined email regular expression, an appropriate error message was shown.

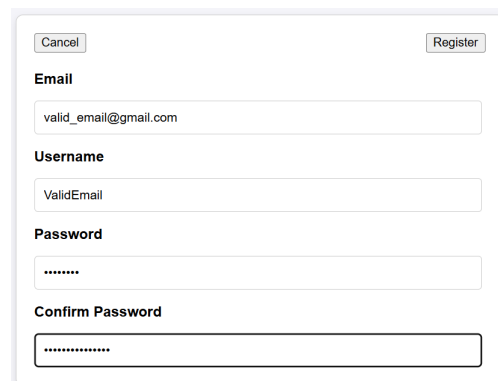


A registration form with a light purple border. At the top left is a 'Cancel' button and at the top right is a 'Register' button. The form contains four input fields: 'Email' with the text 'InvalidEmail', 'Username' with the text 'NEA_test_user', 'Password' with masked characters '*****', and 'Confirm Password' with masked characters '*****'.

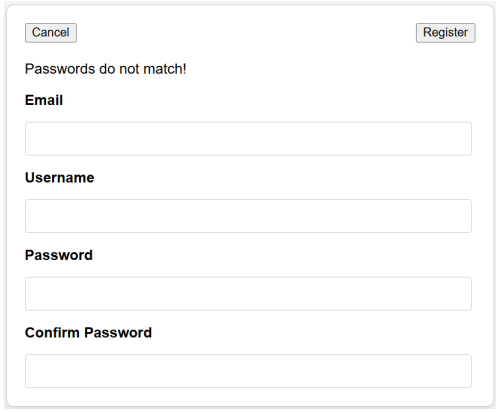


A registration form with a light purple border. At the top left is a 'Cancel' button and at the top right is a 'Register' button. Below the buttons is an error message: 'Email must consist of an '@' between two strings of characters.' The form contains four input fields: 'Email' (empty), 'Username' (empty), 'Password' (empty with a small error message 'Please fill in this field.' below it), and 'Confirm Password' (empty).

As seen below, entering valid details but not matching the password and confirm-password also gave an appropriate error message.



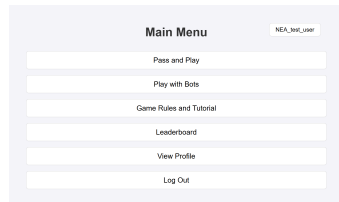
A registration form with a light purple border. At the top left is a 'Cancel' button and at the top right is a 'Register' button. The form contains four input fields: 'Email' with the text 'valid_email@gmail.com', 'Username' with the text 'ValidEmail', 'Password' with masked characters '*****', and 'Confirm Password' with masked characters '*****'.

Test Evidence	 A screenshot of a web registration form. At the top left is a 'Cancel' button and at the top right is a 'Register' button. Below these is a red error message: 'Passwords do not match!'. The form contains four input fields: 'Email', 'Username', 'Password', and 'Confirm Password'. The 'Email' and 'Username' fields are empty. The 'Password' and 'Confirm Password' fields contain text, but the text is not legible due to blurring.
Test Evaluation	Success

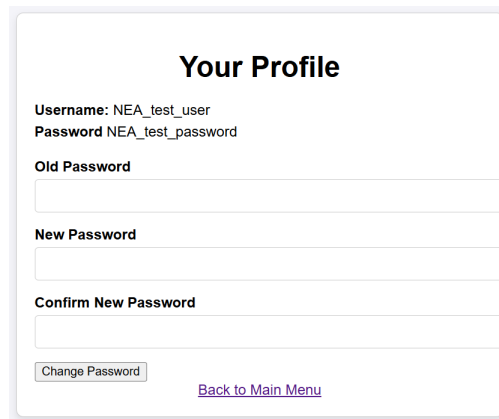
Test Number	2
Objectives Used	1.1-1.3
Test Purpose	Logging into the system, using the registered account from test 1, and ensuring data validation functions properly
Expected Result	If the user enters incorrect details, an appropriate error message should be displayed. However, if the entered details correspond correctly to a registered account, they should be taken to the main menu, and their name should be in the top right of the screen, to verify that they have logged in.
Actual Result	Upon entering the correct user details from the registration process in test 1, the user was taken to the main menu, with their name being correctly displayed in the top right, confirming they were correctly logged in. Furthermore, upon clicking the the view profile button, the correct details were displayed. Secondly, upon retrying the login process with incorrect details, an appropriate error message should be displayed.

**Test
Evi-
dence**

As seen in the screenshot, the home page worked as expected.



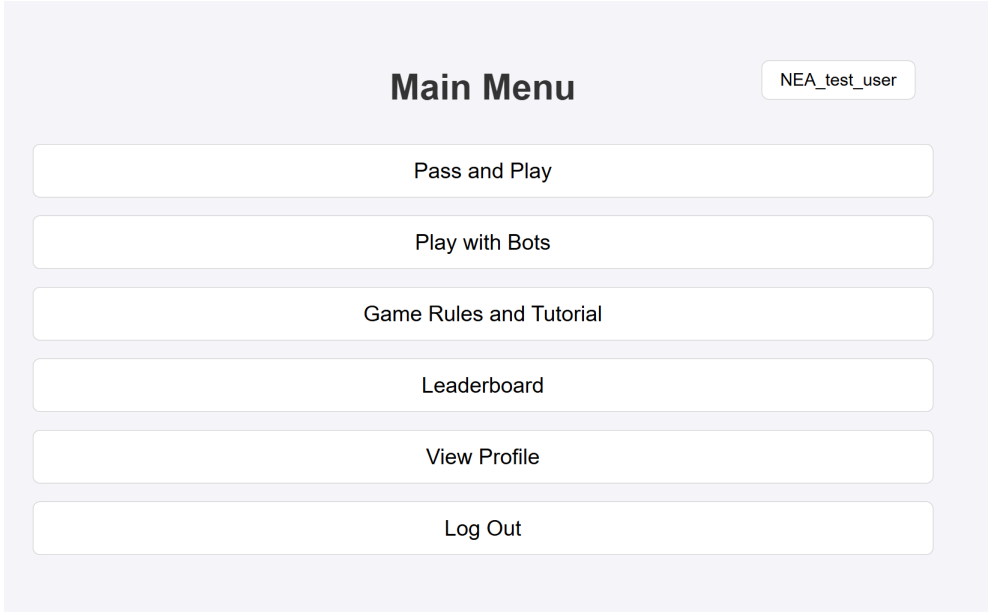
Upon clicking login, with the correct details entered, the user was correctly taken to the main menu, with their username in the top right, verifying they were logged in to the correct account.



This is further reinforced upon selecting 'view profile', with the correct details and password being displayed.

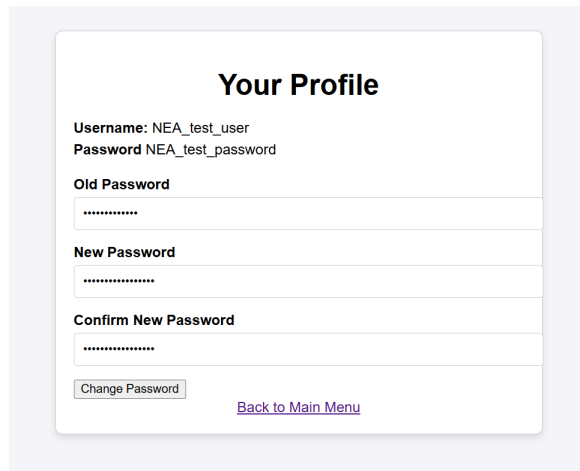
Test Evidence	<i>Testing invalid data entry</i> Perudo WebGame Have an account? Username NEA_test_user_incorrect Password ***** Login If not, Register Create an account Perudo WebGame Have an account? Invalid username or password. Username Password Login If not, Register Create an account Upon entering the incorrect account details, the system correctly rejected this login attempt, displaying an appropriate error message, making sure to redirect the user back to the same login page.
Test Evaluation	Success

Test Number	3
Objectives Used	2.1
Test Purpose	Testing whether or not the main menu operates well, displaying the correct buttons, in a clear and well-presented way.
Expected Result	The main menu should correctly display the various available pages, including 'pass and play', 'bot game', 'view profile', 'game rules and tutorial', 'leaderboard' and the 'log out' button.
Actual Result	As expected, the main menu correctly displayed each page, clearly with correctly labelled buttons.

Test Evidence	 As seen in the screenshot, the main menu contains all needed buttons displayed in a clear menu layout.
Test Evaluation	Success

Test Number	4
Objectives Used	2.2, 2.5
Test Purpose	Testing the view profile page, which should display the correct details, including the user's username and password. Furthermore, the page should give the user the opportunity to change their password.
Expected Result	The page should display the correct username and password of the current account. In addition to this, the user should be given a form to enter their old password and their new password, before confirming this to change their password. This should then update the database with their new password, if the new passwords match and if the old password is valid. Otherwise an appropriate error should be displayed.

Actual Result	The view profile page displayed the Nea_test_user account details correctly, and gave the user the option to enter the form to change their password, by entering their old password as well as their new password before confirming their new password. Once these fields were entered correctly, with the new password being NEA_test_password2, the details were changed correctly, with this password change being seen in the database as well. Afterwards, upon repeating this process with either an incorrect old password or mismatched new passwords, the correct corresponding error messages were displayed.
----------------------	--

**Test
Evi-
dence**

Your Profile

Username: NEA_test_user
Password NEA_test_password

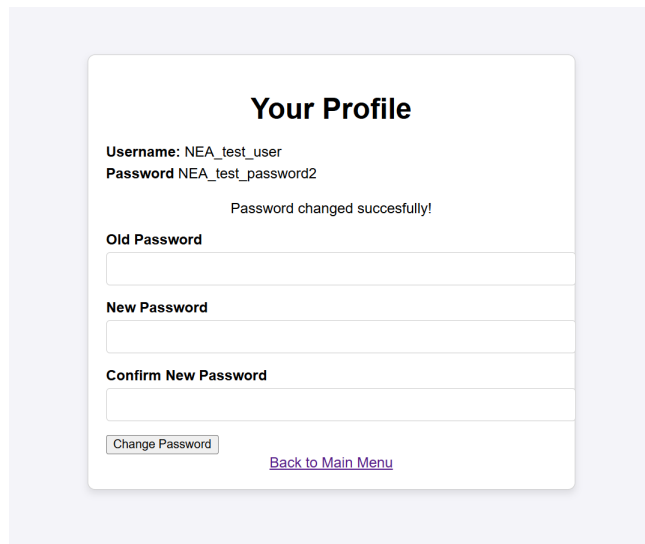
Old Password
.....

New Password
.....

Confirm New Password
.....

[Back to Main Menu](#)

The view profile correctly displayed all the correct information for the account previously created in test 1, as well as allowing the user to enter the fields required to change their password.



Your Profile

Username: NEA_test_user
Password NEA_test_password2

Password changed successfully!

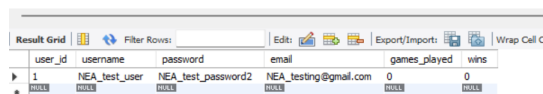
Old Password
.....

New Password
.....

Confirm New Password
.....

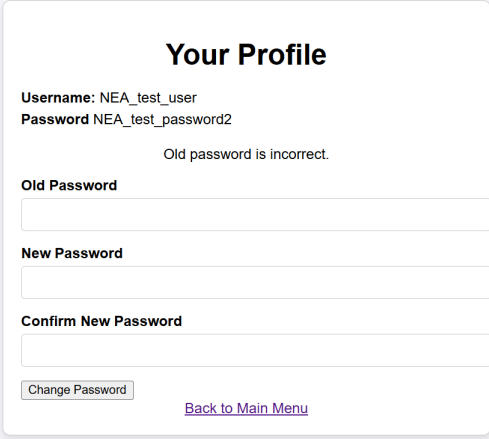
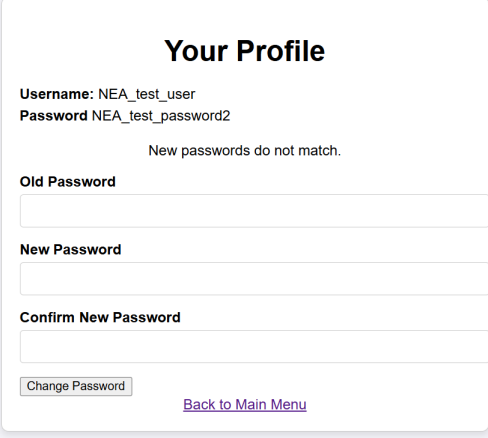
[Back to Main Menu](#)

```
1 • Use NEA_db;  
2 • SELECT * FROM USER;
```



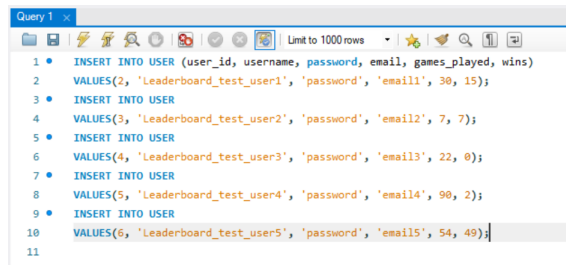
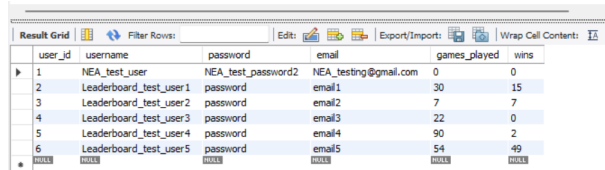
	user_id	username	password	email	games_played	wins
1	1	NEA_test_user	NEA_test_password2	NEA_testing@gmail.com	0	0

Upon entering the correct details, the password was correctly changed to NEA_test_password2 in the 'view profile' page, as well as updating the database as shown. For the purpose of the rest of the tests I chose not to use the hashing algorithm, having already proven it worked, to make verifying the correct password from the table easier.

Test Evidence	 Your Profile Username: NEA_test_user Password NEA_test_password2 Old password is incorrect. Old Password New Password Confirm New Password Change Password Back to Main Menu Upon entering an incorrect old password, the program correctly displayed the shown error message.  Your Profile Username: NEA_test_user Password NEA_test_password2 New passwords do not match. Old Password New Password Confirm New Password Change Password Back to Main Menu When I entered the correct old password, but failed to match the two new password fields, I got an appropriate error message, as seen above.
Test Evaluation	Success
Test Number	5
Objectives Used	2.3

Test Purpose	Testing the game rules and tutorial page, to see if the user can see the rules of the game, any additional tips and have access to a youtube video explaining the rules of Perudo.
Expected Result	Upon clicking the game rules and tutorial page, the user should be taken to a page displaying the game rules, how to win, any handy tips and a tutorial video.
Actual Result	The page displayed a brief summary of the objectives of the game, how to win and some additional tips. In addition to this, the user was also given an option to return to the main menu or watch an embedded youtube video further clarifying the rules. However, this embedded youtube video was externally sourced, and as such, did not provide rules specific to this web perudo implementation.

Test Evidence	Game Rules and Tutorial How to Play Every player starts with a pre-selected number of dice and players take turns making bids over the number of dice with a certain value in the game. Each turn the player can either raise the current bid (or make a bid if no bid has been made) or call the previous player's bluff. If a bluff call is successful, the player who made the bid loses a dice and a new round begins. If it is unsuccessful and their bid turns out to be true, the current player who called the bluff loses a dice, and the game continues. Winning the Game You win the game by outlasting your opponents and being the last player in the game with any dice left. Additional Tips When playing pass and play with friends, make sure to do your best to trick your opponents into losing their dice against you. Bluffing, double bluffing or even triple bluffing can be an extremely effective way of coercing your opponents into making poor decisions. Remember: play the man, not the game. Back to Main Menu If you are still unsure of the rules of Liar's dice, watch this video by 'Gather Together Games' on Youtube. Additional Tips When playing pass and play with friends, make sure to do your best to trick your opponents into losing their dice against you. Bluffing, double bluffing or even triple bluffing can be an extremely effective way of coercing your opponents into making poor decisions. Remember: play the man, not the game. Back to Main Menu If you are still unsure of the rules of Liar's dice, watch this video by 'Gather Together Games' on Youtube. 
Test Evaluation	Success
Test Number	6

Objectives Used	2.4																												
Test Purpose	Testing the functionality of the leaderboard page, as well the sorting system used to order accounts.																												
Expected Result	The leaderboard should display all user’s names and other stats in a correctly ordered way.																												
Actual Result	The leaderboard appeared to function perfectly, displaying all the various test accounts’ information, as well as ordering them correctly based on the chosen criteria.																												
Test Evidence	Before testing this functionality, I had to insert several users into the database, to properly test the leaderboard and sorting system. I inserted 5 test users with made-up stats (and unhashed passwords) using the following SQL query:   Upon navigating (using the nea test user account) to the leaderboard page, the following page was displayed, containing all the correct users. Leaderboard <table><tr><th>Username</th><th>Wins</th><th>Games</th><th>Win Percentage</th></tr><tr><td>NEA_test_user</td><td>0</td><td>0</td><td>0.0%</td></tr><tr><td>Leaderboard_test_user3</td><td>0</td><td>22</td><td>0.0%</td></tr><tr><td>Leaderboard_test_user4</td><td>2</td><td>90</td><td>2.2%</td></tr><tr><td>Leaderboard_test_user2</td><td>7</td><td>7</td><td>100.0%</td></tr><tr><td>Leaderboard_test_user1</td><td>15</td><td>30</td><td>50.0%</td></tr><tr><td>Leaderboard_test_user5</td><td>49</td><td>54</td><td>90.7%</td></tr></table> Sort By: ☒ Wins ☐ Games ☐ Win % Back to Main Menu	Username	Wins	Games	Win Percentage	NEA_test_user	0	0	0.0%	Leaderboard_test_user3	0	22	0.0%	Leaderboard_test_user4	2	90	2.2%	Leaderboard_test_user2	7	7	100.0%	Leaderboard_test_user1	15	30	50.0%	Leaderboard_test_user5	49	54	90.7%
Username	Wins	Games	Win Percentage																										
NEA_test_user	0	0	0.0%																										
Leaderboard_test_user3	0	22	0.0%																										
Leaderboard_test_user4	2	90	2.2%																										
Leaderboard_test_user2	7	7	100.0%																										
Leaderboard_test_user1	15	30	50.0%																										
Leaderboard_test_user5	49	54	90.7%																										

Test Evidence

By default, the accounts are ordered by wins, but by selecting 'games' or 'win %', the records can be ordered by these fields, as seen below:

Leaderboard

Username	Wins	Games	Win Percentage
NEA_test_user	0	0	0.0%
Leaderboard_test_user2	7	7	100.0%
Leaderboard_test_user3	0	22	0.0%
Leaderboard_test_user1	15	30	50.0%
Leaderboard_test_user5	49	54	90.7%
Leaderboard_test_user4	2	90	2.2%

Sort By:

Wins

Games

Win %

Back to Main Menu

Leaderboard

Username	Wins	Games	Win Percentage
NEA_test_user	0	0	0.0%
Leaderboard_test_user3	0	22	0.0%
Leaderboard_test_user4	2	90	2.2%
Leaderboard_test_user1	15	30	50.0%
Leaderboard_test_user5	49	54	90.7%
Leaderboard_test_user2	7	7	100.0%

Sort By:

Wins

Games

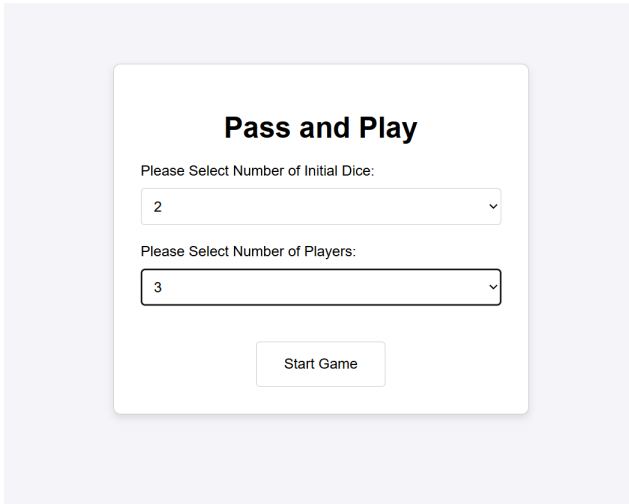
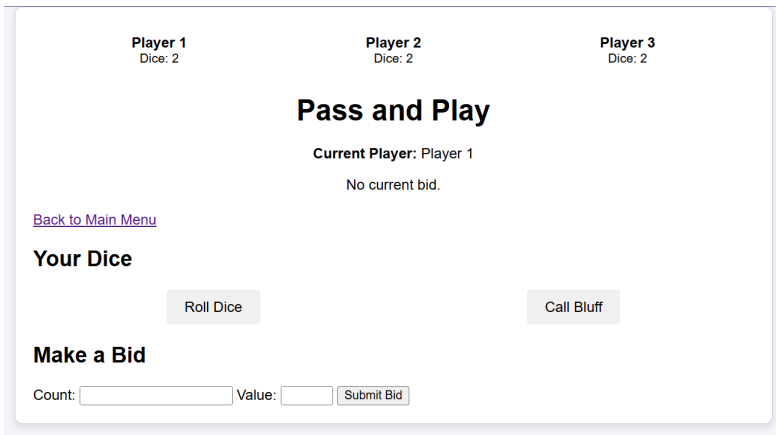
Win %

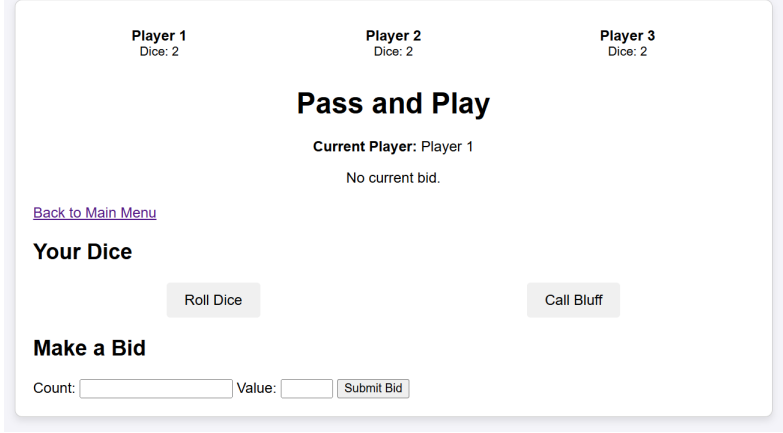
Back to Main Menu

Test Evaluation

Success

Test Number	7
Objectives Used	3.1 (partially), 3.2-3, 6.1-2
Test Purpose	Making sure the user is able to open a new pass-and-play game, and choose their settings, which should be carried over to the new game.

Expected Result	The main menu should provide the options to play 'pass and play', before being taken to a setup page, allowing the user to configure their settings, selecting their number of initial dice or number of players.
Actual Result	Upon clicking pass-and-play from the main menu, the user was taken to the set-up pass-and-play, allowing them to select a number of initial dice and players. Then, upon starting the game, the number of initial dice and players matched those chosen by the user.
Test Evidence	Upon selecting pass-and-play, the setup page was correctly displayed, allowing the user to select the number of initial dice and players. For this test, the initial number of dice selected was 2 and the initial number of players was 3.   Once the game was started, the correct number of players and initial dice were displayed in the game, confirming the user's choices had been used when creating the game.
Test Evaluation	Success

Test Number	8
Objectives Used	3.3
Test Purpose	Testing the user's ability to select an in-game name for themselves and others in the pass-and-play game.
Expected Result	The user should be given the option of selecting other players names, under their default name in the game.
Actual Result	There was not option to select player names from the game page whatsoever.
Test Evidence	Once the game had begun, no option to change names from the default names (player1, player 2, e.t.c) was presented. 
Test Evaluation	Unsuccessful

Test Number	9
Objectives Used	4.1
Test Purpose	Ensuring the players dice are randomised when they roll their dice, and that their value is within the proper range of 1-6.
Expected Result	Upon clicking roll dice, from the game created in test 7, random values should be given to the dice. The user should then submit a bid, allowing the other player to have their turn and roll their dice, further verifying the opponents dice are also random.
Actual Result	Each player in the game rolled their dice and received randomly generated values, which all lay within the valid range of 1-6.

**Test
Evi-
dence**

Upon rolling their dice, each player received randomized values, which all lay within the correct range of possible dice values.

Player 1
Dice: 2

Player 2
Dice: 2

Player 3
Dice: 2

Pass and Play

Current Player: Player 1

No current bid.

[Back to Main Menu](#)

Your Dice

2

3

Roll Dice

Call Bluff

Make a Bid

Count: Value:

Player 1
Dice: 2

Player 2
Dice: 2

Player 3
Dice: 2

Pass and Play

Current Player: Player 2

Current Bid: 1 of 2

[Back to Main Menu](#)

Your Dice

5

4

Roll Dice

Call Bluff

Make a Bid

Count: Value:

Player 1
Dice: 2

Player 2
Dice: 2

Player 3
Dice: 2

Pass and Play

Current Player: Player 3

Current Bid: 2 of 2

[Back to Main Menu](#)

Your Dice

4

1

Roll Dice

Call Bluff

Make a Bid

Count: Value:

Test Evaluation	Success
Test Number	10
Objectives Used	4.2-3
Test Purpose	Testing players' ability to make bids making sure these bids follow the rules of perudo.
Expected Result	When making a bid, with both a valid value, between 1 and 6 and a valid count, ensuring this bid is greater than the previous bid, the bid should replace the current bid. However, if the selected bid's value is not between 1 and 6 or the bid is not raised an error message should show, and the user should be forced to change their bid.
Actual Result	A new game was created with 4 players and 3 initial dice. The bid system worked correctly, returning an appropriate error message if the bid was invalid, both in the case of a value out of range and a bid that was not higher than the previous.

**Test
Evi-
dence**

The screenshot shows the 'Pass and Play' game interface. At the top, four players are listed: Player 1 (Dice: 3), Player 2 (Dice: 3), Player 3 (Dice: 3), and Player 4 (Dice: 3). The title 'Pass and Play' is centered. Below it, 'Current Player: Player 1' and 'No current bid.' are displayed. A link 'Back to Main Menu' is on the left. Under 'Your Dice', three dice are shown with values 1, 3, and 3. There are 'Roll Dice' and 'Call Bluff' buttons. The 'Make a Bid' section has a 'Count' of 2, a 'Value' of 3, and a 'Submit Bid' button.

The screenshot shows the 'Pass and Play' game interface. At the top, four players are listed: Player 1 (Dice: 3), Player 2 (Dice: 3), Player 3 (Dice: 3), and Player 4 (Dice: 3). The title 'Pass and Play' is centered. Below it, 'Current Player: Player 2' and 'Current Bid: 2 of 3' are displayed. A link 'Back to Main Menu' is on the left. Under 'Your Dice', three dice are shown with values 1, 3, and 3. There are 'Roll Dice' and 'Call Bluff' buttons. The 'Make a Bid' section has a 'Count' of 2, a 'Value' of 3, and a 'Submit Bid' button.

As seen in the above screenshots, when a bid of two 3's was made by player 1, it successfully became the current bid and the game progressed to player 2's turn.

**Test
Evi-
dence**

Player 1
Dice: 3

Player 2
Dice: 3

Player 3
Dice: 3

Player 4
Dice: 3

Pass and Play

Current Player: Player 2
Current Bid: 2 of 3

[Back to Main Menu](#)

Your Dice

2

4

1

Roll Dice

Call Bluff

Make a Bid

Count: Value:

Player 1
Dice: 3

Player 2
Dice: 3

Player 3
Dice: 3

Player 4
Dice: 3

Pass and Play

Current Player: Player 2
Current Bid: 2 of 3

Your bid must be higher than the current bid!

[Back to Main Menu](#)

Your Dice

2

4

1

Roll Dice

Call Bluff

Make a Bid

Count: Value:

When player 2 tried to make a bid of two 2's, which was lower than the current bid, the program correctly displayed an error, saying the bid had to be higher.

Player 1
Dice: 3

Player 2
Dice: 3

Player 3
Dice: 3

Player 4
Dice: 3

Pass and Play

Current Player: Player 2
Current Bid: 2 of 3

Your bid must be higher than the current bid!

[Back to Main Menu](#)

Your Dice

2

4

1

Roll Dice

Call Bluff

Make a Bid

Count: Value:

**Test
Evi-
dence**

The screenshot shows the 'Pass and Play' game interface. At the top, four players are listed: Player 1 (Dice: 3), Player 2 (Dice: 3), Player 3 (Dice: 3), and Player 4 (Dice: 3). The title 'Pass and Play' is centered. Below it, 'Current Player: Player 2' and 'Current Bid: 2 of 3' are displayed. A red message states 'Your bid must be higher than the current bid!'. A link 'Back to Main Menu' is on the left. Under 'Your Dice', three dice are shown with values 2, 4, and 1. Buttons for 'Roll Dice' and 'Call Bluff' are present. The 'Make a Bid' section has a 'Count' of 3 and a 'Value' of 10. A 'Submit Bid' button is next to the value. A red error message at the bottom says 'Value must be less than or equal to 6.'

Next, when the user instead entered a bid of three 10's, which was higher than the current bid, but had a value outside of the 1-6 range, an appropriate error message was displayed, as shown above.

This is a duplicate of the first screenshot, showing the same game interface with the 'Value' field set to 10 and the error message 'Value must be less than or equal to 6.'

The screenshot shows the 'Pass and Play' game interface. At the top, four players are listed: Player 1 (Dice: 3), Player 2 (Dice: 3), Player 3 (Dice: 3), and Player 4 (Dice: 3). The title 'Pass and Play' is centered. Below it, 'Current Player: Player 2' and 'Current Bid: 2 of 3' are displayed. A red message states 'Your bid must be higher than the current bid!'. A link 'Back to Main Menu' is on the left. Under 'Your Dice', three dice are shown with values 2, 4, and 1. Buttons for 'Roll Dice' and 'Call Bluff' are present. The 'Make a Bid' section has a 'Count' of 3 and a 'Value' of 3. A 'Submit Bid' button is next to the value.

To double-check that a valid bid would be accepted again, a bid of three 3's was entered, which successfully changed the current bid, and progressed the game to the next player.

Test Evaluation	Success
------------------------	---------

Test Number	11
Objectives Used	4.4
Test Purpose	Testing the bluff calling system and the removal of dice
Expected Result	If a bluff were called and the previous bid were correct, the current player (who called the bluff) should lose a dice, with an appropriate message stating so. However, if the previous bid was incorrect, the bluff call should be unsuccessful, removing a dice from the previous player, with an appropriate message
Actual Result	The bluff calling system seemed to work correctly with dice being removed from the correct players both when a bluff was successful and unsuccessful.

**Test
Evi-
dence**

To test these objectives, a new game was started with 2 players and 3 dice.

Player 1 rolled the dice 4, 5 and 4, as shown, before making a bid of two 4's, which is correct since two 4's have already been rolled.

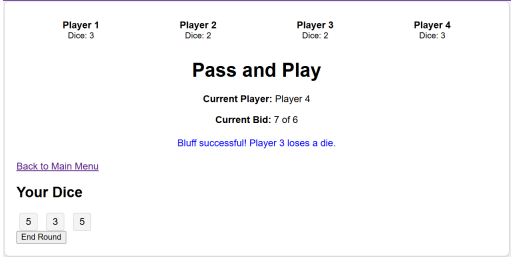
The screenshot shows the 'Pass and Play' game interface. At the top, four player bars are visible: Player 1 (Dice: 3), Player 2 (Dice: 3), Player 3 (Dice: 3), and Player 4 (Dice: 3). The title 'Pass and Play' is centered, followed by 'Current Player: Player 1' and 'No current bid.' Below this is a link 'Back to Main Menu'. The 'Your Dice' section shows three dice with values 4, 5, and 4. There are 'Roll Dice' and 'Call Bluff' buttons. The 'Make a Bid' section has a 'Count' field with the value 2, a 'Value' field with the value 4, and a 'Submit Bid' button.

On player 2's go, when they pressed 'call bluff', the following page was shown, correctly indicating the bluff call as unsuccessful. Furthermore, as seen in the player bar above, player 2 correctly lost 1 die.

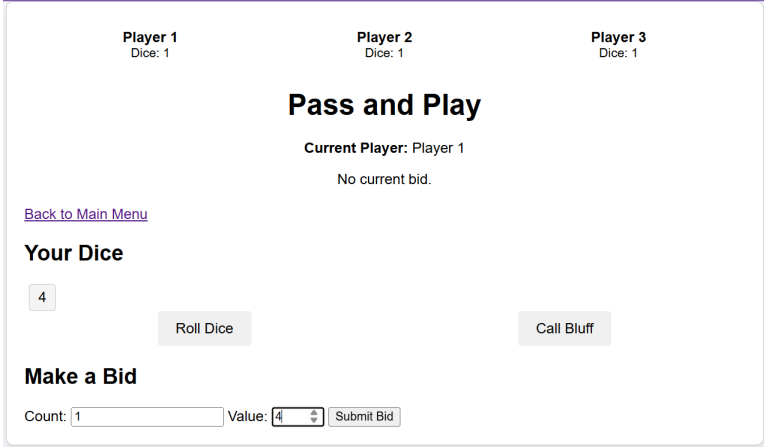
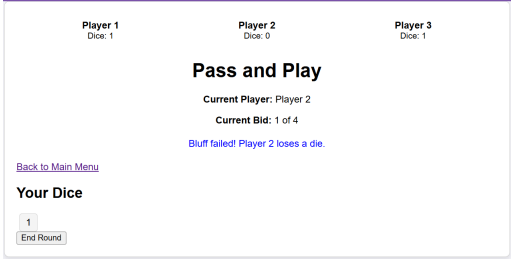
The screenshot shows the 'Pass and Play' game interface for Player 2. The player bars at the top show Player 1 (Dice: 3), Player 2 (Dice: 2), Player 3 (Dice: 3), and Player 4 (Dice: 3). The title 'Pass and Play' is centered, followed by 'Current Player: Player 2' and 'Current Bid: 2 of 4'. Below this is a message 'Bluff failed! Player 2 loses a die.' and a link 'Back to Main Menu'. The 'Your Dice' section shows three dice with values 1, 1, and 5. There is an 'End Round' button.

Now to test if a successful bluff call would be recognized, removing a die from the previous player, player 3 made a bid of seven 6s, which is clearly not possible, since only two players will have rolled their die by the bluff call.

The screenshot shows the 'Pass and Play' game interface for Player 3. The player bars at the top show Player 1 (Dice: 3), Player 2 (Dice: 2), Player 3 (Dice: 3), and Player 4 (Dice: 3). The title 'Pass and Play' is centered, followed by 'Current Player: Player 3' and 'No current bid.' Below this is a link 'Back to Main Menu'. The 'Your Dice' section shows three dice with values 6, 1, and 5. There are 'Roll Dice' and 'Call Bluff' buttons. The 'Make a Bid' section has a 'Count' field with the value 7, a 'Value' field with the value 6, and a 'Submit Bid' button.

Test Evidence	Upon player 4 calling player 3's bluff, as expected, a message was displayed indicating the bluff call was successful. Furthermore, player 3's dice went from 3 to 2, as expected. 
Test Evaluation	Success

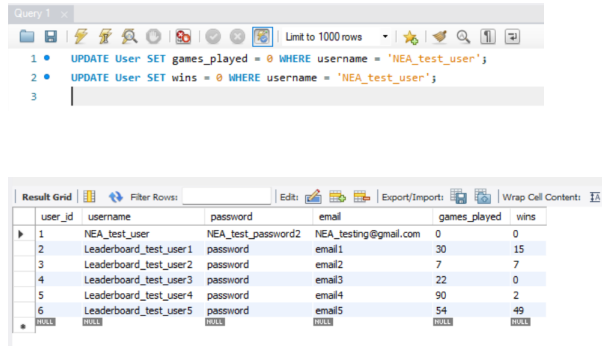
Test Number	12
Objectives Used	4.5
Test Purpose	Verifying that players are eliminated upon losing all their dice.
Expected Result	Once a player loses as many bluff calls as they have dice, they should be removed from the game.
Actual Result	Players were eliminated as expected, once they had lost all their dice.

Test Evidence	To test these objectives, I created a new game, consisting of 3 players, with only 1 dice each, to make this test easier to conduct and document. Initially, player 1 made a correct bid of one 4 as shown below, before player 2 called their bluff unsuccessfully.   Once player 2 lost a die and the round was ended, player 2 was eliminated and removed from the player bar, as seen in the screen-shot below of the start of the second round. 
Test Evaluation	Success

Test Number	13
Objectives Used	4.6, 4.7
Test Purpose	Making sure the game ends once only one player has dice left, updating the user's stats.
Expected Result	Once only one player has dice left, the game should end, displaying the winning player and increasing the user's games played by 1 and their wins by 1, if player 1 won.
Actual Result	The game ended as expected once all but one player lost their dice, correctly updating the NEA_test_user, by incrementing their games played but not their wins (since player 3 won the game, not player 1). Upon repeating a similar game with 2 players and 1 dice each, but letting player 1 win, both NEA_test_user's games played and wins were incremented.

**Test
Evi-
dence**

I continued the game started in test 12, which was left with 2 players each with 1 dice. Before this, however, I reset NEA_test_user's games played and wins to 0, for testing purposes, using the following SQL query.



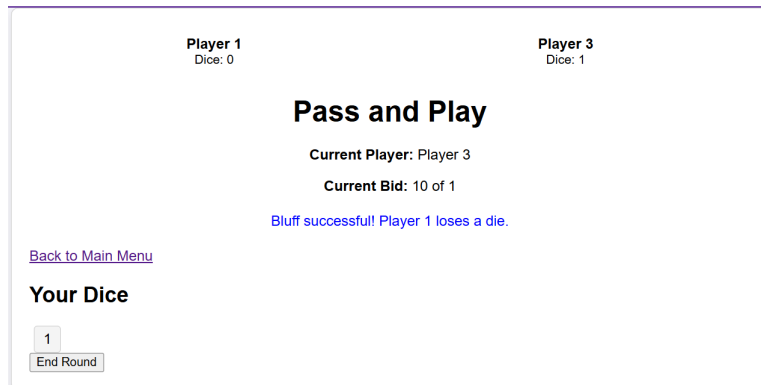
The screenshot shows a database query tool interface. The top section displays two SQL queries:

```
1 • UPDATE User SET games_played = 0 WHERE username = 'NEA_test_user';
2 • UPDATE User SET wins = 0 WHERE username = 'NEA_test_user';
3
```

The bottom section shows a 'Result Grid' with the following data:

user_id	username	password	email	games_played	wins
1	NEA_test_user	NEA_test_password2	NEA_testing@gmail.com	0	0
2	Leaderboard_test_user1	password	email1	30	15
3	Leaderboard_test_user2	password	email2	7	7
4	Leaderboard_test_user3	password	email3	22	0
5	Leaderboard_test_user4	password	email4	90	2
6	Leaderboard_test_user5	password	email5	54	49

Then, as player 1, I made an obviously impossible bid of ten 1's and made player 3 call this bluff successfully, losing player 1 their final dice, as shown.



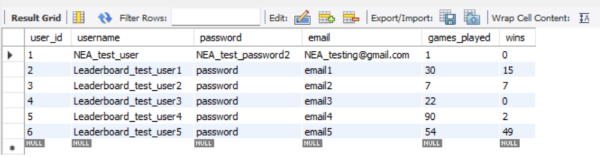
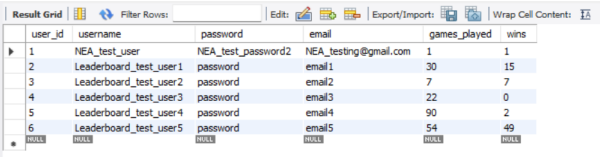
The screenshot shows a game interface titled "Pass and Play". At the top, it displays "Player 1 Dice: 0" and "Player 3 Dice: 1". The current player is "Player 3" with a "Current Bid: 10 of 1". A message states "Bluff successful! Player 1 loses a die." There is a link "Back to Main Menu". Below this, under "Your Dice", there is a single die showing the number "1" and an "End Round" button.

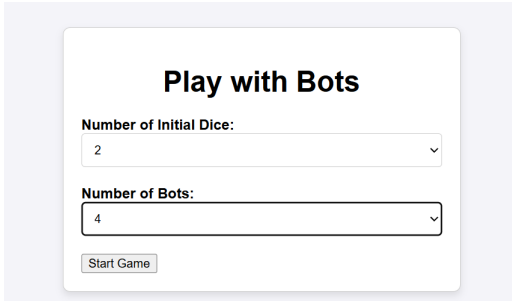
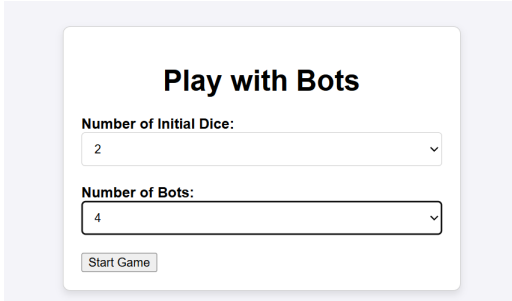
Once I ended the round, the game-over page was correctly shown, displaying the correct winner, player 3.

Game Over

Winner: Player 3

[Back to Menu](#)

Test Evidence	Upon checking the database, the games played field for the NEA_test_user seems to have been correctly incremented, whilst the wins field remains unaltered, since player 3 won not player 1.  Repeating this entire process and resetting the user's stats, but instead letting player 1 win, instead increments both the test user's games played and wins, as demonstrated below. Player 1 Dice: 1 Player 2 Dice: 0 Pass and Play Current Player: Player 2 Current Bid: 1 of 6 Bluff failed! Player 2 loses a die. Back to Main Menu Your Dice 5 End Round Game Over Winner: Player 1 Back to Menu 
Test Evaluation	Success
Test Number	14

Objectives Used	5.1
Test Purpose	Ensuring the user can customise the bot game by choosing the number of bots and initial number of dice.
Expected Result	The user should be given the choice of number of bots to play against and how many dice to start the game with, before creating a game with these settings.
Actual Result	The user was presented with a setup page similar to that of the pass-and-play mode, where they could select number of bots and initial dice. These settings were then successfully carried over to the game.
Test Evidence	Upon selecting play with bots from the main menu, the user was directed to a setup page, allowing the choice of number of bots and initial dice as shown.  Upon starting the game, with the above settings (4 bots, 2 initial dice), the following game was created, matching the chosen settings, as seen in the player bar. 
Test Evaluation	Success

Test Number	15
Objectives Used	5.2
Test Purpose	Verifying that the bot game implements the bid system correctly, only allowing valid bid values and requiring bids be raised, if following a previous bid.

Expected Result	If a bid with a value outside of the 1-6 range is made, or the bid should not raised from any previous bid, an appropriate error message should be shown. However, if the bid is valid, it should become the current bid.
Actual Result	The bid system worked as expected, correctly addressing the following scenarios: 1. If the bid value lay outside of the 1-6 range, the input was correctly rejected 2. When the bot made a bid, it contained valid values and was raised from the previous bid 3. When the user entered a bid that was not higher than the previous bid, it was correctly rejected, with an appropriate error message.

**Test
Evi-
dence**

To test the bidding system, I started a game with another bot and 3 initial dice.

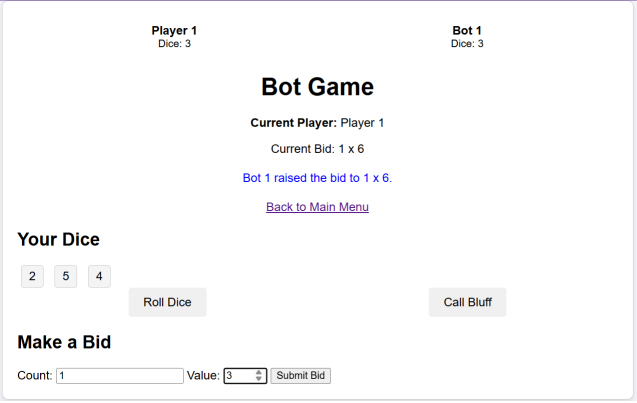
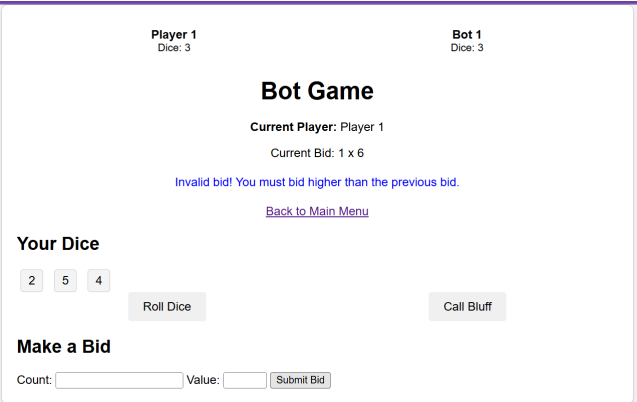
Upon selecting a bid with a value outside of the acceptable 1-6 range, an appropriate form error message was displayed, as seen below.

The screenshot shows the 'Bot Game' interface. At the top, 'Player 1' has a dice of 3, and 'Bot 1' also has a dice of 3. The title 'Bot Game' is centered. Below it, 'Current Player: Player 1' and 'No current bid.' are displayed, along with a 'Back to Main Menu' link. The 'Your Dice' section shows three dice with values 2, 5, and 4, and a 'Roll Dice' button. The 'Make a Bid' section has a 'Count' of 1, a 'Value' input field set to 7, and a 'Submit Bid' button. A red error message at the bottom states: 'Value must be less than or equal to 6.'

When I instead made a valid bid of one 5, it was correctly accepted, and the bot player raised the bid to a bid of one 6, which correctly fulfills perudo's rules, as this bid is higher than that made initially by me.

This screenshot shows the 'Bot Game' interface after a valid bid. The 'Current Player' is still 'Player 1', but the 'Current Bid' is now '1 x 6'. The 'Your Dice' section remains the same. In the 'Make a Bid' section, the 'Value' input field is now set to 5, and the 'Submit Bid' button is visible. The error message is no longer present.

This screenshot shows the 'Bot Game' interface after the bot has made a bid. The 'Current Bid' is now '1 x 6'. A message below the bid states: 'Bot 1 raised the bid to 1 x 6.' The 'Your Dice' section remains the same. In the 'Make a Bid' section, the 'Value' input field is now empty, and the 'Submit Bid' button is visible. The error message is no longer present.

Test Evidence	Next, upon entering a bid one 3, which is invalid since it is not higher than the previous bid of one 6, an appropriate error message was displayed saying the bid had to be higher.  
Test Evaluation	Success

Test Number	16
Objectives Used	5.3
Test Purpose	Testing the bluff system in the bot game, making sure dice are removed correctly, players/bots are eliminated once out of dice and that the game ends once only one player/bot is left, displaying the correct winner.
Expected Result	Upon a bluff call, by either a bot or player, a dice should be removed from the current player if the call is a failure, or from the previous player, if the call is a success. Then, once a player has lost all of their dice, they should be eliminated. If there is only one player left with dice, the game should then end, displaying the correct winner.

Actual Result	The dice removal system, player elimination and game-over system worked exactly as expected.
Test Evidence	To test these scenarios, I created a bot game with two other bots, starting with 2 dice each. Initially, I set the set the bot bids/bluff weights such that they could only make bids for the ease of the first test round. Upon making a bid of one 3 as shown, bot 1 raised the bid to one 4, before bot 2 then raised this bid further to one 5. Player 1 Dice: 2 Bot 1 Dice: 2 Bot 2 Dice: 2 Bot Game Current Player: Player 1 No current bid. Back to Main Menu Your Dice 3 5 Roll Dice Call Bluff Make a Bid Count: 1 Value: 3 Submit Bid Player 1 Dice: 2 Bot 1 Dice: 2 Bot 2 Dice: 2 Bot Game Current Player: Player 1 Current Bid: 1 x 5 Bot 2 raised the bid to 1 x 5. Back to Main Menu Your Dice 6 4 Roll Dice Call Bluff Make a Bid Count: Value: Submit Bid Upon calling this bluff, the program responded correctly, stating the bluff call had been successful since, as seen in the debug print statements for player dice, there was no 5 in the game at the time, among my dice and the bot's dice. Player 1 (Player): [6, 4] Bot 1 (Bot): [6, 3] Bot 2 (Bot): [6, 1] Player 1 Dice: 2 Bot 1 Dice: 2 Bot 2 Dice: 1 Bot Game Current Player: Player 1 Current Bid: 1 x 5 Player 1 called bluff successfully! Bot 2 loses a die. Back to Main Menu Your Dice 6 4 End Round As seen in the player bar, bot 2 correctly lost a dice.

**Test
Evi-
dence**

I continued to play this game for another round to test if successfully calling bot 2's bluff again would eliminate them entirely from the game. As seen below, upon calling bot 2's bluff successfully, they were removed from the game, confirming the player elimination system is functioning well.

Player 1
Dice: 2

Bot 1
Dice: 2

Bot 2
Dice: 0

Bot Game

Current Player: Player 1
Current Bid: 2 x 5

Player 1 called bluff successfully! Bot 2 loses a die.

[Back to Main Menu](#)

Your Dice

3 3

End Round

Player 1
Dice: 2

Bot 1
Dice: 2

Bot Game

Current Player: Player 1
Current Bid: 1 x 3

Bot 1 started the bidding with 1 x 3.

[Back to Main Menu](#)

Your Dice

Roll Dice Call Bluff

Make a Bid

Count: Value: Submit Bid

Now, to test whether or not the game would end if only one player were left, I played two other bot games, with one other bot and 1 die each, to verify the bot game would end both when the bot ran out of dice and the player ran out of dice.

As seen in the first game below, upon calling the bots bluff successfully, the game was ended displaying the correct winner as player 1.

Player 1
Dice: 1

Bot 1
Dice: 1

Bot Game

Current Player: Player 1
Current Bid: 2 x 1

Bot 1 raised the bid to 2 x 1.

[Back to Main Menu](#)

Your Dice

6

Roll Dice Call Bluff

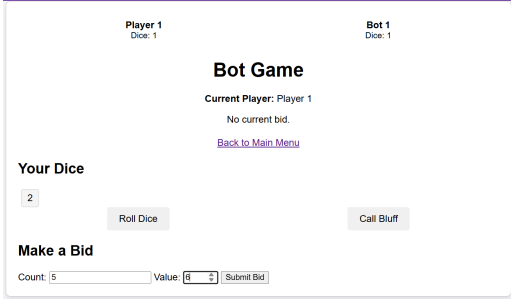
Make a Bid

Count: Value: Submit Bid

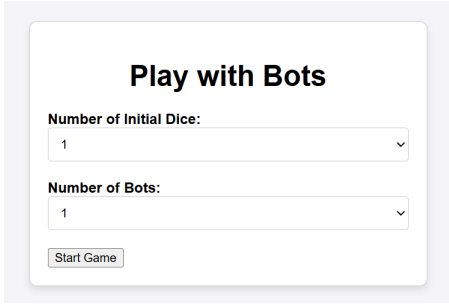
Game Over

Winner: Player 1

[Back to Menu](#)

Test Evidence	Now starting another game with the same settings, but instead disabling bot bids and only making them able to call bluffs, by adjusting the bid/bluff weights, I deliberately made an impossible bid to see if the game would correctly end, displaying the bot as the winner. As seen below, the program worked as expected.  Game Over Winner: Bot 1 Back to Menu
Test Evaluation	Success

Test Number	17
Objectives Used	5.4, 3.4
Test Purpose	Testing whether or not bot difficulty and names could be customized.
Expected Result	The program should give the option of choosing bot difficulty in the setup page, as well as changing the bots' names.
Actual Result	The bot game setup page only displayed the options of choosing the number of bots and initial dice, but not bot difficulty or names.

Test Evidence	As seen in the bot game setup page below, these features are not present. 
Test Evaluation	Unsuccessful

4.2 End-User Testing

In addition to the black-box testing, I recorded three users as they carried out the list of instructions below, before getting them to respond to a questionnaire evaluating many of the program's subsystems.

End-user Testing Instructions – Perudo NEA

1. Please start off by registering an account from the home page, making sure not to use any of your or anyone else's personal information.
2. Next, try and login to the system.
3. Now make sure to explore all pages of the main menu, as well as play perudo against bots or yourself using the pass-and-play game.
4. Throughout the program, feel free to test erroneous or boundary data, to make sure the program responds appropriately.
5. Don't forget to respond to the Google form I sent to your email.

I have uploaded the screen recordings I took of each end-user's usage to Youtube, which can be accessed using the following links.

End-user 1: <https://youtu.be/hsje2cKFssI>

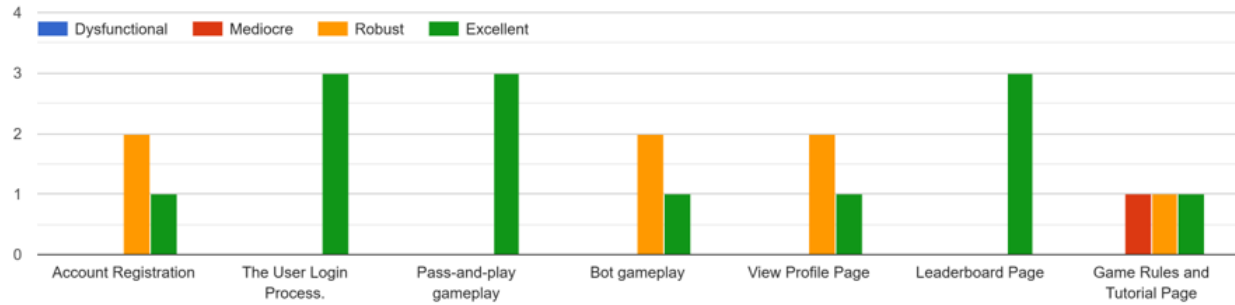
End-user 2: https://youtu.be/_K-MBWv3xo0

End-user 3: <https://youtu.be/-T2BmQa45ys>

4.2.1 End-User Questionnaire Responses

To get an idea of what three of my end-users thought of the program and its systems, I got them to fill out a Google form.

How did you find the following systems of the program?



Firstly, I asked the end-users to rate each subsystem as either dysfunctional, mediocre, robust or excellent. Then, for each subsystem, I asked them what they liked about it and what they thought could be better. As seen from the above table, no systems were rated dysfunctional, however, the tutorial page received a mixed response.

What went well, regarding the Game Rules and Tutorial page?

3 responses

The page displayed some useful instructions and tips, as well as a video which I was able to watch in the page.

the game rules were clear and easy to understand. The embedded video also worked well.

Like the other pages, these pages were correctly displayed and convenient to understand.

What could have been better about the Game Rules and Tutorial page?

3 responses

The instructions and tips could have been better catered to the actual program, maybe using some screenshots from the actual game or something .

The game rules could have been slightly more specific.

The tutorial used seemed to be used for the physical perudo game, but an online perudo tutorial using this program would have been better.

Looking at their responses to what could have been better about the game rules and tutorial page, all of the users thought this page could have been more detailed or specific, especially related to the actual program itself, rather than the general rules of Perudo. Despite this common criticism, the users still said the page 'displayed some useful instructions' and that the rules were generally convenient and easy to understand. They also seemed to like the addition of the Youtube video as well.

What went well, regarding the View profile and Leaderboard page?

3 responses

The view profile worked well displaying the correct username and password. I also managed to changed my password. The leaderboard page displayed my account correctly and using the other leaderboard test accounts, the sorting buttons worked perfectly.

they both displayed the correct information and I liked the sorting options on the leaderboard.

The view profile page was laid out well with the relevant information, and clear option to change password, although i forgot to do this. The leaderboard was also displayed well and I appreciated the sorting options, which i imagine could be very useful if the game were ever played competitively by many players.

What could have been better about the View profile and Leaderboard page?

3 responses

It might have been nice to allow for the user to change their email or username as well as password.

it would be nice to also be able to see the account's email as well in the view profile page.

I can't think of anything in particular for the leaderboard, but the view profile page could have displayed more information, such as the user's other stats.

I also asked these end-users to give feedback on the view profile and leaderboard pages, with both pages receiving fairly positive feedback, especially regarding the sorting features of the leaderboard page. However, a couple of the responses mentioned displaying more information in the view profile page, such as game stats and the user's email. One user also suggested allowing the user to change their email or username as well as their password, which I think would be a useful feature to include if I were to develop this system further.

What went well, regarding the user registration and login process?

3 responses

I was able to register successfully and login using those registered details. There was also sufficient data validation to prevent any erroneous inputs or invalid information from being entered.

the login/registration worked with the website only accepting emails with an '@' in it correctly.

There was data validation when entering the email and confirming my password choice, but i could then create my account and login with ease.

What could have been better about the user registration and login process?

3 responses

An email authentication system could have been useful, as this would have made the account system more secure.

could have used email authentication as well. A forgot password feature would also be good.

Email verification would make this system even better and it would also be better if, once an error message is displayed, the form is not completely reset, since this wasted a lot of time.

Feedback regarding the user registration and log-in system was very positive, with end-users recognizing the implementation of data validation and just generally saying the system functioned. Funnily enough, all three of the users mentioned an email authentication system could have been a good addition, as well as a 'forgot password' system of retrieving account details. One user also disliked the automatic form clearing once an error occurred, which I understand, but did not think was a major issue at all.

What went well, regarding the perudo gameplay (both pass-and-play and bot gameplay)?

3 responses

The game functioned well, fulfilling the rules of liar's dice properly. It was also very easy to navigate and understand, with good exception handling to prevent me from making invalid plays.

The game seemed to function well and the interface was visually appealing and clear.

Both gamemodes worked well and I liked the look of the interface.

What could have been better about the perudo gameplay?

3 responses

Both gamemodes worked really well, but it would have been nice to include some colour to the interface, and perhaps a dice rolling animation and background, to make the game more visually appealing.

it would be cool if you could play with other players and bots, mixing the two gamemodes.

I noticed that the number 1 dice being a wild card was not implemented but im not sure if that is just from a different version of perudo or not, so it's not too much of an issue.

Moving onto the actual perudo gameplay, all the users said the game was functional and the interface was visually appealing. Furthermore, a user complimented the exception handling of the game, when making invalid plays or bids. However, there were a couple of minor issues described afterwards, including a slight discrepancy between some variations of Perudo and this game's rules regarding the treatment of 1's as 'wild cards' which can assume any value mentioned by a bid. When making this game, the rules I saw and tried to use did not include this feature, however, upon further reflection, I have noticed many versions of the game do include this feature, so, if I were to develop this project further, I would consider changing this. One user also mentioned including a slightly more sophisticated user interface, with features such as dice animations or a nice background might add to the user experience, which I do not think is essential to the purpose of this project, but would be a nice touch to add during further development.

Do you have any other comments upon the program as a whole?

3 responses

The user interface graphics looked clean and the UI was pretty easy to navigate.

No.

Not really.

My end-users did not have any major comments on the program as a whole (which is unsurprising considering the number of preceding questions), but complimented the overall look of the program.

5 Program Evaluation

Objective	Performance Criteria	Evaluation	Core /Additional
1.1	The home page should display options of logging in or registering.	<i>Completely Achieved</i> The home page functioned well, displaying all the necessary options for user registration/login.	Core
1.2	A form or method by which users can enter their login details should be provided	<i>Completely Achieved</i> The login form worked perfectly, only accepting correct details, with good data validation, as mentioned in the following objective.	Core
1.3	Appropriate error messages should be shown if entered details are invalid or incorrect, based on predefined rules and the user's previously registered details	<i>Completely Achieved</i> Data entered during the login process was verified correctly using the backend database, and the format of emails being registered with were validated using an appropriate regular expression.	Core
1.4	The registration page should function, allowing users to enter their account details, including email, username and password to create an account.	<i>Completely Achieved</i> The registration process worked well, inserting the correct details into the backend database, and validating data entries. Once these details were registered, the user was able to login with these details immediately, confirming its success.	Core
1.5	Upon registering an authentication email should be sent to the user asking them to verify their email address.	<i>Not Achieved</i> Due to time constraints and some issues setting up an SMTP on the school network and using Microsoft emails, I was unable to implement this objective. However, given more time, this objective should be achievable, although I still do not believe this to be an essential feature of this prototype registration system, although it would be important if this were implemented commercially in the real world.	Additional
1.6	When user details are registered, the chosen password should be hashed using an hashing algorithm, before being stored in the database, for increased security.	<i>Completely Achieved</i> The hashing algorithm implemented worked perfectly, correctly storing the hashed password in the database, protecting security.	Additional

2.1	Main menu should have various buttons redirecting the user to the different parts of the program, including 'view profile', 'game rules and tutorial', 'view leaderboard', 'pass-and-play' and 'bot game'. It should also allow the user to log out.	<i>Completely Achieved</i> The main menu provided all the necessary buttons to redirect the user to the various parts of the program, as well as letting the user log out.	Core
2.2	The 'view profile' page should display the user's username, statistics and other information	<i>Completely Achieved</i> The view profile functioned well, retrieving the correct user's username and password from the backend database.	Core
2.3	The 'game rules and tutorial' page should display an adequate set of rules and tips to the user, as well as include a video tutorial on how to play.	<i>Mostly Achieved</i> The game rules and tutorial page correctly displayed a set of instructions, some additional tips and an embedded video explaining how to explain perudo. However, based on some of the end-user feedback, the page would have better achieved the objective if it included program specific information, as opposed to general Perudo rules for the physical game. Given more time, I could record my own perudo tutorial video, as well as write some more specific instructions to this program, perhaps including screenshots from the program as well.	Additional
2.4	The 'view leaderboard' page should display a clear leaderboard with all user's names and stats.	<i>Completely Achieved</i> The leaderboard page worked fantastically, displaying the correct user details from each account and going beyond this, including several different sorting options, using my merge sort algorithm.	Core
2.5	The 'view profile' page should enable the user to change their username and password.	<i>Mostly Achieved</i> The view profile fulfilled part of this objective, allowing the user to easily change their password, making sure to confirm this password, and correctly updating the backend database accordingly. Unfortunately, however, it failed to allow the user to change their username. However, considering I implemented the password changing system successfully, given more time I think I could comfortably fulfill this objective completely.	Additional

3.1	Allow the user to choose to play either a 'pass-and-play game' or a 'bot game.	<i>Completely Achieved</i> The main menu allowed users to easily navigate between these two game modes.	Core
3.2	Upon clicking a game mode, a setup page should give the user choice of game settings.	<i>Completely Achieved</i> The game setup page provided two easy to use drop down options to select the number of initial dice and number of players. These drop-down menus also ensured data validation, since only the given options for these numbers were possible.	Core
3.3	The selected game settings should be applied to the game they start.	<i>Completely Achieved</i> The game settings were applied to the game created without fail.	Core
3.4	The user should be given the option to choose different players' names in the game.	<i>Not Achieved</i> Due to time constraints I was unable to implement this feature. Although this would be a interesting feature, I, by no means, think it is necessary or even particularly important feature beyond its aesthetic benefit.	Additional
4.1	Randomise dice once rolled and display them.	<i>Completely Achieved</i> The dice rolling functionality worked as intended, randomly generating and displaying the user's dice correctly.	Core
4.2	Provide an option of moves, including making a bid or calling the previous player's bluff.	<i>Completely Achieved</i> The user was given was given both of these options as intended, with a call bluff button and two form entries for the count and value of a bid.	Core
4.3	Only allow for bids that fit the rules of the game, preventing dice values beyond 6 and bids that are not higher than the previous bids.	<i>Completely Achieved</i> The program successfully achieved this objective, with the form only accepting natural numbers less than 7 for the dice number, and an appropriate error message coming up when a bid was not higher than the previous bid.	Core
4.4	Correct dice removal from players should be included, depending on the outcome of a bluff call.	<i>Completely Achieved</i> Dice were always removed from the correct player, following a bluff call, with a message being displayed showing the result of the bluff call.	Core
4.5	Once players run out of dice they should be eliminated from the game.	<i>Completely Achieved</i> Once players round out of dice, but the game had not ended, players were always correctly removed from the game, no longer showing in the player bar and not having dice or being able to make bids/bluff calls.	Core

4.6	Once only one player has dice remaining, the game should end and they should be displayed as the winner.	<i>Completely Achieved</i> The game always ended once only one player had dice left, displaying a simple game over screen, correctly displaying the winner.	Core
4.7	When the game has ended, the current user's stats should be updated appropriately, always incrementing games played and incrementing wins if they won.	<i>Completely Achieved</i> The mySQL backend database was updated upon the start and end of a game, incrementing wins and games played appropriately.	Core
4.8	The game's user interface should be clear and usable.	<i>Mostly Achieved</i> According to the end-user questionnaire feedback, this objective was met well, and the game was relatively easy and convenient to play. However, one user did suggest the addition of more colour and suggested using a background or dice rolling animations to make the game look more life-like. For the sake of this project, I did not think these additions were particularly vital, although I do see the appeal of these kinds of additions.	Additional
5.1	The system should allow the user to choose the number of bots they want to play against, and the number of initial dice they should each have.	<i>Completely Achieved</i> The bot game setup page correctly provided both of these options, utilising drop-down menus, similar to the pass-and-play setup page. Thus, correct data validation was also ensured using these menus, preventing erroneous or unusable data entries.	Core
5.2	As in the pass-and-play gamemode, only valid bids should be made by both players and bots.	<i>Completely Achieved</i> When testing the bot game, if an invalid bid was made by the user, an appropriate error message was shown, however, the bots were also unable to make invalid bids throughout the testing	Core
5.3	As in the pass-and-play mode, dice should be removed from the correct players/bots when a bluff call is made and players should be eliminated or the game should end following the rules stated before.	<i>Completely Achieved</i> When tested, the bot game always ensured the correct removal of dice, as well as displaying the result of a bluff call at a round.	Core

5.4	The user should be given a choice of bot difficulty.	<i>Not Achieved</i> Due to time constraints, I was unable to implement this feature, although think this would be a great feature to develop in the future. As a short term solution, I imagine simply allowing the user to adjust the bid/bluff weights of the bots could partially meet this objective, but a more sophisticated algorithm would be required to fully meet the objective.	Additional
6.1	In each game setup page, the user should be able to choose the number of initial dice each player should be allowed	<i>Completely Achieved</i> This objective was completely fulfilled, with each setup page appropriately displaying a drop down menu of options for the number of initial dice. These option were then appropriately carried into the game, with the correct number of dice being initially obtained by each player or bot.	Core
6.2	In each game setup page, the user should be able to choose the number of other players/bots they play against	<i>Completely Achieved</i> This objective was completely fulfilled, with each setup page appropriately displaying a drop down menu of options for the number of initial players. These option were then appropriately carried into the game, with even larger number of players occupying the player bar and making decisions effectively, without ruining the game.	Core
6.3	In each game setup page, the user should be able to choose the number of dice lost each round.	<i>Not Achieved</i> As a result of a lack of time, I was unable to implement this feature, but for future development this feature would be very attainable.	Additional

5.1 Evaluation based on end-user feedback

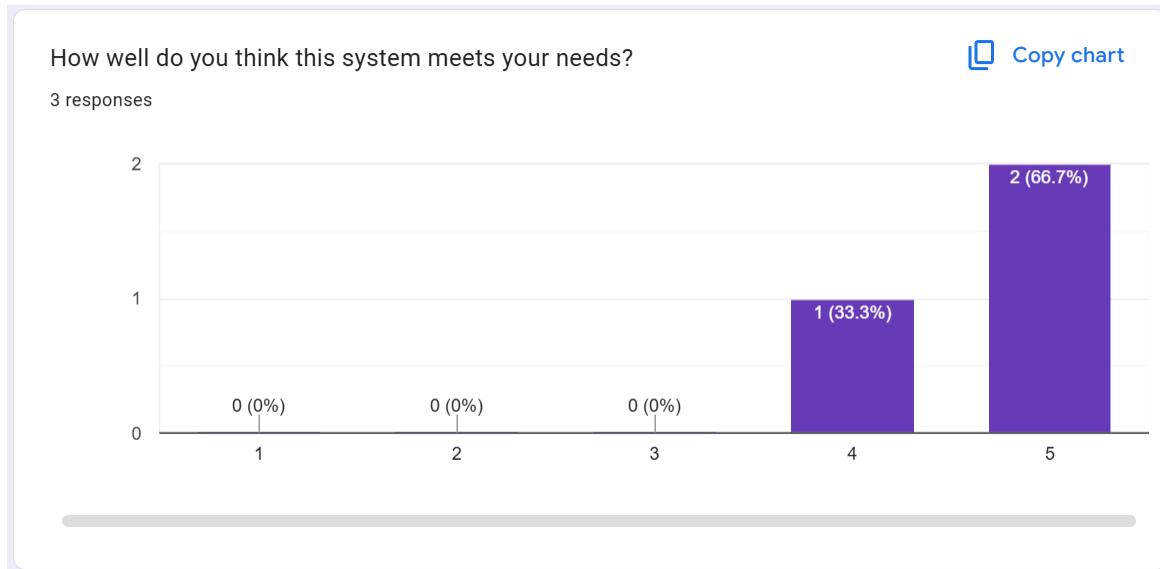


Figure 35: 1(Not at all) - 5 (Perfectly)

Overall, my end-users feedback seemed very positive, with two of my users saying the system fitted their needs perfectly, and the other still rating the system very highly.

Reflecting upon my end-user feedback and testing, there are three areas that I think went very well during this project. The first of these is the general usability of each subsystem within the program, with all my end-users complementing the convenience and presentation of the graphical user interface on many if not all of my webpages. The second area I think this project really excelled in, was the robustness of all the stages of this program. My end-users never once crashed the program or managed to enter any invalid data, with sufficient data validation being implemented at all stages, including the actual game logic. Furthermore, there were also appropriate error messages displayed for almost all of my systems, ensuring the user was aware of what they had done wrong if they inputted invalid data, or made invalid decisions during the games. The final area in which I think my project did well is the overall completeness of the implemented solution, with a really wide range of different convenient features for users, including the leaderboard, profile viewership and game tutorial features. All of these features manage to complement the main gameplay fantastically, ensuring a more enjoyable and well-rounded user experience.

There are also several ways in which this project could be significantly improved. The three areas of the program that could be even better, according to the end-user feedback, are as follows. The first area that could be even better, as mentioned by all of my end-users, would be a slightly more sophisticated registration system, implementing email authentication. In the current day, this kind of feature seems pretty essential to any commercial or publicly available system, so introducing this might have been a useful thing to experiment with in this program. One way this could be implemented, would be by configuring an SMTP server using a service like SendGrid or MailGun, to send verification codes or links. Another area in which this project could be improved would be by further improving visual appeal and user experience through the use of a more sophisticated game interface. As mentioned by one of my end-users, perhaps including some more colour and dice-rolling animations could significantly enhance the gameplay and overall feel of the program. A third way in which this project could be improved would be by implementing a different multiplayer system, whilst still maintaining the pass and play and bot gameplay features. An online perudo game that could be accessed by multiple clients concurrently to play together, would be a natural next step for this program, and might better fulfill certain people's needs, especially if players

are not physically together and do not benefit from the pass and play system as much. This kind of step up could be implemented through the use of pre-existing programs/functionality, such as Django channels, or through the general use and setup of a websocket server for real time online multiplayer gameplay.

5.2 Performance Evaluation

In many respects, I am very happy with my performance throughout this project, especially regarding my ability to complete it within the available time-frame, to a high standard. Unlike many others whom I have observed completing similar projects or Non-Exam Assessments, I have managed to work in a consistent and disciplined way, making sure to give myself enough time to experiment, fine-tune and, most importantly, enjoy this project. However, there were also several unexpected challenges throughout its duration. One major initial challenge was the sheer number of possibilities and decisions I had to make, even when simply choosing what project to pursue. I remember considering many different possible program areas, including a database system, a control system and several others. Even when I had established I wanted to implement one of my favourite board games, I was forced to choose between several competitive options, that I have always loved playing. Another challenge I faced at times was dealing with the, at times, seemingly insurmountable task of debugging algorithms repeatedly for hours. This truly did test my patience and determination, but, by the end of the implementation, had really honed my debugging skills, and enhanced my coding intuition, as I constantly sniffed out errors in my code and solved issues. More generally, in addition to this skill, over the course of this project, my procedural programming skills in python have been thoroughly improved, as well as my SQL and exception handling skills. These areas have not just seen improvements in terms of the quality of my code, but also in terms the efficiency of my code and a reduction in time spent to write various algorithms.

In future projects, I will try and spend even more time during the planning (or documented design) phase, making sure to model or carefully describe any non-trivial part of the program I would be writing. Although I think I planned this project well, If I were to redo it, I would probably still spend even more time doing so, before diving straight into it. Since I started implementing my code pretty soon after my planning, I occasionally lost focus of how the various parts of my code interacted and functioned, and found that looking back on my documented design and even analysis sections, really helped solve many issues I was facing.

Overall, of my 30 core and additional objectives I managed to mostly or completely achieve 26 objectives, giving me a an 87% achievement rate among both core and additional objectives. Of my 22 core objectives alone, I manage to completely achieve all 22, indicating a overwhelming success to meet the core requirements of this project. This, paired with the high satisfaction rates indicated by my end-users in figure 33, have strengthened my sense of success and pride in this project. Consequently, I do think this project has completed what it set out to do and has provided a user-friendly, robust and useful programme, enabling people to play a fun board game with friends or against bots on the go. Furthermore, despite not necessarily implementing every possible feature often seen in similar commercial implementations, I think this project has proven itself to be a convincing proof of concept and strong foundational Perudo Web-app implementation.